

**PENGEMBANGAN SISTEM PEMBANGKITAN
DAN PENYIMPANAN AUDIT LOG PADA SISTEM
RME DECMED**

Laporan Tugas Akhir

Disusun sebagai syarat kelulusan tingkat sarjana

Oleh

Pradipta Rafa Mahesa

NIM: 13522162



**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO & INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

September 2026

**PENGEMBANGAN SISTEM PEMBANGKITAN
DAN PENYIMPANAN AUDIT LOG PADA SISTEM
RME DECMED**

Laporan Tugas Akhir

Oleh

Pradipta Rafa Mahesa

NIM: 13522162

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Telah disetujui dan disahkan sebagai Laporan Tugas Akhir
di Bandung, pada tanggal 4 September 2026

Pembimbing,

Dr. Phil. Eng. Hari Purnama, S.Si., M.Si

NIP. 118110072

LEMBAR PERNYATAAN

Dengan ini saya menyatakan bahwa:

1. Pengerjaan dan penulisan Laporan Tugas Akhir ini dilakukan tanpa menggunakan bantuan yang tidak dibenarkan.
2. Segala bentuk kutipan dan acuan terhadap tulisan orang lain yang digunakan di dalam penyusunan laporan tugas akhir ini telah dituliskan dengan baik dan benar.
3. Laporan Tugas Akhir ini belum pernah diajukan pada program pendidikan di perguruan tinggi mana pun.

Jika terbukti melanggar hal-hal di atas, saya bersedia dikenakan sanksi sesuai dengan Peraturan Akademik dan Kemahasiswaan Institut Teknologi Bandung bagian Penegakan Norma Akademik dan Kemahasiswaan khususnya Pasal 2.1 dan Pasal 2.2.

Bandung, 4 September 2026

Pradipta Rafa Mahesa

NIM 13522162

ABSTRAK

Rekam Medis Elektronik (RME) merupakan komponen fundamental dalam penyelenggaraan pelayanan kesehatan di Indonesia. Peraturan Menteri Kesehatan Nomor 24 Tahun 2022 mewajibkan seluruh fasilitas pelayanan kesehatan untuk menyelenggarakan RME dengan menjamin kerahasiaan, keutuhan, dan ketersediaan data. Untuk memenuhi kebutuhan tersebut, penelitian sebelumnya mengembangkan DecMed, sebuah kerangka kerja manajemen RME terdesentralisasi berbasis DLT IOTA, *Proxy Re-Encryption* (PRE), dan IPFS. Meskipun DecMed berhasil menyelesaikan persoalan kontrol akses, mekanisme pencatatan aktivitas sistem yang ada belum membentuk audit log yang memadai: pencatatan masih tersebar pada masing-masing komponen tanpa pengelola terpusat, cakupan *event* masih sempit, catatan lama dapat tertimpa sehingga melanggar prinsip *append-only*, format belum tersandarisasi, dan bukti digital rentan terhapus akibat mekanisme *data pruning* pada jaringan DLT.

Tugas Akhir ini mengembangkan *Audit Log Service* (ALS), sebuah layanan *backend* independen yang terintegrasi dengan arsitektur DecMed. Perancangan dimulai dengan analisis ancaman menggunakan metode STRIDE terhadap arsitektur DecMed, menghasilkan 25 ancaman yang dipetakan menjadi 13 jenis *audit event* (EV1–EV13). Setiap *event source* (*Patient Client*, *Hospital Client*, *Ministry Client*, dan PRE Server) mengirimkan *audit event* ke ALS dalam format *EncryptedSignedEvent*: *payload event* dienkripsi menggunakan AES-256-GCM dengan kunci sesi acak, kunci sesi dibungkus menggunakan ECIES P-256 dengan kunci publik ALS, dan *ciphertext* ditandatangani menggunakan *IotaKeyPair* aktor yang terlibat (pola *Encrypt-then-Sign*). Sebelum pengiriman, *event* yang sudah dienkripsi dan ditandatangani terlebih dahulu disimpan ke antrian berbasis berkas (*AlsQueue*) untuk menjamin ketercapaian pengiriman meskipun terjadi kegagalan jaringan sementara. ALS memverifikasi tanda tangan *IotaSignature* atas *ciphertext*, mendekripsi *payload*, kemudian membentuk *AuditRecord* terstruktur yang memuat dua nilai waktu secara terpisah: *source_timestamp* yang mencatat waktu kejadian menurut jam *event source*, dan *timestamp* yang mencatat waktu pembentukan *record* di sisi ALS. Mekanisme *hash-chaining* SHA-256 diterapkan antarrecord sehingga setiap *AuditRecord* terikat secara kriptografis pada seluruh rekam sebelumnya dan perubahan apa pun pada posisi mana pun dalam rantai dapat terdeteksi. Secara periodik, ALS merotasi berkas log aktif, mengunggah berkas arsip ke IPFS Cluster dengan mekanisme *pinning* agar tidak terhapus oleh *Garbage Collection*, kemudian menerbitkan metadata rotasi sebagai objek yang dibekukan (*frozen object*) pada *smart contract* Move di jaringan IOTA melalui Gas Station Server yang sudah tersedia pada DecMed.

Hasil implementasi dan pengujian menunjukkan bahwa ALS berhasil mengagregasi *audit event* dari seluruh komponen DecMed ke dalam satu repositori terpusat, merekam 13 jenis *event* yang mencakup seluruh ancaman STRIDE yang teridentifikasi, dan mendukung deteksi kerusakan rekam jejak melalui mekanisme *hash-chaining* pada level *audit record*. Pengujian siklus lengkap (P-INT-01) membuktikan bahwa dari satu *record* EV4 (*Medical Record Access*), sistem dapat membuktikan siapa mengakses, data apa, kapan, dan dengan

otorisasi apa; kerusakan pada posisi mana pun dalam rantai terdeteksi melalui efek domino nilai *hash*; serta berkas log yang telah diarsipkan dapat dipulihkan kembali secara identik dari IPFS. Retensi data jangka panjang dijamin melalui kombinasi *IPFS pinning* dan penyimpanan metadata *on-chain* yang bersifat *immutable*. Terdapat tiga batasan yang perlu dicatat: isi *event* yang tersimpan memerlukan kunci privat ALS untuk didekripsi, fasilitas pencarian dan penyaringan lintas-*record* belum tersedia, dan *non-repudiation* berbasis *IotaKeyPair* berlaku selama kunci aktor tidak dikompromikan dan belum merotasi kunci antara waktu kejadian dan waktu investigasi. ALS tidak mengubah alur bisnis maupun fungsionalitas utama DecMed yang telah berjalan, melainkan memanfaatkan kembali infrastruktur IPFS dan IOTA yang sudah tersedia.

Kata kunci: audit log, DecMed, IOTA, IPFS, hash-chaining, enkripsi end-to-end, ECI-ES, tanda tangan digital, non-repudiation, STRIDE, rekam medis elektronik

KATA PENGANTAR

Puji dan syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas berkat dan rahmatnya, laporan tugas akhir yang berjudul "Pengembangan Sistem Pembangkitan dan Penyimpanan Audit Log pada Sistem RME DecMed " dapat diselesaikan dalam rangka memenuhi syarat kelulusan tingkat sarjana. Perlu diakui pengerjaan tugas akhir ini didukung oleh banyak pihak. Khususnya, penulis ingin mengucapkan terima kasih kepada:

1. Bapak Dr. Phil. Eng. Hari Purnama, S.Si., M.Si, selaku dosen pembimbing atas segala bentuk dukungan yang telah diberikan dan kesabarannya dalam membimbing penulis serta memberikan saran dalam pengerjaan tugas akhir.
2. Bapak Yudistira Dwi Wardhana Asnar, S.T, Ph.D dan Dr. Agung Dewandaru, S.T., M.Sc., selaku dosen penguji atas segala masukan dan kritik yang telah diberikan terhadap tugas akhir penulis.
3. Dicky Prima Satya, S.T, M.T., Bapak Adi Mulyanto, S.T, M.T., Robithoh Annur, S.T., M.Eng., Ph.D., dan Tricya Esterina Widagdo, ST., M.Sc. selaku dosen koordinator tim tugas akhir atas usahanya mengingatkan mahasiswa program studi Teknik Informatika untuk mengerjakan tugas akhirnya.
4. Seluruh dosen program studi Teknik Informatika ITB yang telah memberikan ilmu pengetahuan yang sangat berharga bagi penulis.
5. Keluarga penulis, terutama orang tua dan kakak penulis, yang senantiasa memberikan doa, dukungan moral, serta dukungan lainnya selama penulis menjalani masa perkuliahan hingga menyelesaikan Tugas Akhir ini.
6. Teman-teman dan rekan seperjuangan mahasiswa bimbingan Bapak Dr. Phil. Eng. Hari Purnama, S.Si., M.Si terutamanya Grup DecMed loh dek, yaitu Bagas dan Tazki, sebagai teman perjuangan selama pengembangan DecMed.

7. Sahabat-sahabat terdekat dari grup "Keluarga Acu", dan "mrt gym" yang telah memberikan bantuan, semangat, serta menjadi tempat berdiskusi dalam menghadapi berbagai proses akademik maupun nonakademik selama menempuh pendidikan di Teknik Informatika ITB.
8. Seluruh pihak lain yang tidak bisa disebutkan disini yang telah membantu dalam proses pengerjaan tugas akhir.

Akhir kata, penulis mengucapkan terima kasih kepada semua pihak yang telah terlibat dalam pengerjaan tugas akhir ini. Penulis juga ingin menyampaikan mohon maaf apabila terdapat kesalahan maupun kekurangan dalam laporan tugas akhir ini. Penulis berharap semoga tugas akhir ini dapat bermanfaat bagi pembaca dan riset-riset kedepannya.

Bandung, 4 September 2026

Pradipta Rafa Mahesa

DAFTAR ISI

ABSTRAK	iv
KATA PENGANTAR.....	vi
DAFTAR ISI.....	viii
DAFTAR LAMPIRAN.....	xiv
DAFTAR GAMBAR.....	xv
DAFTAR TABEL	xvi
BAB I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Tujuan dan Ukuran Keberhasilan Pencapaian	3
I.4 Batasan Masalah	4
I.5 Metodologi	4
I.6 Sistematika Pembahasan.....	5
BAB II STUDI LITERATUR	6
II.1 Rekam Medis Elektronik	6
II.1.1 Definisi dan Regulasi RME di Indonesia	6
II.1.2 Kebutuhan Keamanan dan Akuntabilitas Data RME.....	7
II.2 Audit Log.....	8
II.2.1 Definisi dan Konsep Audit Log	8
II.2.2 Karakteristik Audit Log yang Baik.....	9
II.2.3 Komponen Audit Log	10
II.2.3.1 Penentuan Event	10
II.2.3.2 Konten Wajib dan Konten Tambahan pada Audit Record.....	11
II.2.3.3 Pengumpulan Event	11
II.2.3.4 Penyimpanan Event.....	12
II.2.3.5 Pembacaan Event	12
II.2.4 Kebutuhan Audit Log	12
II.2.4.1 Kebutuhan Fungsional	13
II.2.4.2 Kebutuhan Keamanan	13

II.2.5	Mekanisme <i>Tamper-Evident</i> pada Audit Log	14
II.2.5.1	<i>Hash-Chaining</i> untuk Integritas Berurutan	15
II.2.5.2	Tanda Tangan Digital untuk Keaslian dan Non-Repudiation	15
II.2.5.3	Rotasi dan Pengarsipan Log	16
II.3	Keamanan Informasi dan Akuntabilitas	16
II.3.1	Prinsip Dasar Keamanan Informasi	16
II.3.2	Properti Keamanan yang Diperluas	17
II.3.3	Keterkaitan antara Properti Keamanan dan Audit Log	19
II.4	Kerja DecMed	20
II.4.1	Gambaran Umum dan Latar Belakang DecMed	20
II.4.2	Mekanisme Kontrol Akses pada DecMed	21
II.4.3	Keterbatasan Audit Log pada DecMed	22
II.5	Threat Modeling dengan STRIDE	23
II.5.1	Konsep dan Kategori STRIDE	23
II.5.2	Dekomposisi Sistem dan Data Flow Diagram dalam Threat Modeling	24
II.5.3	Pendekatan STRIDE-per-Interaction	25
II.5.4	Penurunan Kebutuhan Audit Event dari Analisis STRIDE ...	25
II.6	Distributed Ledger Technology dan Immutability	26
II.6.1	DLT sebagai Basis Pencatatan Tidak Berubah	26
II.6.2	IOTA	27
II.7	Arsitektur Layanan Pendukung dalam Sistem Terdesentralisasi	27
II.7.1	Pola Layanan Terpisah dalam Arsitektur DecMed	27
II.7.2	Proxy Re-Encryption dan IPFS	28
II.8	Penelitian Terkait	29
BAB III	ANALISIS DAN RANCANGAN SOLUSI	30
III.1	Analisis Masalah	30
III.1.1	Analisis Sistem DecMed	30
III.1.1.1	Implementasi Audit Log pada DecMed	30
III.1.1.2	Keterbatasan <i>PatientAccessLog</i>	31
III.1.1.3	Keterbatasan <i>Transaction Block</i> IOTA	31
III.1.1.4	Keterbatasan Pencatatan Audit Log yang Tersebar pada DecMed	32

III.1.2	Identifikasi Masalah.....	33
III.1.3	Kebutuhan <i>Audit</i>	34
III.1.3.1	Dekomposisi Aplikasi	34
III.1.3.2	Identifikasi Ancaman DecMed	35
III.1.3.3	Identifikasi Audit Event	35
III.1.4	Kebutuhan Sistem	38
III.1.4.1	Kebutuhan Fungsional	39
III.1.4.2	Kebutuhan Nonfungsional.....	39
III.1.4.3	Pemetaan Masalah terhadap Kebutuhan Sistem	40
III.1.5	Analisis Solusi	40
III.1.5.1	Analisis <i>Event Source</i>	41
III.1.5.2	Analisis Arsitektur Agregasi dan Pengumpulan Log	41
III.1.5.3	Analisis Mekanisme Pengiriman Log	42
III.1.5.4	Analisis Keaslian dan Pencegahan <i>Event</i> Palsu	43
III.1.5.5	Analisis Kerahasiaan Log dalam Transit	43
III.1.5.6	Analisis Standardisasi Skema <i>Audit Record</i>	44
III.1.5.7	Analisis Repositori Audit Log	45
III.1.5.8	Analisis Mekanisme Penyimpanan Berkas Log	46
III.1.5.9	Analisis Jaminan Integritas Berkas Log Lokal (<i>Hash Chaining</i>)	47
III.1.5.10	Analisis Strategi Persistensi Data terhadap Risiko <i>Pruning</i>	48
III.1.5.11	Pemetaan Masalah terhadap Solusi	49
III.2	Rancangan	49
III.2.1	Rancangan Arsitektur Solusi	50
III.2.2	Rancangan Skema Audit Record	52
III.2.2.1	Atribut Umum Audit Record	52
III.2.2.2	Atribut Khusus Audit Record	54
III.2.2.3	Rancangan Skema Terperinci AuditEvent dan Enum Atribut	54
III.2.3	Rancangan Spesifikasi <i>Audit Event</i>	56
III.2.3.1	EV1 — Authentication	57
III.2.3.2	EV2 — QR Validation.....	59
III.2.3.3	EV3 — Capability Creation	60

III.2.3.4	EV4 — Medical Record Access	62
III.2.3.5	EV5 — Personnel Activation Key	63
III.2.3.6	EV6 — Facility Registration	63
III.2.3.7	EV7 — PRE Service Operation.....	64
III.2.3.8	EV8 — IOTA Transaction Submission	64
III.2.3.9	EV9 — Gas Sponsorship Request	67
III.2.3.10	EV10 — Redis Operation	68
III.2.3.11	EV11 — IPFS Operation	68
III.2.3.12	EV12 — IOTA Metadata Operation	69
III.2.3.13	EV13 — Capability Validation.....	69
III.2.4	Rancangan Mekanisme Pembangkitan dan Penerimaan Audit Event	71
III.2.4.1	Rancangan Enkapsulasi Audit Event	72
III.2.4.2	Rancangan Mekanisme Pengiriman	73
III.2.4.3	Rancangan Antrian Pengiriman Berbasis Berkas (AlsQueue).....	74
III.2.5	Rancangan Mekanisme Pembentukan dan Penyimpanan Audit Record.....	76
III.2.6	Rancangan Mekanisme Rotasi Berkas Log	77
III.2.7	Rancangan Mekanisme Publikasi Metadata <i>On-Chain</i> ke IOTA	78
III.2.7.1	Skema Metadata Rotasi Log	78
III.2.7.2	Penerbitan Metadata melalui <i>Smart Contract Move</i>	80
III.2.7.3	Rantai Metadata <i>On-Chain</i>	80
III.2.8	Pemetaan Rancangan terhadap Solusi.....	81
BAB IV	IMPLEMENTASI DAN PENGUJIAN.....	85
IV.1	Lingkungan Implementasi	85
IV.2	Implementasi Sistem.....	86
IV.2.1	Batasan Implementasi.....	86
IV.2.2	Implementasi Skema Tipe Data Audit	86
IV.2.2.1	Tipe Data EncryptedSignedEvent.....	89
IV.2.2.2	Tipe Data AuditEvent dan Event.....	90
IV.2.2.3	Enum AuditOutcome dan AuditSourceComponent	90
IV.2.2.4	Enum AuditActionType	91

IV.2.2.5	Enum AuditActorType dan AuditTargetObjectType	91
IV.2.2.6	Tipe Data AuditEventDetails.....	93
IV.2.2.7	Tipe Data AuditRecord	94
IV.2.2.8	Tipe Data AuditBatch	95
IV.2.3	Implementasi Mekanisme Enkapsulasi Event pada Event Source	95
IV.2.4	Implementasi AlsQueue dan Retry Worker	96
IV.2.5	Implementasi Audit Receiver	97
IV.2.6	Implementasi Audit Logger	98
IV.2.7	Implementasi Log Rotation dan Archival Worker	99
IV.2.8	Implementasi Smart Contract Move	100
IV.2.9	Implementasi Integrasi ALS ke DecMed	100
IV.2.9.1	Tipe Data Bersama	101
IV.2.9.2	Implementasi ALSClient	101
IV.2.9.3	Integrasi ke State Komponen	102
IV.2.9.4	Distribusi Event per Event Source	102
IV.3	Pengujian	102
IV.3.1	Strategi Pengujian	102
IV.3.2	Pengujian Kebutuhan Fungsional	103
IV.3.2.1	P-KF-01: Penerimaan dan Agregasi <i>Audit Event</i> dari Berbagai <i>Event Source</i>	103
IV.3.2.2	P-KF-02: Pencatatan Keseluruhan Jenis <i>Audit Event</i> EV1–EV13	103
IV.3.2.3	P-KF-03: Standardisasi Format <i>Audit Record</i>	104
IV.3.2.4	P-KF-04: Penyimpanan <i>Audit Record</i> secara <i>Append-Only</i>	104
IV.3.2.5	P-KF-05: Retensi Jangka Panjang melalui Rotasi Log, Pengarsipan IPFS, dan Publikasi Metadata IOTA	105
IV.3.3	Pengujian End-to-end Audit Log	105
IV.3.3.1	Langkah 1: Event yang Terbentuk	106
IV.3.3.2	Langkah 2: Record yang Tersimpan	107
IV.3.3.3	Langkah 3: Hubungan Antar-Record dan Rantai Hash	108

IV.3.3.4	Langkah 4: Verifikasi Integritas dan Deteksi Per-	
	usakan	108
IV.3.3.5	Langkah 5: Kesimpulan dari Satu Record	109
IV.3.3.6	Langkah 6: Batas Kesimpulan	109
IV.3.4	Pengujian Kebutuhan Nonfungsional	110
IV.3.4.1	Pengujian KNF-TA-01: Integritas <i>Audit Record</i>	
	yang <i>Tamper-Proof</i>	111
IV.3.4.2	Pengujian KNF-TA-02: Interoperabilitas dan Ke-	
	terbacaan Format Data Log	111
IV.3.4.3	Pengujian KNF-TA-03: Ketersediaan Data <i>Audit</i>	
	<i>Log</i> dalam Jangka Panjang	112
IV.3.4.4	Pengujian KNF-TA-04: Konsistensi Pencatatan	
	melalui Verifikasi Tanda Tangan Digital.....	113
IV.4	Evaluasi.....	113
IV.4.1	Evaluasi terhadap Kebutuhan Sistem	114
IV.4.2	Keterbatasan Implementasi	116
BAB V	PENUTUP.....	119
V.1	Kesimpulan.....	119
V.2	Saran.....	120
DAFTAR PUSTAKA		122

DAFTAR LAMPIRAN

Lampiran A.	Dekomposisi Aplikasi DecMed	123
Lampiran B.	Hasil Analisis Ancaman STRIDE	131
Lampiran C.	Atribut Tambahan Audit Event	134
Lampiran D.	Pemetaan Audit Event ke Event Source	137
Lampiran E.	Implementasi Tipe AuditEventDetails	139
Lampiran F.	Kode Pengujian	142
F.1	Fungsi Pendukung Pengujian.....	142

DAFTAR GAMBAR

Gambar II.1.	Hierarki Konsep Audit Menurut ISO 27789:2021	8
Gambar III.1.	Data Flow Diagram Sistem DecMed.....	35
Gambar III.2.	Rancangan Arsitektur Solusi Audit Log DecMed	51
Gambar III.3.	Skema Rantai Metadata Rotasi Log <i>On-Chain</i> pada IOTA	81

DAFTAR TABEL

Tabel II.1.	Keterkaitan Properti Keamanan dengan Mekanisme Audit Log....	20
Tabel II.2.	Kategori Ancaman STRIDE dan Properti Keamanan yang Di- langgar	23
Tabel II.3.	Relevansi Kategori STRIDE terhadap Jenis Elemen DFD	25
Tabel III.1.	Identifikasi Masalah Audit Log DecMed	34
Tabel III.2.	Pemetaan Ancaman ke Audit Event.....	36
Tabel III.3.	Kebutuhan Fungsional Sistem Audit Log	39
Tabel III.4.	Kebutuhan Nonfungsional Sistem Audit Log	40
Tabel III.5.	Pemetaan Masalah terhadap Kebutuhan Sistem Audit Log	40
Tabel III.6.	Distribusi <i>Audit Event</i> Berdasarkan <i>Source</i>	41
Tabel III.7.	Analisis Pendekatan Arsitektur Pengumpulan Log	42
Tabel III.8.	Analisis Pendekatan Penyimpanan Audit Log	46
Tabel III.9.	Daftar Solusi yang Diusulkan Berdasarkan Hasil Analisis.....	50
Tabel III.10.	Pemetaan Kebutuhan Bukti Umum ke Atribut Audit Record	53
Tabel III.11.	Varian <i>AuditSourceComponent</i>	55
Tabel III.12.	Varian <i>AuditActionType</i>	56
Tabel III.13.	Varian <i>AuditActorType</i>	56
Tabel III.14.	Varian <i>AuditTargetObjectType</i>	57
Tabel III.15.	Kemungkinan Nilai Atribut <i>outcome</i>	57
Tabel III.16.	Kemungkinan Nilai Atribut <i>action_type</i>	58
Tabel III.17.	Atribut Umum EV1 Authentication	59
Tabel III.18.	Atribut Tambahan EV1 Authentication	59
Tabel III.19.	Atribut Umum EV2 QR Validation.....	60
Tabel III.20.	Atribut Tambahan EV2 QR Validation	60
Tabel III.21.	Atribut Umum EV3 Capability Creation.....	61
Tabel III.22.	Atribut Tambahan EV3 Capability Creation.....	61
Tabel III.23.	Atribut Umum EV4 Medical Record Access	62
Tabel III.24.	Atribut Tambahan EV4 Medical Record Access	62
Tabel III.25.	Atribut Umum EV5 Personnel Activation Key	63

Tabel III.26.	Atribut Tambahan EV5 Personnel Activation Key	63
Tabel III.27.	Atribut Umum EV6 Facility Registration	64
Tabel III.28.	Atribut Tambahan EV6 Facility Registration	64
Tabel III.29.	Atribut Umum EV7 PRE Service Operation.....	65
Tabel III.30.	Atribut Tambahan EV7 PRE Service Operation.....	65
Tabel III.31.	Atribut Umum EV8 IOTA Transaction Submission.....	66
Tabel III.32.	Atribut Tambahan EV8 IOTA Transaction Submission.....	66
Tabel III.33.	Atribut Umum EV9 Gas Sponsorship Request	67
Tabel III.34.	Atribut Tambahan EV9 Gas Sponsorship Request	67
Tabel III.35.	Atribut Umum EV10 Redis Operation	68
Tabel III.36.	Atribut Tambahan EV10 Redis Operation	68
Tabel III.37.	Atribut Umum EV11 IPFS Operation	69
Tabel III.38.	Atribut Tambahan EV11 IPFS Operation	69
Tabel III.39.	Atribut Umum EV12 IOTA Metadata Operation	70
Tabel III.40.	Atribut Tambahan EV12 IOTA Metadata Operation	70
Tabel III.41.	Atribut Umum EV13 Capability Validation	71
Tabel III.42.	Atribut Tambahan EV13 Capability Validation.....	71
Tabel III.43.	Skema Metadata Rotasi Log pada <i>Smart Contract</i> IOTA.....	79
Tabel III.45.	Pemetaan Kebutuhan ke Mekanisme Rancangan ALS.....	81
Tabel III.44.	Pemetaan Rancangan terhadap Solusi	84
Tabel IV.1.	Daftar Implementasi Sistem Audit Log.....	87
Tabel IV.2.	Dependensi Utama Audit Log Service.....	88
Tabel IV.3.	Status Implementasi Komponen Audit Receiver	98
Tabel IV.4.	Evaluasi Pemenuhan Kebutuhan Sistem <i>Audit Log</i>	114
Tabel A.1.	Dependensi Eksternal Sistem DecMed	123
Tabel A.2.	Titik Masuk Sistem DecMed	124
Tabel A.3.	Aset Sistem DecMed.....	128
Tabel A.4.	Level Kepercayaan Sistem DecMed	129
Tabel B.1.	Identifikasi Ancaman Sistem DecMed.....	131
Tabel C.1.	Atribut Audit Tambahan untuk Setiap Audit Event.....	134

Tabel D.1. Daftar Identifikasi <i>Audit Event</i> Sistem DecMed	137
---	-----

BAB I

PENDAHULUAN

I.1 Latar Belakang

Rekam Medis Elektronik (RME) merupakan komponen fundamental dalam penyelenggaraan pelayanan kesehatan di Indonesia. Peraturan Menteri Kesehatan Republik Indonesia (Permenkes) Nomor 24 Tahun 2022 tentang Rekam Medis mewajibkan seluruh fasilitas pelayanan kesehatan (fasyankes) untuk menyelenggarakan rekam medis dalam bentuk elektronik, serta mengharuskan penyelenggaraan tersebut menjamin kerahasiaan, keutuhan (integritas), dan ketersediaan data rekam medis (Kementerian Kesehatan Republik Indonesia, 2022). Tinjauan sistematis terhadap implementasi RME turut menunjukkan bahwa keamanan dan kerahasiaan data medis pasien menjadi perhatian utama dalam berbagai studi, mengingat data tersebut bersifat sensitif dan berdampak langsung pada kepercayaan pasien terhadap sistem pelayanan kesehatan (Pramesti dkk., 2024). Kewajiban ini menegaskan bahwa keamanan data RME bukan sekadar kebutuhan teknis, melainkan juga kepatuhan regulasi yang mengikat bagi seluruh penyelenggara pelayanan kesehatan.

Untuk menjawab tantangan tersebut, penelitian sebelumnya mengembangkan DecMed, sebuah kerangka kerja manajemen RME terdesentralisasi yang menggunakan Distributed Ledger Technology (DLT) IOTA sebagai basis kontrol akses (Purnama dkk., 2026). DecMed mengintegrasikan Capability-Based Access Control (CapBAC), Proxy Re-Encryption (PRE), dan InterPlanetary File System (IPFS) (Benet, 2014) untuk memastikan bahwa setiap akses terhadap data RME hanya dapat dilakukan atas persetujuan aktif pasien, dengan durasi dan cakupan akses yang dapat dibatasi oleh pasien sendiri (Purnama dkk., 2026).

Meskipun DecMed berhasil menjawab persoalan kontrol akses dan kerahasiaan data, penelitian tersebut secara eksplisit menyatakan bahwa pengelolaan RME juga membutuhkan *audit log* yang andal dan dapat diverifikasi yang mensyaratkan adanya audit log yang mampu merekam seluruh aktivitas sistem secara kronologis, karena pencatatan log merupakan komponen inti dari kontrol audit dan umumnya

disyaratkan untuk mendukung kepatuhan regulasi maupun investigasi insiden (Purnama dkk., 2026). Klaim ini sejalan dengan kerangka kontrol audit pada NIST Special Publication 800-53 Revisi 5, yang menempatkan pencatatan dan peninjauan peristiwa keamanan (kontrol keluarga *Audit and Accountability*) sebagai salah satu persyaratan dasar tata kelola keamanan sistem informasi (National Institute of Standards and Technology, 2020), sejalan pula dengan kontrol *logging* pada ISO/IEC 27002:2022 (International Organization for Standardization, 2022). Catatan audit yang lengkap juga berperan penting sebagai sumber data untuk mendeteksi dan memantau potensi ancaman dalam layanan kesehatan.

Namun demikian, mekanisme pencatatan yang tersedia pada DecMed saat ini belum sepenuhnya memenuhi karakteristik audit log yang baik. Terdapat dua celah utama yang teridentifikasi. *Pertama*, sebagian besar mekanisme yang tercatat pada jaringan IOTA merupakan entri *capability* atau izin akses (*access grant*), yaitu metadata yang menyatakan siapa berhak mengakses data apa dan sampai kapan, bukan catatan atas peristiwa akses yang benar-benar terjadi (*access event*). Akibatnya, sistem belum dapat merekonstruksi riwayat siapa yang benar-benar membaca atau mengubah data RME pada waktu tertentu. *Kedua*, sejumlah operasi pada DecMed, seperti pencabutan akses (*revoke_access*) dan pembaruan data administratif maupun klinis (*update_administrative_data*, *update_medical_record*), menyebabkan entri lama pada state tergantikan oleh entri baru alih-alih ditambahkan sebagai riwayat baru yang bersifat *append-only*. Kondisi ini bertentangan dengan karakteristik dasar audit log yang baik, yaitu integritas kronologis dan kelengkapan pencatatan, yang mensyaratkan bahwa setiap catatan peristiwa harus bersifat permanen dan *append-only*, perubahan kondisi dicatat sebagai entri baru, bukan pembaruan entri lama, sehingga riwayat tindakan dapat diverifikasi keutuhannya (International Organization for Standardization, 2021, National Institute of Standards and Technology, 2020).

Celah tersebut menjadi lebih krusial apabila ditinjau dari perspektif ancaman keamanan sistem. Metode STRIDE (*Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege*) merupakan salah satu metode pemodelan ancaman yang umum digunakan untuk mengidentifikasi risiko keamanan pada suatu sistem informasi secara sistematis, yang kemudian dapat dilanjutkan dengan penilaian risiko menggunakan model DREAD (Laksono dan Prayudi,

2021). Salah satu kategori ancaman pada STRIDE, yaitu *repudiation*, merupakan kondisi ketika pengguna sistem dapat menyangkal bahwa ia telah melakukan suatu tindakan tertentu; ancaman ini secara langsung berkaitan dengan ketiadaan atau ketidaklengkapan pencatatan log dan mekanisme audit pada suatu sistem (Laksono dan Prayudi, 2021). Dengan kata lain, kebutuhan pencatatan peristiwa (*event*) pada suatu sistem idealnya tidak ditentukan secara ad hoc, melainkan diturunkan dari analisis ancaman yang relevan terhadap arsitektur sistem yang bersangkutan.

Berdasarkan pemaparan tersebut, Tugas Akhir ini bermaksud menganalisis kebutuhan audit log pada sistem DecMed melalui pendekatan pemodelan ancaman STRIDE, guna menentukan jenis peristiwa (*event*) apa saja yang perlu dicatat agar sistem memiliki audit log yang memenuhi karakteristik integritas dan non-repudiation. Hasil analisis tersebut selanjutnya digunakan sebagai dasar perancangan dan implementasi sebuah layanan audit log (*audit log service*) yang terintegrasi dengan arsitektur DecMed yang telah ada, mengikuti pola layanan pendukung (*supporting service*) yang serupa dengan *proxy re-encryption server* yang sudah digunakan pada DecMed (Purnama dkk., 2026). Metadata dari setiap peristiwa yang tercatat akan disimpan pada jaringan IOTA untuk menjamin integritas dan keterlacakannya, sedangkan berkas log peristiwa akan disimpan pada IPFS (Benet, 2014), konsisten dengan pola pemisahan penyimpanan on-chain dan off-chain yang telah diterapkan pada DecMed untuk data RME (Purnama dkk., 2026).

I.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan pada Subbab I.1, terdapat dua rumusan masalah utama dalam Tugas Akhir ini, yaitu

1. Peristiwa (*event*) apa saja yang perlu dicatat oleh mekanisme audit log pada sistem DecMed?
2. Bagaimana rancangan dan implementasi layanan audit log yang tamper-evident dan sesuai dengan arsitektur DecMed?

I.3 Tujuan dan Ukuran Keberhasilan Pencapaian

Tujuan dari Tugas Akhir ini adalah

1. Menghasilkan daftar kebutuhan peristiwa (*event requirements*) audit log pada sistem DecMed berdasarkan analisis ancaman menggunakan metode STRIDE.
2. Merancang dan mengimplementasikan layanan audit log yang terintegrasi dengan arsitektur DecMed.

Ukuran keberhasilan pencapaian Tugas Akhir ini didasarkan pada kesesuaian daftar kebutuhan *event* yang dihasilkan terhadap kategori ancaman STRIDE yang teridentifikasi, serta keberhasilan layanan audit log yang dikembangkan dalam mencatat, menyimpan, dan menyajikan kembali peristiwa-peristiwa tersebut secara benar dan tidak dapat diubah (*immutable*).

I.4 Batasan Masalah

Terdapat beberapa batasan masalah dari Tugas Akhir ini, antara lain

1. Analisis ancaman menggunakan STRIDE dilakukan terhadap arsitektur sistem DecMed yang sudah ada, bukan terhadap perancangan ulang sistem DecMed secara keseluruhan;
2. Layanan audit log yang dikembangkan bersifat pencatat pasif (*passive recorder*) yang menerima dan menyimpan peristiwa, tanpa melakukan validasi otorisasi maupun keputusan kontrol akses terhadap peristiwa yang dicatat; dan
3. Implementasi dilakukan pada lingkungan pengembangan yang sama dengan yang digunakan pada penelitian DecMed sebelumnya.

I.5 Metodologi

Metodologi yang digunakan dalam Tugas Akhir ini terdiri dari empat tahap pelaksanaan, sebagai berikut.

1. **Tahap Perencanaan.** Tahap ini berfokus pada pemahaman arsitektur dan mekanisme sistem DecMed yang telah ada, kajian literatur terkait audit log, threat modeling STRIDE, serta penelitian terkait lainnya sebagai dasar analisis.

2. **Tahap Analisis Ancaman.** Pada tahap ini dilakukan identifikasi aset, aliran data, dan batas kepercayaan (*trust boundary*) pada arsitektur DecMed, dilanjutkan dengan penerapan metode STRIDE untuk mengidentifikasi ancaman pada setiap komponen. Hasil identifikasi ancaman tersebut kemudian diturunkan menjadi daftar kebutuhan peristiwa (*event requirements*) yang perlu dicatat oleh audit log.
3. **Tahap Perancangan dan Implementasi.** Berdasarkan daftar kebutuhan *event* yang telah ditentukan, dilakukan perancangan arsitektur layanan audit log beserta skema penyimpanan metadata pada IOTA dan berkas log pada IPFS. Selanjutnya, rancangan tersebut diimplementasikan sebagai layanan yang terintegrasi dengan komponen-komponen DecMed yang sudah ada.
4. **Tahap Pengujian dan Evaluasi.** Tahap terakhir bertujuan menguji dan mengevaluasi apakah layanan audit log yang dikembangkan berhasil mencatat seluruh *event* yang telah didefinisikan, serta memvalidasi bahwa data yang tercatat memenuhi karakteristik integritas dan non-repudiation yang diharapkan.

I.6 Sistematika Pembahasan

Berikut merupakan sistematika pembahasan dari Tugas Akhir ini.

Bab I menjelaskan gagasan utama dari Tugas Akhir ini yang meliputi latar belakang, rumusan masalah, tujuan, batasan, metodologi, hingga sistematika pembahasan.

Bab II memaparkan studi literatur yang dilakukan, mencakup RME, audit log, threat modeling STRIDE, kontrol akses, DLT dan IOTA, serta teknologi pendukung lain yang digunakan pada DecMed.

Bab III menjelaskan analisis ancaman menggunakan STRIDE terhadap sistem DecMed serta rancangan solusi layanan audit log yang diusulkan.

Bab IV menjelaskan hasil implementasi dari rancangan yang dipaparkan pada Bab III, beserta mekanisme dan hasil pengujian yang dilakukan.

Bab V merupakan bab penutup yang berisi kesimpulan dan saran dari penulis.

BAB II

STUDI LITERATUR

II.1 Rekam Medis Elektronik

II.1.1 Definisi dan Regulasi RME di Indonesia

Sesuai dengan Pasal 1 ayat (1) dan (2) Peraturan Menteri Kesehatan Republik Indonesia Nomor 24 Tahun 2022 tentang Rekam Medis, rekam medis adalah dokumen yang berisikan data identitas pasien, pemeriksaan, pengobatan, tindakan, dan pelayanan lain yang telah diberikan kepada pasien, sedangkan Rekam Medis Elektronik (RME) adalah rekam medis yang dibuat dengan menggunakan sistem elektronik yang diperuntukkan bagi penyelenggaraan rekam medis (Kementerian Kesehatan Republik Indonesia, 2022). Berdasarkan definisi tersebut, RME bukan sekadar bentuk digital dari berkas rekam medis konvensional, melainkan mencakup keseluruhan proses penyelenggaraan yang berkelanjutan, mulai dari bagaimana data dibuat, diakses, disimpan, hingga didistribusikan kepada pihak-pihak tertentu untuk memenuhi kebutuhan pelayanan kesehatan. Permenkes Nomor 24 Tahun 2022 juga mewajibkan seluruh fasilitas pelayanan kesehatan (fasyankes) di Indonesia, mulai dari praktik mandiri tenaga kesehatan hingga rumah sakit, untuk menyelenggarakan RME sebagai pengganti rekam medis konvensional (Kementerian Kesehatan Republik Indonesia, 2022).

Selain mewajibkan bentuk elektronik, Permenkes Nomor 24 Tahun 2022 turut mengatur hak pasien atas data rekam medis miliknya, termasuk larangan akses oleh pihak yang tidak berwenang (Kementerian Kesehatan Republik Indonesia, 2022). Ketentuan ini menjadi dasar regulasi bahwa setiap akses terhadap data RME seorang pasien pada dasarnya harus melalui mekanisme yang dapat mempertanggungjawabkan siapa yang mengakses, kapan, dan untuk keperluan apa. Hal tersebut sejalan dengan Undang-Undang Nomor 19 Tahun 2016 tentang Perubahan atas Undang-Undang Nomor 11 Tahun 2008 tentang Informasi dan Transaksi Elektronik, yang mengategorikan RME sebagai dokumen elektronik sehingga harus dibuat dan dikelola

dalam sistem elektronik dengan jaminan keamanan dan akuntabilitas agar dapat digunakan sebagai alat bukti yang sah secara hukum (Pemerintah Republik Indonesia, 2016). Dengan kata lain, regulasi di Indonesia tidak hanya menuntut RME diselenggarakan secara elektronik, tetapi juga menuntut adanya mekanisme yang menjamin kerahasiaan dan keutuhan data tersebut sepanjang siklus penyelenggaraannya, sebuah kebutuhan yang menjadi dasar bagi berbagai penelitian pengembangan sistem manajemen RME yang berfokus pada keamanan data, salah satunya DecMed (Purnama dkk., 2026).

II.1.2 Kebutuhan Keamanan dan Akuntabilitas Data RME

Data RME tergolong sebagai data yang sensitif karena memuat informasi identitas dan kondisi kesehatan pasien, sehingga keamanan dan kerahasiaannya menjadi perhatian utama dalam berbagai penelitian sistem informasi kesehatan. Tinjauan sistematis terhadap implementasi RME menunjukkan bahwa aspek keamanan dan kerahasiaan data medis pasien secara konsisten menjadi isu sentral, dan kegagalan menjaga aspek tersebut berdampak langsung pada menurunnya kepercayaan pasien terhadap sistem pelayanan kesehatan yang digunakan (Pramesti dkk., 2024). Kerahasiaan data RME tidak dapat dijaga hanya dengan membatasi siapa saja yang boleh mengakses data (kontrol akses), tetapi juga membutuhkan mekanisme yang dapat menunjukkan bahwa pembatasan tersebut benar-benar dipatuhi dan dapat ditelusuri kembali ketika terjadi insiden.

Kebutuhan penelusuran kembali (*traceability*) inilah yang menjadikan akuntabilitas sebagai aspek keamanan yang tidak dapat dipisahkan dari kerahasiaan dan integritas data RME. Akuntabilitas dalam konteks ini merujuk pada kemampuan suatu sistem untuk menghubungkan setiap tindakan yang dilakukan terhadap data dengan identitas pihak yang melakukannya, sehingga setiap pihak yang mengakses maupun mengubah data RME dapat dimintai pertanggungjawaban atas tindakannya. Sebagai contoh, kerangka kerja DecMed yang menjadi basis pengembangan pada Tugas Akhir ini dirancang agar seluruh akses terhadap data RME hanya dapat dilakukan atas persetujuan aktif pasien melalui mekanisme kontrol akses berbasis *capability* (Purnama dkk., 2026). Namun demikian, tanpa mekanisme yang menjamin akuntabilitas, kontrol akses yang ketat sekalipun tidak dapat membuktikan bahwa suatu pelanggaran keamanan benar-benar terjadi, siapa pihak yang bertanggung jawab,

maupun bagaimana kronologi pelanggaran tersebut (Purnama dkk., 2026). Kebutuhan inilah yang menjadi latar belakang pentingnya mekanisme audit log sebagai pelengkap dari kontrol akses pada sistem manajemen RME, yang akan dibahas lebih lanjut pada Subbab II.2.

II.2 Audit Log

II.2.1 Definisi dan Konsep Audit Log

Audit log didefinisikan sebagai kumpulan *audit record* yang tersusun secara kronologis (International Organization for Standardization, 2021). Definisi ini menekankan dua hal sekaligus: pertama, audit log merupakan kumpulan catatan, bukan catatan tunggal; dan kedua, urutan kronologis adalah sifat yang melekat, sehingga setiap *record* di dalamnya terkait dengan waktu kejadiannya dan dapat ditelusuri berdasarkan rentang waktu tertentu.

ISO 27789:2021 membedakan empat istilah yang berkaitan dalam hierarki yang berjenjang, sebagaimana ditunjukkan pada Gambar II.1. *Audit event* adalah satu peristiwa pada sistem yang memicu pembangkitan catatan audit (International Organization for Standardization, 2021), misalnya seorang pengguna membaca data rekam medis. *Audit record* adalah catatan dari satu peristiwa spesifik tunggal dalam siklus hidup rekam medis elektronik (International Organization for Standardization, 2021). *Audit log* adalah kumpulan *audit record* yang tersusun secara kronologis (International Organization for Standardization, 2021). Pada level tertinggi, *audit log* adalah kumpulan satu atau lebih *audit log* yang secara keseluruhan membentuk riwayat lengkap atas suatu entitas atau proses (International Organization for Standardization, 2021). Dengan kata lain, satu *audit event* menghasilkan satu *audit record*; sejumlah *audit record* yang tersusun membentuk sebuah *audit log*; dan satu atau lebih *audit log* secara kolektif membentuk sebuah *audit trail*.

Gambar II.1: Hierarki Konsep Audit Menurut ISO 27789:2021

Tugas Akhir ini berfokus pada level *audit log*: pengembangan mekanisme pembangkitan *audit record* dari berbagai komponen sistem, pengumpulannya ke dalam sebuah *audit log* yang terstruktur, serta penyimpanannya secara *tamper-evident*.

Mekanisme ini merupakan prasyarat sebelum *audit log* yang utuh dapat terbentuk dari satu atau lebih *audit log* yang dihasilkan.

Konsep audit log perlu dibedakan secara tegas dari konsep pencatatan izin akses (*access grant log*). Audit log bersifat komplementer terhadap kontrol akses, bukan pengganti maupun bagian darinya: kontrol akses menentukan kebijakan mengenai siapa *berhak* mengakses data, sedangkan audit log mencatat apa yang *benar-benar terjadi* pada sistem, termasuk apabila terjadi penyimpangan dari kebijakan tersebut (International Organization for Standardization, 2021). Perbedaan ini penting untuk digarisbawahi karena suatu sistem dapat saja memiliki kontrol akses yang ketat dan mencatat metadata pemberian izin secara lengkap, namun tetap tidak memiliki audit log yang memadai apabila tidak ada catatan terpisah mengenai peristiwa akses aktual (*access event*) yang terjadi setelah izin tersebut diberikan. Dengan kata lain, audit log idealnya mencatat peristiwa (*event*) sebagai representasi dari tindakan nyata yang terjadi pada sistem, bukan semata representasi dari kebijakan atau otorisasi yang berlaku.

II.2.2 Karakteristik Audit Log yang Baik

Agar dapat berfungsi sebagai bukti yang dapat diandalkan, audit log perlu memenuhi sejumlah karakteristik tertentu. Karakteristik pertama adalah kemampuan mengidentifikasi pengguna secara tidak ambigu (*unambiguous identification*), yaitu audit log harus menyediakan data yang cukup untuk mengidentifikasi secara pasti setiap pengguna maupun sistem eksternal yang telah mengakses atau menerima data rekam medis (International Organization for Standardization, 2021). Karakteristik kedua adalah integritas (*integrity*) dan kerahasiaan (*confidentiality*) dari audit log itu sendiri, yaitu jaminan bahwa catatan yang telah dibuat terlindungi dari upaya perusakan (*tampering*), termasuk perlunya audit log mencatat setiap tindakan yang dilakukan terhadap sistem audit itu sendiri, seperti kapan sistem audit tidak berfungsi atau dinonaktifkan (International Organization for Standardization, 2021).

Karakteristik lain yang tidak kalah penting adalah kelengkapan (*completeness*) dan ketersediaan (*availability*). Kelengkapan berarti audit log harus mencatat seluruh tindakan yang relevan terhadap data – termasuk pembuatan, pembacaan, pembaruan, maupun pengarsipan – setiap kali tindakan tersebut dilakukan terhadap data kesehatan pribadi (International Organization for Standardization, 2021), sedangkan

ketersediaan berarti sistem audit harus memastikan entri tercatat setiap kali sistem informasi kesehatan sedang beroperasi (International Organization for Standardization, 2021). Selain keempat karakteristik tersebut, audit log yang baik juga perlu mendukung kemampuan deteksi terhadap potensi penyalahgunaan oleh pihak internal (*insider threat*), mengingat pelaku *insider* pada umumnya memiliki akses sah terhadap sistem sehingga satu-satunya cara untuk mendeteksi maupun membuktikan penyalahgunaan tersebut adalah melalui pencatatan aktivitas yang menyeluruh dan tidak dapat dimanipulasi oleh pelaku itu sendiri (Theis dkk., 2022). Karakteristik-karakteristik tersebut menjadi acuan utama dalam menilai apakah suatu mekanisme pencatatan pada sistem informasi, termasuk pada DecMed, telah dapat disebut sebagai audit log yang memadai.

II.2.3 Komponen Audit Log

Untuk dapat berfungsi secara menyeluruh, suatu sistem audit log perlu dibangun melalui beberapa komponen fungsional yang saling berurutan. Kerangka kerja audit untuk rekam medis elektronik membagi persoalan menjadi penentuan peristiwa apa saja yang perlu dipicu untuk dicatat (*trigger events*), format dan isi dari setiap catatan yang dihasilkan, hingga bagaimana catatan tersebut dikelola secara aman sepanjang siklus hidupnya (International Organization for Standardization, 2021). Pembagian yang serupa juga ditemukan pada kerangka manajemen log komputer secara umum, yang memandang manajemen log sebagai suatu infrastruktur yang digunakan untuk menghasilkan (*generate*), mengirimkan (*transmit*), menyimpan (*store*), serta menganalisis (*analyze*) data log (Kent dan Souppaya, 2006). Berdasarkan kedua kerangka tersebut, komponen audit log pada Tugas Akhir ini dikelompokkan menjadi empat bagian: penentuan event, pengumpulan event, penyimpanan event, dan pembacaan event.

II.2.3.1 Penentuan Event

Komponen pertama adalah penentuan peristiwa (*event*) apa saja yang perlu memicu pencatatan pada audit log. Peristiwa pemicu (*trigger events*) pada sistem rekam medis elektronik pada umumnya ditentukan berdasarkan skala, tujuan, serta kebijakan privasi dan keamanan dari masing-masing sistem informasi kesehatan, dengan cakupan minimal berupa peristiwa akses (*access events*) dan peristiwa kueri (*query*

events) terhadap data kesehatan pribadi (International Organization for Standardization, 2021). Standar tersebut turut memberikan contoh peristiwa yang secara eksplisit berada di luar cakupan pencatatan audit rekam medis, seperti peristiwa mulai/berhenti aplikasi, peristiwa autentikasi pengguna, maupun peristiwa yang dihasilkan oleh sistem operasi (International Organization for Standardization, 2021).

Di luar peristiwa akses dan kueri, penentuan *trigger events* pada suatu sistem juga dapat diturunkan melalui pendekatan pemodelan ancaman (*threat modeling*), seperti metode STRIDE, yang mengidentifikasi ancaman keamanan pada suatu sistem secara sistematis dan dapat digunakan sebagai dasar penentuan kebutuhan pencatatan (Laksono dan Prayudi, 2021).

II.2.3.2 Konten Wajib dan Konten Tambahan pada Audit Record

Setiap *audit record*, terlepas dari jenis peristiwa apa yang dicatatnya, memuat sekumpulan bidang data minimal yang bersifat wajib (*mandatory*). Format umum suatu *audit record* pada rekam medis elektronik sekurang-kurangnya memuat identitas unik dari peristiwa yang terjadi (*EventID*), jenis tindakan yang dilakukan (*EventActionCode*), waktu terjadinya peristiwa (*EventDateTime*), identitas pengguna atau proses yang melakukan tindakan (*UserID*), serta identitas objek data yang menjadi sasaran tindakan tersebut (*ParticipantObjectID*) (International Organization for Standardization, 2021).

Di samping bidang wajib tersebut, *audit record* dapat pula memuat konten tambahan (*optional*) yang bersifat kontekstual bergantung pada jenis *event*. Bidang tambahan ini dapat mencakup informasi mengenai hasil dari tindakan yang dilakukan (*outcome*), parameter permintaan yang dikirimkan, maupun atribut bukti digital yang spesifik diperlukan untuk keperluan investigasi ancaman tertentu (International Organization for Standardization, 2021).

II.2.3.3 Pengumpulan Event

Komponen kedua adalah pengumpulan event, yaitu bagaimana setiap peristiwa yang terjadi pada sistem direkam menjadi suatu *audit record* dengan format dan isi yang konsisten. Pengumpulan event pada praktiknya menghadapi tantangan tersendiri, terutama pada sistem dengan banyak sumber log (*log sources*) yang berbeda. Setiap sumber log cenderung mencatat informasi dengan format dan kelengkapan yang

berbeda-beda, sehingga menyulitkan proses menghubungkan (*correlate*) peristiwa yang tercatat oleh satu sumber dengan peristiwa yang tercatat oleh sumber lainnya (Kent dan Souppaya, 2006).

II.2.3.4 Penyimpanan Event

Komponen ketiga adalah penyimpanan event, yaitu bagaimana catatan peristiwa yang telah dikumpulkan disimpan dan dikelola secara aman sepanjang siklus hidupnya. Audit log perlu menyediakan langkah pengamanan yang cukup untuk melindungi isinya dari perusakan (*tampering*), termasuk mengamankan akses terhadap *audit record*, mengamankan akses terhadap perangkat audit itu sendiri, serta mencatat seluruh tindakan yang dilakukan terhadap audit log dengan menyertakan waktu, jenis tindakan, dan pelakunya (International Organization for Standardization, 2021). Selain itu, *audit record* juga perlu tunduk pada kebijakan retensi (*retention policy*) yang idealnya disesuaikan dengan masa berlaku data rekam medis yang bersangkutan, mengingat *audit record* dapat dibutuhkan sebagai bukti dalam jangka waktu yang panjang (International Organization for Standardization, 2021).

Pada level infrastruktur, pertimbangan penyimpanan log turut mencakup keseimbangan antara volume data log yang dihasilkan dengan kapasitas penyimpanan yang tersedia, baik untuk penyimpanan daring (*online*) maupun luring (*offline*), serta kebutuhan keamanan atas data tersebut (Kent dan Souppaya, 2006).

II.2.3.5 Pembacaan Event

Komponen keempat adalah pembacaan event, yaitu bagaimana catatan peristiwa yang telah tersimpan dapat diakses kembali dan dianalisis oleh pihak yang berwenang. Akses terhadap audit log harus dikendalikan secara ketat dan akses itu sendiri menjadi objek audit (*itself subject to audit*), dengan akses yang idealnya dilakukan melalui sistem informasi yang dapat menegakkan kontrol tersebut, bukan akses langsung terhadap berkas log (International Organization for Standardization, 2021).

II.2.4 Kebutuhan Audit Log

Berdasarkan definisi, karakteristik, dan komponen yang telah diuraikan pada subbab sebelumnya, dapat diidentifikasi kebutuhan-kebutuhan yang harus dipenuhi

oleh suatu sistem audit log agar dapat berfungsi secara efektif sebagai bukti digital dalam proses audit maupun investigasi insiden keamanan. Kebutuhan tersebut dapat dikelompokkan menjadi kebutuhan fungsional dan kebutuhan keamanan.

II.2.4.1 Kebutuhan Fungsional

Kebutuhan fungsional berkaitan dengan kemampuan yang harus dimiliki sistem audit log untuk menjalankan fungsinya:

- **Pencatatan event yang komprehensif.** Sistem harus mampu mencatat seluruh aktivitas yang relevan dari berbagai komponen sistem. Cakupan minimal mencakup peristiwa akses dan peristiwa kueri terhadap data sensitif (International Organization for Standardization, 2021).
- **Format record yang konsisten.** Setiap *audit record* harus memiliki struktur dan atribut yang seragam, memuat setidaknya informasi mengenai jenis event, waktu kejadian, sumber event, hasil event, serta identitas aktor dan objek yang terlibat (International Organization for Standardization, 2021).
- **Penyimpanan terpusat dan terdedikasi.** Seluruh *audit record* harus tersimpan pada repositori khusus yang terpisah dari data operasional sistem, sehingga dapat diakses secara independen untuk keperluan audit (International Organization for Standardization, 2021).
- **Kemampuan pencarian dan analisis.** Sistem harus menyediakan fasilitas untuk mengambil dan menganalisis *audit record* berdasarkan berbagai kombinasi atribut seperti rentang waktu, jenis event, aktor, maupun objek yang terlibat (International Organization for Standardization, 2021).
- **Retensi jangka panjang.** *Audit record* harus dipertahankan selama periode yang sesuai dengan kebutuhan organisasi dan regulasi yang berlaku, mempertimbangkan bahwa investigasi dapat dilakukan jauh setelah kejadian (International Organization for Standardization, 2021).

II.2.4.2 Kebutuhan Keamanan

Kebutuhan keamanan berkaitan dengan sifat-sifat yang harus dimiliki *audit record* agar dapat dipercaya sebagai bukti digital:

- **Integritas.** *Audit record* yang telah tersimpan harus terlindungi dari modifikasi yang tidak sah. Setiap perubahan pada *audit record* harus dapat terdeteksi (International Organization for Standardization, 2021).
- **Keaslian (*authenticity*).** Harus tersedia mekanisme untuk memverifikasi bahwa *audit record* benar-benar dibangkitkan oleh komponen sistem yang diklaim sebagai sumbernya, bukan oleh pihak lain yang berpura-pura menjadi sumber tersebut (International Organization for Standardization, 2021).
- **Non-repudiation.** *Audit record* harus dapat digunakan untuk membuktikan bahwa suatu aktivitas benar-benar terjadi dan dilakukan oleh entitas tertentu, sehingga entitas tersebut tidak dapat menyangkal keterlibatannya (Shostack, 2014, International Organization for Standardization, 2021).
- **Sifat *append-only*.** Mekanisme penyimpanan *audit record* harus bersifat tambah-saja, artinya entri yang telah tercatat tidak dapat diubah maupun dihapus. Perubahan kondisi akibat aktivitas berikutnya dicatat sebagai entri baru yang terpisah, bukan sebagai pembaruan entri lama (International Organization for Standardization, 2021).
- **Akses terkendali.** Akses terhadap *audit record* harus dikendalikan secara ketat dan akses itu sendiri harus menjadi objek audit, sehingga tidak ada pihak yang dapat mengakses maupun memanipulasi *audit record* tanpa meninggalkan jejak (International Organization for Standardization, 2021).

Kebutuhan-kebutuhan tersebut bersifat saling melengkapi: sistem yang hanya memenuhi kebutuhan fungsional tanpa memenuhi kebutuhan keamanan tidak dapat diandalkan sebagai bukti digital, sedangkan sistem yang memenuhi kebutuhan keamanan namun tidak memenuhi kebutuhan fungsional tidak dapat digunakan untuk investigasi yang komprehensif. Kebutuhan inilah yang akan digunakan sebagai tolok ukur dalam mengevaluasi kondisi audit log pada sistem DecMed di Bab III.

II.2.5 Mekanisme *Tamper-Evident* pada Audit Log

Untuk memenuhi kebutuhan keamanan yang telah diidentifikasi pada Subbab II.2.4, khususnya integritas, keaslian, dan non-repudiation, terdapat beberapa mekanisme kriptografis yang lazim digunakan dalam perancangan sistem audit log.

II.2.5.1 *Hash-Chaining* untuk Integritas Berurutan

Hash-chaining adalah mekanisme yang menjamin integritas berurutan (*sequential integrity*) dari sebuah audit log dengan cara menyertakan nilai *hash* kriptografis dari *record* sebelumnya ke dalam setiap *record* baru yang dibuat. Dengan demikian, setiap *audit record* secara kriptografis bergantung pada seluruh *record* yang mendahuluinya (Kent dan Souppaya, 2006).

Implikasi dari mekanisme ini adalah bahwa modifikasi atas satu *record* dalam rantai – baik dengan mengubah isi *record*, menghapusnya, maupun menyisipkan *record* baru di tengah rantai – akan menyebabkan nilai *hash* dari seluruh *record* berikutnya menjadi tidak valid. Ketidaksesuaian ini dapat dideteksi dengan memverifikasi ulang rantai *hash* dari awal hingga akhir, sehingga audit log bersifat *tamper-evident*: kerusakan yang terjadi dapat dideteksi meski tidak dapat dicegah secara langsung pada tingkat berkas lokal.

II.2.5.2 Tanda Tangan Digital untuk Keaslian dan Non-Repudiation

Tanda tangan digital merupakan mekanisme kriptografis yang memungkinkan suatu entitas membuktikan kepemilikan atas sebuah pesan atau dokumen. Prinsip kerjanya adalah: pengirim menandatangani data menggunakan *private key* miliknya, menghasilkan tanda tangan yang dapat diverifikasi oleh siapa pun yang memiliki *public key* bersesuaian (Shostack, 2014).

Dalam konteks audit log, tanda tangan digital memenuhi dua kebutuhan sekaligus. *Pertama*, keaslian (*authenticity*): karena hanya pemegang *private key* yang dapat menghasilkan tanda tangan yang valid untuk kunci publik tertentu, verifikasi tanda tangan membuktikan bahwa audit event benar-benar dibangkitkan oleh komponen yang mengklaim sebagai sumbernya. *Kedua*, non-repudiation: apabila suatu entitas telah menandatangani sebuah event, entitas tersebut tidak dapat menyangkal pernah membangkitkan event tersebut selama *private key*-nya tidak dikompromikan (Shostack, 2014).

Salah satu skema tanda tangan digital yang banyak digunakan untuk keperluan ini adalah Ed25519, yang berbasis kurva eliptik Edwards25519. Skema ini bersifat deterministik (tanda tangan yang dihasilkan selalu sama untuk input yang sama), memiliki ukuran kunci dan tanda tangan yang kecil (32 byte untuk kunci publik, 64

byte untuk tanda tangan), dan efisien secara komputasi, menjadikannya pilihan yang sesuai untuk lingkungan yang membutuhkan verifikasi tanda tangan dalam volume tinggi.

II.2.5.3 Rotasi dan Pengarsipan Log

Rotasi log adalah praktik manajemen log di mana berkas log aktif secara berkala diganti dengan berkas baru, sementara berkas lama diarsipkan untuk keperluan re-tensi (Kent dan Souppaya, 2006). Praktik ini memiliki tiga manfaat utama dalam konteks audit log.

Pertama, manajemen ukuran: berkas log yang terus tumbuh tanpa batas akan menyulitkan pengelolaan dan dapat menghabiskan kapasitas penyimpanan. Dengan rotasi berkala, setiap berkas arsip memiliki ukuran yang dapat diprediksi. *Kedua*, efisiensi analisis: investigasi seringkali berfokus pada rentang waktu tertentu; berkas arsip yang terorganisasi berdasarkan periode waktu memudahkan pencarian data yang relevan tanpa harus memindai seluruh riwayat log. *Ketiga*, keamanan penyimpanan: berkas arsip yang telah dirotasi dapat segera dipindahkan ke media penyimpanan yang lebih aman dan tahan lama, terpisah dari lingkungan operasional yang lebih rentan terhadap gangguan (Kent dan Souppaya, 2006).

Kombinasi antara *hash-chaining* pada level *record*, tanda tangan digital untuk keaslian *event*, dan rotasi log untuk pengarsipan berkala membentuk tiga lapisan mekanisme yang saling melengkapi dalam menjamin integritas, keaslian, dan ketersediaan jangka panjang dari sebuah sistem audit log yang bersifat *tamper-evident*.

II.3 Keamanan Informasi dan Akuntabilitas

II.3.1 Prinsip Dasar Keamanan Informasi

Keamanan informasi secara klasik didefinisikan melalui tiga properti utama yang dikenal sebagai *CIA Triad*: *Confidentiality* (kerahasiaan), *Integrity* (integritas), dan *Availability* (ketersediaan) (National Institute of Standards and Technology, 2020). Ketiga properti ini membentuk landasan konseptual bagi sebagian besar standar dan kerangka kerja keamanan sistem informasi, termasuk NIST SP 800-53 dan ISO/IEC 27002.

Confidentiality merujuk pada jaminan bahwa informasi hanya dapat diakses oleh pihak-pihak yang berwenang, sehingga melindungi data dari pengungkapan yang tidak sah (National Institute of Standards and Technology, 2020). Dalam konteks RME, kerahasiaan berarti bahwa data klinis dan administratif pasien tidak boleh dapat dibaca oleh pihak mana pun tanpa otorisasi yang sah dari pasien maupun prosedur yang berlaku.

Integrity merujuk pada jaminan bahwa informasi tidak dimodifikasi atau dihancurkan dengan cara yang tidak sah, termasuk jaminan ketidakpenyangkalan dan keaslian informasi (National Institute of Standards and Technology, 2020). Integritas tidak hanya mencakup perlindungan terhadap perubahan isi data, tetapi juga mencakup jaminan bahwa data yang tersimpan mencerminkan kondisi yang sesungguhnya terjadi pada sistem, bukan hasil manipulasi pihak tidak berwenang.

Availability merujuk pada jaminan bahwa sistem informasi dan data yang dikelolanya dapat diakses dan digunakan oleh pihak yang berwenang pada saat dibutuhkan (National Institute of Standards and Technology, 2020). Ketersediaan menjadi relevan tidak hanya bagi fungsionalitas layanan, tetapi juga bagi sistem audit log: rekaman yang tidak dapat diakses saat dibutuhkan untuk investigasi tidak dapat berfungsi sebagai bukti yang andal.

Ketiga properti tersebut saling berkaitan dan dalam praktiknya sering kali memiliki tegangan satu sama lain. Upaya meningkatkan kerahasiaan melalui enkripsi data, misalnya, dapat mempersulit verifikasi integritas secara langsung. Sebaliknya, mekanisme yang menjamin ketersediaan data secara luas dapat meningkatkan risiko terhadap kerahasiaannya. Ketegangan ini menjadi salah satu alasan mengapa perancangan sistem keamanan perlu dilakukan secara holistik dengan mempertimbangkan ketiga properti secara bersamaan (Shostack, 2014).

II.3.2 Properti Keamanan yang Diperluas

Seiring perkembangan sistem informasi yang semakin kompleks, komunitas keamanan informasi mengakui bahwa *CIA Triad* tidak sepenuhnya mencakup seluruh kebutuhan keamanan yang relevan. Beberapa properti tambahan telah diidentifikasi sebagai pelengkap yang diperlukan, khususnya dalam konteks sistem yang melibatkan banyak pihak dengan tingkat kepercayaan yang berbeda (International Or-

ganization for Standardization, 2021).

Authenticity (keaslian) adalah properti yang menjamin bahwa suatu entitas — baik pengguna, proses, maupun data — benar-benar merupakan entitas yang diklaim, bukan pihak lain yang berpura-pura (National Institute of Standards and Technology, 2020). Keaslian mencakup verifikasi identitas aktor yang berinteraksi dengan sistem, maupun verifikasi bahwa data yang diterima berasal dari sumber yang sah dan tidak mengalami perubahan selama pengiriman. Dalam konteks audit log, keaslian berarti bahwa setiap catatan peristiwa harus dapat dibuktikan berasal dari komponen sistem yang diklaim sebagai sumbernya.

Non-Repudiation (ketidakpenyangkalan) adalah properti yang menjamin bahwa suatu entitas tidak dapat menyangkal telah melakukan suatu tindakan tertentu setelah tindakan tersebut terjadi (National Institute of Standards and Technology, 2020). Non-repudiation merupakan properti yang beroperasi pada lapisan bisnis dan hukum, bukan semata pada lapisan teknis: tujuannya adalah menyediakan bukti yang secara meyakinkan dapat membuktikan keterlibatan suatu entitas dalam suatu peristiwa, sehingga penyangkalan atas peristiwa tersebut tidak dapat diterima. Non-repudiation umumnya diwujudkan melalui kombinasi tanda tangan digital dan sistem pencatatan yang tidak dapat dimanipulasi (Shostack, 2014).

Accountability (akuntabilitas) adalah properti yang menjamin bahwa tindakan suatu entitas pada suatu sistem dapat ditelusuri secara unik kepada entitas tersebut (National Institute of Standards and Technology, 2020). Akuntabilitas merupakan properti yang lebih luas dari non-repudiation: non-repudiation berfokus pada pembuktian bahwa suatu tindakan tertentu pernah dilakukan oleh entitas tertentu, sedangkan akuntabilitas mencakup kemampuan sistem untuk menghubungkan setiap tindakan yang terjadi dengan pelakunya secara menyeluruh, tidak hanya pada kasus yang disengketakan. Standar NIST SP 800-53 menetapkan akuntabilitas sebagai prasyarat bagi penerapan tanggung jawab individual (*individual accountability*) pada sistem informasi, yang mensyaratkan bahwa setiap pengguna sistem dapat dimintai pertanggungjawaban atas tindakannya (National Institute of Standards and Technology, 2020).

Reliability (keandalan) dan **Trustworthiness** (keterpercayaan) merupakan properti pendukung yang menjamin bahwa sistem berperilaku secara konsisten sesuai de-

ngan yang dimaksudkan, dan bahwa keluaran yang dihasilkan oleh sistem dapat dipercaya sebagai representasi yang akurat dari kondisi yang sesungguhnya terjadi (International Organization for Standardization, 2021).

II.3.3 Keterkaitan antara Properti Keamanan dan Audit Log

Sistem audit log tidak hanya merupakan mekanisme untuk memenuhi kebutuhan keamanan tertentu, tetapi juga merupakan komponen yang secara langsung merealisasikan beberapa properti keamanan sekaligus. Keterkaitan antara properti-properti keamanan yang telah diuraikan dan mekanisme audit log dapat dijabarkan sebagai berikut.

Pertama, audit log berkontribusi secara langsung terhadap *integrity*. Ketika suatu sistem memiliki rekaman kronologis atas setiap tindakan yang dilakukan terhadap data, rekaman tersebut dapat digunakan untuk memverifikasi bahwa status data saat ini konsisten dengan seluruh tindakan yang pernah tercatat. Apabila terdapat perbedaan antara status data yang teramati dan riwayat tindakan yang tercatat, hal tersebut mengindikasikan kemungkinan adanya modifikasi yang tidak tercatat, yaitu pelanggaran integritas (International Organization for Standardization, 2021).

Kedua, audit log merupakan mekanisme utama untuk mewujudkan *accountability*. Tanpa pencatatan yang menyeluruh atas tindakan yang dilakukan oleh setiap pengguna dan komponen sistem, akuntabilitas hanya bersifat normatif (kebijakan yang menyatakan siapa bertanggung jawab atas apa) tanpa dapat diwujudkan secara teknis (kemampuan sistem untuk membuktikan siapa melakukan apa dan kapan). NIST SP 800-53 secara eksplisit menempatkan keluarga kontrol *Audit and Accountability* (AU) sebagai mekanisme teknis utama untuk merealisasikan akuntabilitas pada sistem informasi (National Institute of Standards and Technology, 2020).

Ketiga, audit log yang dilengkapi dengan mekanisme tanda tangan digital merealisasikan properti *non-repudiation*. Apabila setiap catatan peristiwa ditandatangani secara kriptografis oleh komponen yang membangkitkannya, maka komponen tersebut tidak dapat menyangkal pernah membangkitkan peristiwa tersebut selama kunci privatnya tidak dikompromikan (Shostack, 2014). Dalam konteks sistem multipartai seperti DecMed, non-repudiation melalui audit log menjadi penting karena banyak tindakan yang memiliki konsekuensi hukum — seperti pemberian akses ke

data pasien, pembuatan rekam medis, atau pencabutan akses — perlu dapat dibuktikan secara meyakinkan bahwa tindakan tersebut benar-benar dilakukan oleh pihak yang mengklaim atau yang diklaim melakukannya.

Keempat, audit log yang bersifat *tamper-evident* dan disimpan secara terpisah dari data operasional berkontribusi terhadap *availability* dalam arti yang lebih luas: ketersediaan bukti. Dalam investigasi insiden keamanan, ketersediaan bukti yang otentik, lengkap, dan tidak dapat dimanipulasi menentukan apakah investigasi tersebut dapat menghasilkan kesimpulan yang meyakinkan atau tidak (International Organization for Standardization, 2021).

Hubungan antara properti keamanan dan mekanisme audit log disarikan pada Tabel II.1.

Tabel II.1: Keterkaitan Properti Keamanan dengan Mekanisme Audit Log

Properti Keamanan	Mekanisme Audit Log	Kontribusi
<i>Integrity</i>	<i>Hash-chaining</i> , penyimpanan <i>append-only</i>	Memungkinkan deteksi modifikasi data yang tidak tercatat melalui inkonsistensi antara status data dan riwayat tindakan.
<i>Availability</i>	Retensi jangka panjang, penyimpanan terdesentralisasi	Menjamin ketersediaan bukti saat dibutuhkan untuk investigasi, tidak hanya ketersediaan layanan operasional.
<i>Authenticity</i>	Tanda tangan digital pada <i>audit event</i>	Membuktikan bahwa catatan peristiwa berasal dari komponen yang diklaim sebagai sumbernya.
<i>Non-Repudiation</i>	Tanda tangan digital, pencatatan <i>immutable</i>	Mencegah penyangkalan atas tindakan yang telah dilakukan dan tercatat dalam sistem.
<i>Accountability</i>	Pencatatan menyeluruh atas seluruh tindakan dan aktornya	Menghubungkan setiap tindakan pada sistem dengan identitas pelakunya secara dapat dibuktikan.

II.4 Kerja DecMed

II.4.1 Gambaran Umum dan Latar Belakang DecMed

DecMed adalah kerangka kerja manajemen RME terdesentralisasi yang dikembangkan untuk menjawab kebutuhan pengelolaan data rekam medis yang aman, akun-

tabel, dan berpusat pada kendali pasien (Purnama dkk., 2026). Kerangka kerja ini mengintegrasikan tiga teknologi utama: DLT IOTA sebagai lapisan penyimpanan *on-chain* yang *immutable* untuk data kontrol akses, IPFS sebagai penyimpanan *off-chain* untuk berkas klinis terenkripsi berukuran besar, serta PRE sebagai mekanisme kriptografi yang memungkinkan distribusi akses data kepada pihak yang berwenang tanpa mengungkap isi data kepada perantara (Purnama dkk., 2026). Kombinasi ketiga teknologi ini dirancang agar setiap akses terhadap data RME hanya dapat dilakukan atas persetujuan aktif pasien, dengan durasi dan cakupan akses yang dapat dibatasi oleh pasien sendiri melalui mekanisme CapBAC (Purnama dkk., 2026).

Arsitektur DecMed mengikuti pola tiga lapis yang telah diuraikan pada Subbab II.7.1. Tiga aplikasi *client* utama, yaitu *Patient Client* (berbasis mobile), *Hospital Client* (berbasis desktop), dan *Ministry Client* (berbasis desktop), berfungsi sebagai antarmuka bagi masing-masing peran aktor dalam sistem. Seluruh aplikasi *client* dibangun menggunakan Tauri dengan *frontend* SvelteKit/TypeScript, sementara logika *backend* ditulis dalam Rust (Purnama dkk., 2026). Identitas kriptografi pengguna dikelola secara *non-custodial* menggunakan skema BIP39, di mana setiap pengguna menyimpan kunci privat mereka sendiri dalam bentuk *mnemonic* 12 kata tanpa bergantung pada pihak ketiga (Purnama dkk., 2026).

II.4.2 Mekanisme Kontrol Akses pada DecMed

Mekanisme kontrol akses pada DecMed dibangun di atas model CapBAC yang memanfaatkan *smart contract* Move pada jaringan IOTA sebagai repositori *capability* yang *immutable* (Purnama dkk., 2026). Ketika seorang pasien memberikan akses kepada tenaga medis atau tenaga administratif, pasien membuat entri *capability* pada *smart contract* yang memuat atribut seperti identitas penerima akses, jenis akses yang diberikan (*read* atau *update*), dan waktu kedaluwarsa akses. Entri ini tersimpan secara *on-chain* dan tidak dapat diubah oleh pihak lain selain pasien selaku pemilik data (Purnama dkk., 2026).

Setiap kali tenaga medis mengajukan permintaan akses terhadap data RME, PRE Server memverifikasi keberadaan entri *capability* yang bersesuaian pada *smart contract* sebelum melakukan *re-encryption* data. Proses ini memastikan bahwa akses hanya diberikan kepada pihak yang memiliki *capability* aktif dan belum kedaluwarsa (Purnama dkk., 2026). Selain memberikan akses, pasien juga dapat mencabut ak-

ses (*revoke access*) sewaktu-waktu, yang menyebabkan entri *capability* pada *smart contract* diperbarui dengan status *revoked* (Purnama dkk., 2026).

II.4.3 Keterbatasan Audit Log pada DecMed

DecMed mengklaim menghasilkan mekanisme kontrol akses yang didukung oleh *audit log* yang tahan terhadap perusakan (*tamper-evident audit trails*) (Purnama dkk., 2026). Klaim tersebut didasarkan pada mekanisme bahwa setiap akses yang diberikan oleh pasien dicatat sebagai entri *capability* pada penyimpanan *on-chain* di jaringan IOTA, dan sifat *immutability* dari penyimpanan *on-chain* tersebut menjamin bahwa entri tidak dapat diubah oleh pihak lain, dengan setiap permintaan akses divalidasi terhadap keberadaan entri yang bersesuaian melalui *smart contract* (Purnama dkk., 2026). Peristiwa akses, pemberian persetujuan (*consent*), dan pelanggaran kebijakan pada DecMed dinyatakan tercatat secara *tamper-evident*, sehingga menyediakan audit log yang dapat diandalkan untuk mendukung akuntabilitas lintas batas organisasi (Purnama dkk., 2026).

Meskipun demikian, apabila ditinjau terhadap karakteristik audit log yang baik sebagaimana diuraikan pada Subbab II.2.2, mekanisme pencatatan yang tersedia pada DecMed memiliki sejumlah keterbatasan mendasar. *Pertama*, sebagian besar mekanisme yang tercatat pada jaringan IOTA merupakan entri *capability* atau izin akses (*access grant*), yaitu metadata yang menyatakan siapa berhak mengakses data apa dan sampai kapan, bukan catatan atas peristiwa akses yang benar-benar terjadi (*access event*), sehingga sistem belum dapat merekonstruksi riwayat siapa yang benar-benar membaca atau mengubah data RME pada waktu tertentu. *Kedua*, sejumlah operasi pada DecMed, seperti pencabutan akses (*revoke_access*) dan pembaruan data administratif maupun klinis, menyebabkan entri lama pada state tergantikan oleh entri baru alih-alih ditambahkan sebagai riwayat baru yang bersifat *append-only*, yang bertentangan dengan karakteristik dasar audit log mengenai integritas dan *non-repudiation* (Purnama dkk., 2026). Keterbatasan-keterbatasan inilah yang menjadi motivasi utama pengembangan mekanisme audit log yang memadai pada Tugas Akhir ini, sebagaimana akan dianalisis lebih lanjut pada Bab III.

II.5 Threat Modeling dengan STRIDE

II.5.1 Konsep dan Kategori STRIDE

Threat modeling (pemodelan ancaman) merupakan proses terstruktur untuk mengidentifikasi, mengenumerasi, dan memprioritaskan ancaman keamanan pada suatu sistem sebelum ancaman tersebut benar-benar dieksploitasi (Shostack, 2014). Proses ini umumnya terdiri atas empat pertanyaan pokok: apa yang sedang dibangun, apa yang dapat salah, apa yang perlu dilakukan terhadap hal tersebut, dan apakah pekerjaan tersebut telah dilakukan dengan cukup baik (Shostack, 2014).

STRIDE merupakan salah satu kerangka kerja *threat modeling* yang dikembangkan oleh Loren Kohnfelder dan Praerit Garg di Microsoft, dengan nama yang dibentuk dari huruf pertama masing-masing kategori ancaman (Shostack, 2014). Kerangka kerja ini dirancang untuk membantu pengembang perangkat lunak mengidentifikasi jenis-jenis serangan yang cenderung dialami suatu sistem. Setiap kategori STRIDE merupakan kebalikan dari properti keamanan yang diinginkan pada suatu sistem, sebagaimana ditunjukkan pada Tabel II.2.

Tabel II.2: Kategori Ancaman STRIDE dan Properti Keamanan yang Dilanggar

Kategori	Properti yang Dilanggar	Definisi
Spoofing	Authentication	Berpura-pura menjadi sesuatu atau seseorang selain diri sendiri.
Tampering	Integrity	Memodifikasi sesuatu pada disk, jaringan, atau memori.
Repudiation	Non-repudiation	Mengklaim tidak melakukan sesuatu atau tidak bertanggung jawab atas suatu tindakan.
Information Disclosure	Confidentiality	Memberikan informasi kepada pihak yang tidak berwenang untuk melihatnya.
Denial of Service	Availability	Menyerap sumber daya yang dibutuhkan untuk menyediakan layanan.
Elevation of Privilege	Authorization	Memungkinkan seseorang melakukan sesuatu yang tidak diizinkan untuk dilakukan.

Kategori *Repudiation* memiliki karakteristik yang berbeda dari kategori lainnya: ancaman ini beroperasi pada lapisan bisnis, bukan semata pada lapisan teknis jaringan atau aplikasi (Shostack, 2014). Penting untuk dipahami bahwa repudiasi tidak selalu merupakan tindakan jahat, misalnya seseorang dapat secara jujur menyatakan

tidak melakukan sesuatu karena kegagalan teknologi atau proses. Oleh karena itu, pertanyaan kunci bagi perancang sistem adalah: *bukti apa yang dimiliki sistem untuk membantah atau membuktikan suatu klaim?* Apabila sistem tidak memiliki log, tidak menyimpan log, atau tidak dapat menganalisis log, ancaman repudiasi menjadi sangat sulit untuk dibantah (Shostack, 2014). Hal ini menegaskan keterkaitan langsung antara kategori *Repudiation* pada STRIDE dengan kebutuhan mekanisme audit log pada suatu sistem.

II.5.2 Dekomposisi Sistem dan Data Flow Diagram dalam Threat Modeling

Sebelum mengidentifikasi ancaman menggunakan STRIDE, suatu sistem perlu didekomposisi terlebih dahulu untuk memperoleh pemahaman menyeluruh mengenai cara kerja sistem dan bagaimana sistem tersebut berinteraksi dengan entitas eksternal (OWASP Foundation, 2026). Dekomposisi ini umumnya mencakup identifikasi deskripsi sistem, dependensi eksternal, titik masuk (*entry points*), aset yang perlu dilindungi, dan tingkat kepercayaan (*trust level*) dari setiap entitas yang terlibat.

Data Flow Diagram (DFD) merupakan alat utama yang digunakan dalam tahap dekomposisi ini. DFD menggambarkan bagaimana data mengalir di antara elemen-elemen sistem, sehingga memungkinkan analisis untuk mengidentifikasi di mana ancaman keamanan paling mungkin muncul (OWASP Foundation, 2026). DFD dalam konteks *threat modeling* terdiri atas empat elemen dasar:

- **Proses** (*Process*): elemen yang menerima, mengolah, dan mengirimkan data, biasanya direpresentasikan sebagai lingkaran atau persegi panjang dengan sudut membulat.
- **Penyimpanan Data** (*Data Store*): tempat data disimpan secara persisten; direpresentasikan sebagai dua garis paralel.
- **Aliran Data** (*Data Flow*): perpindahan data antar elemen; direpresentasikan sebagai panah berlabel.
- **Entitas Eksternal** (*External Entity*): elemen yang berada di luar batas sistem yang dianalisis namun berinteraksi dengannya; direpresentasikan sebagai persegi panjang.

Selain keempat elemen dasar tersebut, DFD untuk *threat modeling* turut menandai *trust boundary*, yaitu batas antara dua area dengan tingkat kepercayaan yang berbeda. Setiap aliran data yang melintasi *trust boundary* menjadi kandidat utama untuk dianalisis ancamannya menggunakan STRIDE, karena pada titik-titik tersebut data berpindah dari satu konteks kepercayaan ke konteks lain yang berpotensi lebih tidak terpercaya (OWASP Foundation, 2026).

II.5.3 Pendekatan STRIDE-per-Interaction

Dalam penerapan STRIDE, terdapat dua pendekatan utama: STRIDE *per element* dan STRIDE *per interaction*. Pendekatan STRIDE *per element* menganalisis setiap elemen DFD secara individual dan menerapkan seluruh enam kategori STRIDE pada elemen tersebut; pendekatan ini mudah diterapkan namun cenderung menghasilkan banyak ancaman yang tidak relevan karena tidak semua kategori STRIDE berlaku untuk setiap jenis elemen (Shostack, 2014). Pendekatan STRIDE *per interaction*, sebaliknya, menganalisis setiap aliran data (*interaction*) yang melintasi *trust boundary* dan hanya menerapkan kategori STRIDE yang relevan berdasarkan jenis interaksi tersebut, menghasilkan daftar ancaman yang lebih terfokus dan relevan (Shostack, 2014).

Relevansi kategori STRIDE terhadap jenis elemen DFD pada pendekatan *per interaction* ditunjukkan pada Tabel II.3.

Tabel II.3: Relevansi Kategori STRIDE terhadap Jenis Elemen DFD

Kategori STRIDE	Proses	Data Store	Data Flow	External Entity
Spoofing	✓		✓	✓
Tampering	✓	✓	✓	
Repudiation	✓	✓		✓
Information Disclosure	✓	✓	✓	
Denial of Service	✓	✓	✓	
Elevation of Privilege	✓			

II.5.4 Penurunan Kebutuhan Audit Event dari Analisis STRIDE

Hasil analisis STRIDE terhadap suatu sistem tidak hanya mengidentifikasi ancaman, tetapi juga secara langsung menunjukkan jenis-jenis peristiwa yang perlu dicatat untuk mendeteksi, menginvestigasi, maupun mencegah ancaman tersebut. Seti-

ap ancaman yang teridentifikasi dapat diterjemahkan menjadi satu atau lebih *audit event* yang perlu dicatat oleh sistem (Laksono dan Prayudi, 2021).

Sebagai contoh, ancaman *Spoofing* menunjukkan kebutuhan untuk mencatat peristiwa autentikasi beserta hasilnya (berhasil atau gagal), sehingga percobaan penya-mararan identitas dapat terdeteksi dari pola kegagalan autentikasi berulang. Ancaman *Repudiation* menunjukkan kebutuhan untuk mencatat setiap tindakan yang memiliki konsekuensi signifikan beserta identitas pelakunya, sehingga penyangkalan di kemudian hari dapat dibantah menggunakan rekaman yang ada. Ancaman *Tampering* menunjukkan kebutuhan untuk mencatat setiap operasi tulis atau modifikasi terhadap data sensitif, sehingga perubahan yang tidak sah dapat ditelusuri (Laksono dan Prayudi, 2021).

Pendekatan berbasis analisis ancaman ini menjamin bahwa cakupan *audit event* yang ditentukan tidak bersifat ad hoc, melainkan didasarkan pada risiko keamanan yang terukur dan dapat dipertanggung jawabkan. Pendekatan inilah yang digunakan pada Tugas Akhir ini untuk menentukan jenis-jenis *audit event* yang perlu dicatat pada sistem DecMed, sebagaimana diuraikan pada Bab III.

II.6 Distributed Ledger Technology dan Immutability

II.6.1 DLT sebagai Basis Pencatatan Tidak Berubah

Distributed Ledger Technology (DLT) merupakan sistem terdesentralisasi yang memungkinkan data direkam dan disimpan pada sejumlah *node* dalam suatu jaringan, sehingga seluruh salinan *ledger* tetap tersinkronisasi dan konsisten satu sama lain (Purnama dkk., 2026). Karakteristik desentralisasi ini menghilangkan kebutuhan akan otoritas terpusat, sehingga meningkatkan transparansi dalam sistem, dan menjadikan DLT banyak digunakan di berbagai sektor, termasuk keuangan, *supply chain*, kesehatan, dan energi (Purnama dkk., 2026).

Salah satu karakteristik utama DLT yang relevan bagi kebutuhan audit log adalah *immutability*, yaitu sifat data yang telah tersimpan pada *ledger* tidak dapat diubah maupun dihapus setelah tercatat. Karakteristik ini menjadikan DLT sebagai salah satu teknologi yang cocok untuk diintegrasikan dengan mekanisme kontrol akses

berbasis *capability* (CapBAC), karena DLT dapat berfungsi sebagai media penyimpanan bagi *capability* yang membutuhkan jaminan integritas (Purnama dkk., 2026).

II.6.2 IOTA

IOTA merupakan salah satu jenis DLT yang mengadopsi struktur jaringan berbasis *Directed Acyclic Graph* (DAG). Struktur jaringan DAG memungkinkan IOTA mendukung *throughput* yang tinggi, karena mampu memvalidasi transaksi baru secara lebih cepat dibandingkan *blockchain* konvensional seperti Bitcoin maupun Ethereum (Purnama dkk., 2026). Pada IOTA Rebased, setiap transaksi dikenakan biaya (*gas fee*) (Purnama dkk., 2026).

IOTA Rebased menandai evolusi signifikan dari protokol IOTA, bertransisi dari struktur Tangle yang mendukung transaksi tanpa biaya dan bergantung pada *co-ordinator* terpusat, menuju *object-based ledger* yang dibangun di atas DAG. Pada *object-based ledger* ini, data direpresentasikan sebagai objek-objek yang memiliki pemilik dan dapat dimanipulasi melalui transaksi, sementara logika bisnis diekspresikan melalui *smart contract* yang ditulis dalam bahasa Move (Purnama dkk., 2026).

Dalam arsitektur DecMed, jaringan IOTA berperan sebagai *trust anchor* yang *immutable* untuk kebijakan kontrol akses. Setiap entri *capability* yang dibuat oleh pasien maupun pembaruan statusnya dicatat sebagai objek pada *smart contract* Move, sehingga dapat diverifikasi oleh PRE Server pada saat permintaan akses masuk (Purnama dkk., 2026). Pada Tugas Akhir ini, jaringan IOTA juga dimanfaatkan sebagai media penyimpanan metadata rotasi berkas audit log, sehingga *hash* integritas setiap berkas arsip log terkunci secara *immutable* dan tidak dapat dimanipulasi.

II.7 Arsitektur Layanan Pendukung dalam Sistem Terdesentralisasi

II.7.1 Pola Layanan Terpisah dalam Arsitektur DecMed

Arsitektur DecMed dirancang mengikuti pola tiga lapis (*three-tier architecture*), yang terdiri atas *Client Layer*, *Proxy Layer*, dan *Storage Layer* (Purnama dkk.,

2026). *Client Layer* berfungsi sebagai antarmuka utama bagi seluruh aktor untuk berinteraksi dengan sistem, dengan tiga aplikasi klien berbeda yang disesuaikan dengan kebutuhan masing-masing peran. *Proxy Layer* berperan sebagai *middleware* keamanan yang menegakkan kebijakan kontrol akses dan mengelola operasi kriptografi seperti Proxy Re-Encryption (PRE), sedangkan *Storage Layer* terbagi menjadi penyimpanan *on-chain* yang menjamin keutuhan *capability* akses, dan penyimpanan *off-chain* yang menangani data klinis terenkripsi dalam volume besar (Purnama dkk., 2026).

Lapisan *Proxy* dan *Storage* pada DecMed diimplementasikan melalui beberapa layanan (*server*) yang terpisah satu sama lain, masing-masing dengan tanggung jawab yang spesifik. Layanan *PRE Server* bertanggung jawab menangani proses *re-encryption*, di mana setiap kali menerima permintaan data, *server* tersebut terlebih dahulu memverifikasi entri *capability* pada jaringan IOTA sebelum melakukan *re-encryption* terhadap data yang diambil dari IPFS untuk pemohon, dengan Redis digunakan untuk menyimpan data sementara seperti *nonce* guna mencegah *replay attack* (Purnama dkk., 2026). Layanan *Gas Station Server* bertanggung jawab mensponsori transaksi pada jaringan IOTA, sehingga aktor yang tidak memiliki kepentingan finansial dapat berinteraksi dengan *ledger* tanpa perlu memiliki *token* (Purnama dkk., 2026).

Pola pemisahan layanan pada DecMed ini menjadi dasar perancangan komponen Audit Log Service (ALS) pada Tugas Akhir ini. ALS dirancang mengikuti pola layanan pendukung (*supporting service*) yang sama dengan PRE Server: berdiri sebagai layanan *backend* independen dengan tanggung jawab yang spesifik, memanfaatkan kembali infrastruktur IPFS dan IOTA yang sudah tersedia, dan tidak mengubah alur bisnis komponen DecMed yang telah ada.

II.7.2 Proxy Re-Encryption dan IPFS

Proxy Re-Encryption (PRE) memanfaatkan *proxy* yang bersifat *semi-trusted* untuk mentransformasikan suatu *ciphertext* menjadi *ciphertext* baru yang isinya tetap sama, namun hanya dapat didekripsi menggunakan *secret key* milik subjek yang dituju (Purnama dkk., 2026). *Ciphertext* awal dihasilkan dari enkripsi data menggunakan *public key* milik *delegator* (pemilik data), dan dapat disimpan pada suatu lokasi penyimpanan seperti *cloud storage* maupun sistem DLT. Ketika *delegatee* (pene-

rima akses) meminta akses terhadap data terenkripsi tersebut, *delegator* membuat *re-encryption key* menggunakan *secret key* miliknya dan *public key* milik *delegatee*; *re-encryption key* tersebut dikirimkan kepada *proxy*, yang kemudian menggunakannya bersama *ciphertext* awal dan *public key* kedua belah pihak untuk menghasilkan *ciphertext* baru yang hanya dapat didekripsi oleh *delegatee* (Purnama dkk., 2026).

InterPlanetary File System (IPFS) adalah suatu sistem berkas terdistribusi *peer-to-peer* yang bertujuan menghubungkan seluruh perangkat komputasi ke dalam satu sistem berkas (Benet, 2014). IPFS menggunakan pengalamatan berbasis konten (*content-based addressing*), yaitu data diidentifikasi dan diambil berdasarkan *hash* kriptografisnya yang disebut *Content Identifier* (CID) (Purnama dkk., 2026). Ketika suatu berkas ditambahkan ke jaringan IPFS, berkas tersebut dipecah menjadi bagian-bagian kecil, di-*hash* secara kriptografis, dan diberikan CID yang unik sebagai penanda permanen dan *immutable* dari data tersebut; apabila isi berkas berubah, *hash* kriptografisnya turut berubah dan menghasilkan CID yang sepenuhnya baru (Purnama dkk., 2026).

Sebuah berkas yang telah di-*pin* pada suatu *node* akan tetap tersimpan pada *node* tersebut selama penanda *pin* tidak dicabut secara eksplisit, terlepas dari seberapa jarang berkas tersebut diakses. Dalam konteks penyimpanan berkas audit log yang membutuhkan retensi jangka panjang, mekanisme *pinning* menjadi prasyarat agar data yang diunggah ke IPFS dapat diandalkan ketersediaannya dalam jangka waktu yang panjang. IPFS Cluster, yang digunakan oleh DecMed untuk penyimpanan data klinis, menyediakan fungsionalitas *pinning* terdistribusi yang memastikan CID yang di-*pin* direplikasi dan dipertahankan pada seluruh *node* dalam klaster, sehingga ketidaktersediaan satu *node* tidak menyebabkan hilangnya data (Purnama dkk., 2026).

II.8 Penelitian Terkait

BAB III

ANALISIS DAN RANCANGAN SOLUSI

III.1 Analisis Masalah

III.1.1 Analisis Sistem DecMed

Sebagaimana dijelaskan pada Subbab II.8, DecMed adalah kerangka kerja manajemen RME terdesentralisasi dengan menggunakan DLT IOTA sebagai penyimpanan *on-chain*, IPFS sebagai penyimpanan *off-chain*, serta PRE untuk mendukung distribusi akses data yang aman Purnama dkk., 2026. Arsitektur ini sudah memberikan fondasi untuk menjaga integritas data dan mengurangi ketergantungan pada penyimpanan terpusat, sebagaimana telah diuraikan pada Subbab II.6.1, II.6.2, dan II.7.1.

Meskipun arsitektur tersebut telah berhasil meningkatkan aspek keamanan penyimpanan data, DecMed lebih berfokus pada pencegahan serangan dan perlindungan data melalui kontrol akses. Aspek audit log sebagai mekanisme untuk menghasilkan, menyimpan, dan mengelola bukti digital selama proses investigasi insiden keamanan masih belum menjadi fokus pembahasan, sebagaimana ditunjukkan pada Subbab II.8, di mana DecMed baru mengklaim ketersediaan *audit log* secara umum tanpa merinci komponen-komponen audit log yang telah diuraikan pada Subbab II.2.3.

III.1.1.1 Implementasi Audit Log pada DecMed

Dalam operasionalnya, implementasi DecMed sebenarnya telah menyediakan beberapa bentuk pencatatan aktivitas sistem. Pencatatan ini dilakukan melalui dua komponen utama, yaitu *PatientAccessLog* yang dikelola di dalam sistem dan *transaction block* bawaan pada jaringan IOTA. Kedua komponen tersebut telah mendokumentasikan sebagian aktivitas yang terjadi selama sistem beroperasi. Meskipun demikian, apabila dinilai terhadap karakteristik audit log yang baik pada Subbab II.2.2, kedua upaya ini belum membentuk sebuah audit log yang utuh dan mampu memenuhi

kebutuhan audit maupun investigasi insiden keamanan karena masing-masing memiliki keterbatasan yang mendasar.

III.1.1.2 Keterbatasan *PatientAccessLog*

PatientAccessLog digunakan untuk mencatat aktivitas pemberian hak akses dari pasien kepada tenaga medis. Informasi yang tersimpan telah mencakup atribut penting seperti identitas pasien, identitas tenaga medis, waktu pemberian akses (*timestamp*), serta informasi lain yang berkaitan dengan hak akses tersebut.

Namun demikian, *PatientAccessLog* memiliki dua keterbatasan kritis sebagai sebuah komponen audit log. Pertama, cakupan pencatatannya masih sangat terbatas pada proses pemberian hak akses. Berbagai aktivitas lain yang memiliki nilai audit tinggi, seperti autentikasi, otorisasi, perubahan konfigurasi, maupun interaksi kueri terhadap data rekam medis belum terdokumentasi. Hal ini bertentangan dengan cakupan minimal peristiwa pemicu (*trigger events*) yang telah diuraikan pada Subbab II.2.3.1. Kedua, mekanisme ini rentan terhadap perubahan sehingga tidak memenuhi karakteristik kekekalan data (*append-only*). *PatientAccessLog* dikaitkan secara langsung dengan fungsionalitas pencabutan hak akses (*revoke access*). Ketika hak akses dicabut, informasi pada entri log yang sama akan diperbarui dengan menimpa (*overwrite*) status akses sebelumnya menjadi *revoked*. Pendekatan ini melanggar karakteristik integritas audit log pada Subbab II.2.2, yang mensyaratkan bahwa catatan historis tidak boleh berubah dan aktivitas baru seharusnya dicatat sebagai *audit record* terpisah sesuai definisi pada Subbab II.2.1.

III.1.1.3 Keterbatasan *Transaction Block* IOTA

Selain pencatatan internal, DecMed memanfaatkan *transaction block* pada jaringan IOTA sebagai bukti transaksi. Setiap pemanggilan fungsi *smart contract* akan menghasilkan *transaction block* yang tersimpan secara terdistribusi. Mekanisme ini memberikan jaminan integritas yang tinggi karena data bersifat *immutable* dan sulit dimodifikasi, sejalan dengan karakteristik DLT pada Subbab II.6.1.

Meskipun unggul dalam hal integritas, pemanfaatan *transaction block* IOTA sebagai sumber audit log masih memiliki beberapa kelemahan utama. Kelemahan pertama adalah inkonsistensi format data, di mana *audit record* yang dihasilkan belum terstandarisasi karena struktur data transaksi sangat bervariasi bergantung pada jenis

fungsi atau operasi spesifik yang sedang dieksekusi oleh sistem. Selanjutnya, terdapat keterbatasan pada variasi *event* yang dicatat. Cakupan peristiwa yang terekam sangat sempit karena komponen ini hanya mendokumentasikan eksekusi operasi *move call* pada *smart contract*, sehingga berbagai interaksi sistem yang tidak memicu pemanggilan fungsi tersebut akan luput dari pencatatan. Kelemahan terakhir berkaitan dengan permasalahan retensi data. Ketersediaan bukti digital dalam jangka panjang tidak dapat dijamin secara pasti karena data transaksi pada *node* IOTA rentan terhapus akibat mekanisme *data pruning* yang rutin dilakukan untuk membebaskan ruang penyimpanan *node*. Kondisi ini membuat pemanfaatan *transaction block* belum memenuhi persyaratan retensi komponen penyimpanan *event* sebagaimana diuraikan pada Subbab II.2.3.4.

III.1.1.4 Keterbatasan Pencatatan Audit Log yang Tersebar pada DecMed

Implementasi pencatatan aktivitas pada DecMed saat ini masih bersifat tersebar dan terisolasi pada masing-masing komponen internal tanpa adanya pengelola terpusat. Kondisi arsitektur pencatatan yang terfragmentasi ini menimbulkan sejumlah keterbatasan kritis yang menyulitkan pelaksanaan pembuktian dan investigasi digital ketika terjadi insiden keamanan.

Keterbatasan pertama terletak pada fragmentasi format dan ketiadaan standar log. Karena setiap komponen penghasil kejadian (*event source*) memproduksi log secara independen, skema dan atribut data yang dihasilkan menjadi sangat heterogen. Ketiadaan standarisasi ini secara langsung menyulitkan proses korelasi dan analisis silang (*cross-analysis*) antar-komponen secara otomatis. Selain itu, sistem pencatatan yang tersebar mengandalkan pewaktu lokal dari masing-masing lingkungan (*environment*) operasional. Perbedaan sumber pewaktu ini memunculkan risiko kejanggalan waktu (*time-skew*) antar-log, sehingga rekonstruksi kronologi kejadian yang presisi menjadi sangat sulit untuk dilakukan.

Keterbatasan selanjutnya adalah kesulitan dalam melakukan agregasi bukti digital secara menyeluruh. Tidak tersedianya layanan pengumpul terpusat (*centralized collector*) menyebabkan pemantauan aktivitas sistem tidak dapat dilakukan secara holistik, dan pengumpulan bukti harus ditarik secara parsial dari berbagai tempat yang berbeda. Hal ini diperparah oleh ketiadaan strategi retensi data yang seragam. Komponen yang tersebar memiliki mekanisme dan batas kapasitas penyimpanan

masing-masing. Sebagian catatan audit mungkin tersimpan di basis data lokal tanpa jaminan perlindungan terhadap manipulasi, sementara catatan lainnya yang berada pada jaringan DLT rentan mengalami penghapusan permanen akibat mekanisme pembersihan (*pruning*) secara berkala.

III.1.2 Identifikasi Masalah

Berdasarkan analisis terhadap implementasi audit log DecMed pada Subbab III.1.1, sejumlah masalah diidentifikasi agar DecMed dapat mendukung pembentukan dan pengelolaan audit log secara memadai. Permasalahan utama pada sistem saat ini adalah pencatatan aktivitas yang masih tersebar sehingga tidak sinkron dalam berbagai aspek dan masih memiliki kekurangan masing-masing.

Selain masalah utama tersebut, terdapat beberapa masalah turunan yang perlu diperhatikan agar rancangan solusi dapat ditegakkan secara andal. Masalah turunan ini bersumber dari keterbatasan masing-masing komponen pencatatan bawaan. Pertama, cakupan *event* masih sangat terbatas sehingga belum mampu merekam aktivitas bernilai audit tinggi secara menyeluruh. Kedua, data log pada penyimpanan internal rentan mengalami perubahan atau tertimpa (*overwrite*) seiring perubahan status, sehingga melanggar prinsip *append-only*. Ketiga, format data transaksi dari DLT belum terstandarisasi. Keempat, mekanisme penyimpanan jaringan DLT rentan mengalami *data pruning* yang mengancam ketersediaan data dalam jangka panjang.

Agar keterhubungan antara analisis masalah, kebutuhan sistem, rancangan, implementasi, dan pengujian dapat ditelusuri, permasalahan tersebut dirinci menggunakan ID M-TA pada Tabel III.1.

Berdasarkan Tabel III.1, keterbatasan pencatatan aktivitas pada DecMed tidak hanya dipandang sebagai persoalan komponen secara individual, tetapi sebagai rangkaian masalah yang saling bergantung. M-TA-01 merupakan masalah inti yang berkaitan dengan absennya wadah pengelola terpusat. Sementara itu, M-TA-02, M-TA-03, M-TA-04, dan M-TA-05 merupakan masalah turunan yang perlu diselesaikan agar audit log yang dikembangkan dapat menjamin integritas data, konsistensi format, dan retensi penyimpanan jangka panjang. ID masalah tersebut kemudian digunakan sebagai dasar pemetaan kebutuhan, rancangan, implementasi, dan pe-

Tabel III.1: Identifikasi Masalah Audit Log DecMed

ID	Masalah	Dasar Analisis
M-TA-01	Belum adanya mekanisme audit log yang terintegrasi untuk mengelola bukti digital secara terpusat.	III.1.1.4
M-TA-02	Cakupan pencatatan <i>event</i> masih sempit dan belum mencakup keseluruhan aktivitas bernilai audit tinggi.	III.1.1.2 & III.1.1.3
M-TA-03	<i>Audit record</i> masih bisa mengalami modifikasi (<i>overwrite</i>) sehingga tidak bersifat <i>append-only</i> .	III.1.1.2
M-TA-04	Format <i>audit record</i> masih belum terstandarisasi sehingga menyulitkan proses pembacaan.	III.1.1.3
M-TA-05	Bukti digital rentan terhapus akibat <i>data pruning</i> sehingga tidak menjamin retensi jangka panjang.	III.1.1.3

ngujian pada bagian selanjutnya.

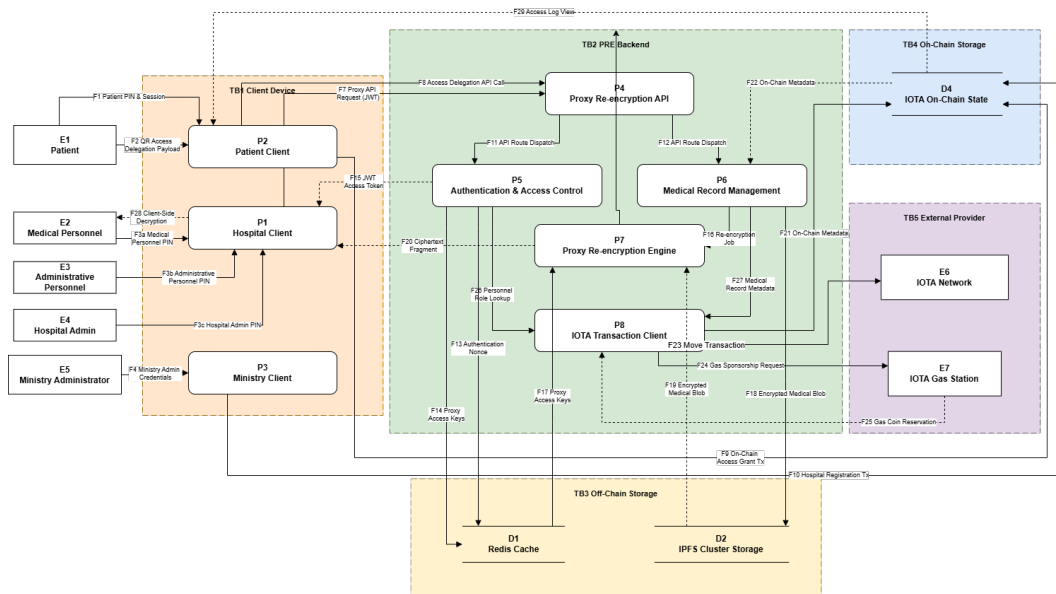
III.1.3 Kebutuhan *Audit*

Dalam merancang sebuah subsistem *audit log* yang andal, langkah pertama yang harus dilakukan adalah mengidentifikasi peristiwa (*event*) apa saja yang esensial untuk direkam sebagai bukti digital. Penentuan kebutuhan *audit event* ini tidak dilakukan secara acak, melainkan didasarkan pada pendekatan berbasis risiko. Oleh karena itu, analisis pada subbab ini diawali dengan mengevaluasi potensi ancaman keamanan pada arsitektur DecMed menggunakan pemodelan ancaman, yang hasilnya kemudian dipetakan menjadi daftar klasifikasi *event* beserta atribut bukti yang dibutuhkan untuk keperluan mitigasi dan investigasi.

III.1.3.1 Dekomposisi Aplikasi

Dekomposisi aplikasi merupakan tahap awal *threat modeling* untuk memahami arsitektur sistem DecMed dan interaksinya dengan entitas eksternal. Tahap ini mencakup identifikasi dependensi eksternal, titik masuk (*entrypoints*), aset, serta level kepercayaan berbasis pendekatan gabungan (*role* dan *capability*). Rincian hasil identifikasi masing-masing komponen disajikan pada Lampiran, yang meliputi dependensi eksternal (Tabel A.1), titik masuk (Tabel A.2), aset (Tabel A.3), dan level

kepercayaan (Tabel A.4). Selanjutnya, komponen-komponen ini dipetakan ke dalam *Data Flow Diagram* (DFD) pada Gambar III.1 untuk menggambarkan aliran data antara entitas, proses, dan penyimpanan data.



Gambar III.1: Data Flow Diagram Sistem DecMed

III.1.3.2 Identifikasi Ancaman DecMed

Identifikasi ancaman pada sistem DecMed dilakukan menggunakan metode STRIDE terhadap elemen-elemen yang dipetakan pada *Data Flow Diagram* (Gambar III.1). Setiap elemen dan aliran data dianalisis untuk mengidentifikasi potensi ancaman berdasarkan enam kategori STRIDE: *Spoofing*, *Tampering*, *Repudiation*, *Information Disclosure*, *Denial of Service*, dan *Elevation of Privilege*.

Elemen E6 IOTA Network dan E7 IOTA Gas Station pada TB5 External Provider diperlakukan sebagai infrastruktur eksternal tepercaya di luar kendali sistem DecMed. Oleh karena itu, potensi ancaman terhadap operasional internal keduanya berada di luar cakupan *threat model* ini, dan analisis dibatasi hanya pada interaksinya dengan komponen sistem (melalui proses P8).

Hasil identifikasi dan klasifikasi ancaman berbasis STRIDE pada sistem DecMed secara lengkap disajikan pada Tabel B.1.

III.1.3.3 Identifikasi Audit Event

Berdasarkan hasil analisis ancaman menggunakan metode STRIDE pada III.1.3.2, tahap selanjutnya adalah menentukan kebutuhan audit log yang diperlukan untuk

menghasilkan bukti digital terhadap setiap ancaman yang teridentifikasi. Kebutuhan tersebut diperoleh dengan memetakan setiap ancaman ke aktivitas sistem yang perlu dicatat sebagai audit event. Hasil pemetaan antara ancaman, kebutuhan bukti digital, dan audit event yang dihasilkan ditunjukkan pada Tabel III.2.

Tabel III.2: Pemetaan Ancaman ke Audit Event

Event ID	Event Name	Threat	Bukti yang Dibutuhkan
EV1	Authentication	T1, T2, T3	Identitas aktor yang melakukan autentikasi, waktu autentikasi, objek autentikasi, metode autentikasi, hasil autentikasi, perangkat yang digunakan, serta frekuensi percobaan autentikasi.
EV2	QR Access Delegation Validation	T4, T5	Identitas aktor yang membuat dan memindai QR, waktu validasi, objek QR Access Delegation, hasil validasi, serta payload QR Access Delegation.
EV3	Capability Creation	T5,T6	Identitas aktor yang membuat <i>capability</i> , waktu pembuatan, objek <i>capability</i> dan akses yang didelegasikan, penerima akses, identitas pasien, hasil pembuatan, serta <i>transaction digest</i> .
EV4	Medical Record Access	T7, T13	Identitas aktor yang mengakses rekam medis, waktu akses, objek rekam medis, jenis operasi (<i>read/write</i>), <i>capability</i> yang digunakan, hasil validasi otorisasi, serta hasil operasi akses.

Event ID	Event Name	Threat	Bukti yang Dibutuhkan
EV5	Hospital Personnel Activation Key Generation	T8	Identitas aktor yang menghasilkan activation key, waktu pembuatan, objek activation key dan personel yang terkait, identitas Admin Fasyankes, identitas personel, serta hasil pembuatan activation key.
EV6	Healthcare Facility Registration	T9	Identitas aktor yang melakukan registrasi, waktu registrasi, objek fasilitas pelayanan kesehatan, identitas Ministry Administrator, identitas fasyankes yang diregistrasi, serta hasil registrasi.
EV7.1	PRE Service Request	T10, T11, T12	Identitas aktor atau pemohon, waktu permintaan, objek layanan PRE yang diminta, endpoint tujuan, jenis layanan yang diminta, serta hasil pemrosesan permintaan.
EV7.2	PRE Service Response	T10, T11, T12	Identitas aktor atau pemohon, waktu permintaan, objek layanan PRE yang diminta, endpoint tujuan, jenis layanan yang diminta, serta hasil pemrosesan permintaan.
EV8	IOTA Transaction Submission	T14	Identitas aktor atau penandatanganan, waktu pengiriman, objek transaksi IOTA, <i>transaction digest</i> , payload transaksi, hasil pengiriman transaksi, serta status konfirmasi pada jaringan IOTA.

Event ID	Event Name	Threat	Bukti yang Dibutuhkan
EV9	Gas Sponsorship Request	T15, T16, T17	Identitas aktor atau pemohon sponsorship, waktu permintaan, objek transaksi yang diajukan untuk sponsorship, <i>transaction digest</i> , kebutuhan gas transaksi, serta hasil pengajuan sponsorship.
EV10	Redis Operation	T18, T19, T20	Identitas aktor atau proses yang melakukan operasi, waktu operasi, objek Redis yang diakses, jenis operasi, hasil operasi, serta informasi TTL apabila relevan.
EV11	IPFS Object Access	T21, T22	Identitas aktor atau pemohon, waktu akses, objek IPFS yang diakses berupa Content Identifier (CID), jenis operasi, hasil operasi, serta ukuran data apabila relevan.
EV12	On-chain Metadata Access	T23	Identitas aktor atau pemohon, waktu akses, objek metadata <i>on-chain</i> yang diakses, jenis operasi akses, serta hasil operasi.
EV13	Capability Validation	T24, T25	Identitas aktor atau pemohon, waktu validasi, objek <i>capability</i> yang divalidasi berupa Capability ID, hasil validasi otorisasi, <i>required scope</i> , <i>actual scope</i> , serta alasan penolakan apabila validasi gagal.

III.1.4 Kebutuhan Sistem

Berdasarkan masalah yang telah dipetakan pada Tabel III.1, kebutuhan sistem pada Tugas Akhir ini dirumuskan khusus untuk mengembangkan mekanisme audit log terintegrasi yang mampu mengelola, memvalidasi, dan menyimpan bukti digital

dari berbagai komponen DecMed. Kebutuhan ini dirancang untuk menyelesaikan masalah belum terintegrasikannya log aktivitas, keterbatasan cakupan *event*, kerentanan modifikasi log, ketidakstandaran format, serta ancaman *data pruning* pada jaringan DLT.

Kebutuhan sistem dibagi menjadi kebutuhan fungsional dan kebutuhan nonfungsional. Kebutuhan fungsional menjelaskan kemampuan otomatis yang harus disediakan oleh sistem dalam menangkap, memformat, dan menyimpan *audit record*. Sementara itu, kebutuhan nonfungsional menjelaskan batasan kualitas, integritas, dan ketersediaan data log dalam jangka panjang.

III.1.4.1 Kebutuhan Fungsional

Kebutuhan fungsional dirumuskan dari sudut pandang interaksi antar-proses (*system/process actors*) antara komponen internal DecMed dan repositori audit log. Rangkuman kebutuhan fungsional ditunjukkan pada Tabel III.3.

Tabel III.3: Kebutuhan Fungsional Sistem Audit Log

ID	Deskripsi Kebutuhan
KF-TA-01	Sistem harus dapat mengagregasi dan mengelola data log dari berbagai komponen DecMed ke dalam satu repositori audit log terpusat.
KF-TA-02	Sistem harus dapat menangkap dan merekam keseluruhan jenis <i>event</i> yang sudah ditentukan pada Tabel III.2.
KF-TA-03	Sistem harus memformat dan menstandarisasi <i>audit record</i> ke dalam skema data terstruktur sebelum disimpan.
KF-TA-04	Sistem harus menyimpan <i>audit record</i> ke dalam media penyimpanan yang bersifat <i>append-only</i> untuk mencegah modifikasi (<i>overwrite</i>).
KF-TA-05	Sistem harus menyediakan mekanisme penyimpanan cadangan/persistensi (<i>long-term retention</i>) untuk mengantisipasi kehilangan data akibat <i>pruning</i> pada jaringan DLT.

III.1.4.2 Kebutuhan Nonfungsional

Kebutuhan nonfungsional difokuskan untuk memastikan atribut kualitas keamanan, integritas, dan keandalan data log. Rangkuman kebutuhan nonfungsional ditunjukkan pada Tabel III.4.

Tabel III.4: Kebutuhan Nonfungsional Sistem Audit Log

ID	Deskripsi Kebutuhan
KNF-TA-01	Integritas: Sistem harus menjamin bahwa data <i>audit record</i> yang telah disimpan bersifat <i>tamper-evident</i> .
KNF-TA-02	Interoperabilitas & Keterbacaan: Format data log yang dihasilkan harus konsisten, terstruktur, dan mudah dibaca/diproses oleh komponen sistem.
KNF-TA-03	Ketersediaan Data: Sistem harus menjamin ketersediaan dan keteraksesan data audit log dalam jangka panjang.
KNF-TA-04	Konsistensi Pencatatan: Sistem harus memastikan setiap <i>event</i> yang terjadi dicatat secara presisi tanpa ada kejadian bernilai audit tinggi yang terlewat.

III.1.4.3 Pemetaan Masalah terhadap Kebutuhan Sistem

Untuk memastikan bahwa setiap masalah yang diidentifikasi pada Tabel III.1 memiliki solusi yang sesuai, dilakukan pemetaan terhadap kebutuhan fungsional dan nonfungsional. Hasil pemetaan ditunjukkan pada Tabel III.5.

Tabel III.5: Pemetaan Masalah terhadap Kebutuhan Sistem Audit Log

ID Masalah	Kebutuhan Fungsional	Kebutuhan Nonfungsional
M-TA-01	KF-TA-01	KNF-TA-02, KNF-TA-04
M-TA-02	KF-TA-02	KNF-TA-04
M-TA-03	KF-TA-04	KNF-TA-01
M-TA-04	KF-TA-03	KNF-TA-02
M-TA-05	KF-TA-05	KNF-TA-03

Tabel III.5 menunjukkan bahwa seluruh masalah (M-TA-01 hingga M-TA-05) telah ditangani secara langsung oleh kombinasi kebutuhan fungsional dan nonfungsional. Pemetaan ini menjadi dasar penentuan rancangan arsitektur dan skenario pengujian pada bab selanjutnya.

III.1.5 Analisis Solusi

Guna mengatasi permasalahan pencatatan dan pengelolaan bukti digital pada De-cMed yang telah dipetakan pada Subbab III.1.1, diperlukan pengembangan mekanisme audit log yang terintegrasi. Berdasarkan kebutuhan sistem pada Subbab III.1.3, analisis solusi disusun dalam urutan berikut. Pertama, analisis pendekatan arsitektur agregasi log untuk menyatukan pencatatan terpisah. Kedua, analisis

skema pembentukan struktur data audit terstandar. Ketiga, analisis mekanisme penyimpanan *append-only* dan *tamper-evident* pada berkas log lokal. Keempat, analisis strategi persistensi data (*long-term retention*) untuk mengantisipasi risiko *data pruning* pada jaringan DLT. Analisis ini menjadi dasar pemilihan pendekatan sebelum rancangan teknis dijelaskan pada Subbab III.3.

III.1.5.1 Analisis Event Source

Sebelum menganalisis mekanisme audit log, perlu ditentukan sumber-sumber dari audit tersebut. Sumber event akan ditentukan berdasarkan kebutuhan event yang sudah diidentifikasi pada Tabel III.2. Pemetaan *event* ke *event source* dapat dilihat pada Tabel D.1.

Berdasarkan hasil pemetaan tersebut, terdapat empat komponen utama yang bertindak sebagai *event source*, yaitu *Patient Client*, *Hospital Client*, *Ministry Client*, dan PRE Server. Setiap komponen ini memiliki tanggung jawab untuk menghasilkan log aktivitas sesuai perannya masing-masing. Distribusi *event* berdasarkan *event source* dapat dilihat pada Tabel III.6.

Tabel III.6: Distribusi *Audit Event* Berdasarkan *Source*

<i>Source</i>	<i>Jumlah Event</i>	<i>Event yang Berasal dari Source</i>
PRE Server	7	EV7, EV8, EV9, EV10, EV11, EV12, EV13
Patient Client	5	EV1, EV2, EV3, EV8, EV9
Hospital Client	4	EV1, EV4, EV5, EV8, EV9
Ministry Client	3	EV6, EV8, EV9

III.1.5.2 Analisis Arsitektur Agregasi dan Pengumpulan Log

Hasil analisis masalah menunjukkan bahwa log aktivitas pada DecMed masih dikelola secara parsial oleh masing-masing komponen internal (seperti *PatientAccess-Log* pada basis data lokal dan *Transaction Block* pada DLT). Kondisi ini menyebabkan absennya wadah pengelola terpusat (M-TA-01) serta sempitnya cakupan *event* bernilai audit tinggi yang dapat terekam (M-TA-02).

Terdapat dua pendekatan yang dipertimbangkan untuk mengagregasi data log dari berbagai komponen sistem. Perbandingan kedua pendekatan tersebut dirangkum

pada Tabel III.7.

Tabel III.7: Analisis Pendekatan Arsitektur Pengumpulan Log

Pendekatan	Keunggulan	Keterbatasan
<i>Decentralized Point-to-Point</i>	Tidak memerlukan komponen perantara (<i>middleware</i>), setiap komponen menyimpan log masing-masing secara independen.	Sulit merekonstruksi kronologi kejadian secara utuh, format log bervariasi, dan cakupan <i>event</i> terbatas pada operasi komponen lokal.
<i>Centralized Audit Collector</i>	Menyediakan repositori tunggal untuk mengumpulkan, memvalidasi, dan menyelaraskan <i>event</i> dari seluruh komponen.	Membutuhkan komponen <i>ingestion</i> terpisah dan penyesuaian pengiriman log pada tiap komponen DecMed.

Berdasarkan Tabel III.7, pendekatan *Centralized Audit Collector* dipilih. Pendekatan ini memungkinkan seluruh event penting yang dikirim dari *event source* ditangkap melalui satu pintu sebelum diproses lebih lanjut, sehingga menyelesaikan masalah M-TA-01 dan M-TA-02.

III.1.5.3 Analisis Mekanisme Pengiriman Log

Setelah menentukan sumber kejadian (*event source*) dan memilih pendekatan *Centralized Audit Collector*, langkah selanjutnya adalah menganalisis mekanisme pengiriman log dari setiap *event source* ke komponen pengumpul. Mekanisme ini harus mampu menjamin keandalan pengiriman log tanpa mengganggu performa layanan utama DecMed.

Dalam menentukan mekanisme pengiriman, dianalisis dua dimensi teknis pengiriman data. Pertama, dari segi pola komunikasi, dipertimbangkan antara pendekatan *Pull-based (polling)* dan *Push-based*. Pendekatan *Pull-based* memberikan kendali beban bagi pengumpul, tetapi menimbulkan latensi pencatatan dan memaksa *event source* menyediakan penyimpanan sementara lokal. Sebaliknya, pendekatan *Push-based* memungkinkan log dikirimkan secara langsung sesaat setelah kejadian berlangsung. Kedua, dari segi sifat eksekusi, dipertimbangkan antara *Synchronous* dan *Asynchronous*. Pengiriman *Synchronous* memastikan log diterima sebelum transaksi utama selesai, tetapi menambah latensi dan berisiko menggagalkan transaksi utama jika pengumpul bermasalah. Sementara itu, pengiriman *Asynchronous* bersifat *non-blocking* sehingga tidak mengganggu *response time* transaksi klinis maupun

kriptografis utama.

III.1.5.4 Analisis Keaslian dan Pencegahan *Event* Palsu

Penggunaan protokol komunikasi terenkripsi seperti TLS/HTTPS hanya mengamankan jalur pengiriman (*transport layer*), tetapi belum memberikan jaminan bahwa *event* yang diterima ALS benar-benar berasal dari komponen sistem yang sah. Tanpa mekanisme verifikasi di tingkat aplikasi, ALS tidak dapat membedakan *event* yang dikirimkan oleh komponen DecMed yang legitim dengan *event* palsu yang disisipkan oleh pihak tidak berwenang. Untuk mengatasi keterbatasan tersebut, setiap *event source* diwajibkan menandatangani *payload event* sebelum dikirim ke ALS, sehingga ALS dapat memverifikasi keaslian setiap *event* yang masuk sebelum dicatat ke dalam audit log.

Mekanisme penandatanganan yang dipilih adalah tanda tangan digital berbasis kunci privat IOTA (*IotaKeyPair*) milik aktor yang terlibat pada saat kejadian berlangsung. Pendekatan ini memiliki keunggulan dibandingkan alternatif seperti penggunaan kunci Ed25519 yang dibangkitkan secara acak pada saat inisialisasi komponen: kunci privat IOTA sudah terikat pada identitas aktor yang terdaftar pada *ledger* IOTA, sehingga tanda tangan yang dihasilkan dapat diverifikasi secara kriptografis terhadap identitas yang terregistrasi pada sistem. Dengan demikian, ALS dapat memastikan bahwa setiap *event* yang masuk ke dalam audit log berasal dari komponen atau aktor yang memang terdaftar dalam sistem DecMed, bukan dari sumber yang tidak dikenal atau tidak berwenang.

Dengan menerapkan mekanisme *signed payload* berbasis kunci IOTA aktor, ALS dapat menolak seluruh *event* yang tidak memiliki tanda tangan valid sebelum diproses lebih lanjut. Hal ini menjamin bahwa isi audit log hanya memuat catatan aktivitas yang berasal dari sumber yang terverifikasi, sehingga keaslian dan keandalan audit log sebagai catatan aktivitas sistem dapat terjaga.

III.1.5.5 Analisis Kerahasiaan Log dalam Transit

Selain integritas dan *non-repudiation*, *audit event* yang dikirimkan dari *event source* ke ALS berpotensi mengandung informasi sensitif, seperti identitas aktor, identitas objek yang diakses, maupun detail operasi yang dilakukan. Apabila *event* dikirimkan dalam bentuk *plaintext* atau hanya mengandalkan enkripsi pada lapisan transport

(TLS), maka ALS sebagai pihak penerima akan memiliki akses penuh terhadap isi *event* pada saat penerimaan. Pemisahan antara kemampuan menerima *event* dan kemampuan membaca isi *event* menjadi kebutuhan yang perlu dipertimbangkan, terutama untuk memastikan bahwa log yang tersimpan tidak dapat dibaca oleh pihak yang seharusnya hanya berperan sebagai pengumpul dan penyimpanan, bukan pembaca.

Untuk memenuhi kebutuhan ini, diterapkan mekanisme enkripsi *end-to-end* pada level aplikasi menggunakan skema *hybrid encryption*. Setiap *payload audit event* dienkripsi menggunakan AES-256-GCM dengan kunci simetris yang dibangkitkan secara acak untuk setiap pengiriman. Kunci AES tersebut kemudian dienkripsi menggunakan kunci publik P-256 milik ALS melalui mekanisme ECIES (*Elliptic Curve Integrated Encryption Scheme*) yang menggabungkan pertukaran kunci Diffie-Hellman efemeral dan fungsi derivasi kunci berbasis SHA-256. Hanya ALS yang memiliki kunci privat P-256 yang dapat mendekripsi kunci AES dan selanjutnya mendekripsi isi *event*. Urutan operasi yang diterapkan adalah *Encrypt-then-Sign*: *payload* dienkripsi terlebih dahulu, kemudian *ciphertext* yang dihasilkan ditandatangani menggunakan kunci privat IOTA aktor. Urutan ini dipilih karena tanda tangan atas *ciphertext* membuktikan bahwa pengirim adalah pihak yang membuat *ciphertext* tersebut, bukan sekadar meneruskan ciphertext milik pihak lain.

Konsekuensi dari penerapan enkripsi *end-to-end* ini adalah bahwa berkas log yang tersimpan secara lokal di ALS maupun yang diarsipkan ke IPFS memuat *audit event* dalam bentuk terenkripsi. Dekripsi hanya dilakukan pada saat pembacaan log untuk keperluan audit oleh pihak yang berwenang dengan menggunakan kunci privat ALS, bukan pada saat penerimaan *event*. Pendekatan ini memisahkan tanggung jawab antara pengumpulan bukti (*collection*) dan pembacaan bukti (*inspection*), serta memberikan jaminan kerahasiaan isi *audit event* selama masa penyimpanan.

III.1.5.6 Analisis Standardisasi Skema *Audit Record*

Data log aktivitas pada DecMed eksisting memiliki format yang heterogen dan tidak terstandarisasi, sehingga menyulitkan proses pencarian, validasi, dan analisis keterlacakan bukti digital (M-TA-04). Untuk mengatasi masalah tersebut, diperlukan analisis terhadap pendekatan skema data yang mampu menyelaraskan format rekaman log dari berbagai sumber.

Berdasarkan tabel III.2 dapat dilihat bahwa ada bukti yang dibutuhkan oleh semua event dan ada yang unik untuk masing-masing event. Maka dari itu standarisasi skema dirancang menggunakan pendekatan bertingkat yang memisahkan antara atribut umum dan rincian kejadian. Pendekatan ini diwujudkan melalui dua tingkatan skema utama:

1. Skema Umum (*General Audit Record Schema*): Berfungsi sebagai pembungkus standar untuk seluruh jenis log audit. Skema ini menetapkan atribut universal yang wajib dimiliki oleh setiap rekaman.
2. Skema Detail (*Detailed Audit Event Schema*): Berfungsi sebagai struktur data dinamis yang mengkapsulasi informasi spesifik sesuai dengan jenis kejadian yang terjadi. Dengan memisahkan skema detail dari skema umum, sistem dapat mengakomodasi berbagai karakteristik log tanpa merusak konsistensi struktur utama rekaman audit.

Melalui analisis pemisahan skema umum dan skema detail ini, seluruh log yang dihasilkan DecMed dapat diformat secara konsisten dan terstruktur, sehingga menyelesaikan masalah M-TA-04.

III.1.5.7 Analisis Repositori Audit Log

Penyimpanan data log pada DecMed saat ini belum sepenuhnya menjamin sifat *append-only* dan *tamper-evident* (M-TA-03). Pada implementasi eksisting, komponen `PatientAccessLog` disimpan pada *smart contract* Move, namun struktur kode yang ada masih menyediakan fungsi untuk memperbarui (*update*) data log sehingga bersifat *mutable*. Sementara itu, komponen TX Block disimpan secara langsung pada jaringan IOTA.

Untuk membangun repositori bukti audit yang *tamper-evident* tanpa membebani ukuran penyimpanan *on-chain*, dianalisis pemanfaatan infrastruktur IPFS yang sudah tersedia pada DecMed dikombinasikan dengan *smart contract* Move di jaringan IOTA. Perbandingan pendekatan penyimpanan dirangkum pada Tabel III.8.

Berdasarkan analisis pada Tabel III.8, pendekatan IPFS + Move Smart Contract dipilih sebagai solusi. Mekanismenya dirancang sebagai berikut:

1. Berkas Audit Log Utama (*Off-chain* IPFS): Seluruh *payload* dan berkas *au-*

Tabel III.8: Analisis Pendekatan Penyimpanan Audit Log

Pendekatan	Penjelasan	Keunggulan	Keterbatasan
Penyimpanan <i>On-Chain</i> Penuh	Seluruh <i>payload</i> dan data log disimpan langsung ke dalam <i>smart contract</i> Move pada jaringan IOTA.	Data log tersimpan langsung di DLT tanpa keterangan pada media penyimpanan luar.	<i>PatientAccessLog</i> eksisting masih memiliki fungsi modifikasi yang melanggar <i>append-only</i> , serta biaya <i>gas</i> dan konsumsi <i>storage</i> sangat tinggi.
IPFS + Move	<i>File</i> log utuh disimpan di IPFS, sedangkan <i>hash</i> IPFS (CID) dan metadata verifikasi dikunci di <i>On-Chain</i> .	IPFS lebih cocok untuk berkas besar dan mengunci metadata secara <i>immutable</i> tanpa fungsi pembaruan.	Membutuhkan mekanisme integrasi antara <i>node</i> IPFS dan repositori Move untuk sinkronisasi metadata.

dit record terstruktur akan disimpan ke dalam IPFS. Pendekatan ini memanfaatkan infrastruktur IPFS yang sudah ada pada DecMed untuk menangani penyimpanan berkas log berukuran besar secara efisien.

2. Metadata Verifikasi (*On-chain* Move): *Content Identifier* (CID) dari IPFS beserta metadata penting (seperti *timestamp*, ID transaksi, dan *digest* integritas) disimpan ke dalam *smart contract* Move pada jaringan IOTA.

Pada *smart contract* Move yang baru, tidak akan ada fungsi modifikasi/pembaruan log dan struktur objek dirancang sebagai *immutable object*. Dengan demikian, jika ada upaya mengubah isi berkas log di IPFS, *hash* berkas tersebut tidak akan cocok lagi dengan metadata yang terkunci di *smart contract* Move, sehingga jaminan integritas dan *tamper-evident* bukti digital (M-TA-03) dapat ditegakkan.

III.1.5.8 Analisis Mekanisme Penyimpanan Berkas Log

Setelah menentukan pendekatan arsitektur pengumpulan, langkah selanjutnya adalah menganalisis mekanisme penyimpanan berkas log di IPFS dan pencatatan metadatanya di dalam *smart contract* Move. Mekanisme ini harus mampu menjamin integritas, keterlacakan, dan ketersediaan data log dalam jangka panjang secara efisien.

Secara konseptual, audit log yang ideal harus mengadopsi prinsip *append-only*, di

mana rekaman kejadian baru hanya dapat ditambahkan pada bagian akhir berkas tanpa mengubah maupun menghapus riwayat sebelumnya. Namun, pemanfaatan IPFS sebagai media penyimpanan utama menghadirkan tantangan teknis tersendiri terkait sifat kekekalan (*immutability*) datanya. IPFS menggunakan sistem pengalamatan berbasis konten (*content-addressed storage*), yang berarti setiap berkas akan direpresentasikan oleh *Content Identifier* (CID) yang dihasilkan dari fungsi *hash* terhadap isi berkas tersebut. Dengan arsitektur ini, IPFS secara bawaan tidak mendukung operasi modifikasi atau penambahan data (*append*) pada berkas yang telah dipublikasikan; modifikasi apa pun akan menghasilkan entitas berkas baru dengan nilai CID yang sepenuhnya berbeda.

Untuk mengatasi batasan arsitektur IPFS tersebut, sistem tidak dimungkinkan untuk mengunggah *audit record* satu per satu secara langsung sesaat setelah *event* diterima. Sebagai solusinya, diperlukan mekanisme pengelompokan (*batching*) dan perputaran log (*log rotation*) pada sisi komponen pengumpul. Layanan audit log tersentralisasi akan menampung dan menulis aliran *audit record* yang masuk secara *append-only* pada sebuah berkas arsip lokal sementara (*buffer*).

Proses pengumpulan ini akan berlangsung hingga berkas log lokal mencapai ambang batas (*threshold*) tertentu yang ditentukan berdasarkan durasi interval waktu pengumpulan. Setelah batas waktu tersebut tercapai, berkas log yang sedang berjalan akan ditutup (*sealed*) dan proses rotasi akan memulai berkas log baru untuk menampung *event* berikutnya. Berkas log periodik yang telah final inilah yang kemudian diunggah secara utuh ke jaringan IPFS sebagai satu kesatuan arsip persisten. Setelah pengunggahan selesai, CID dari berkas arsip tersebut baru akan dikirimkan untuk dicatat secara kekal (*on-chain*) pada *smart contract* Move di jaringan IOTA. Melalui mekanisme *batching* berbasis interval waktu ini, DecMed dapat menyimulasikan kelangsungan pencatatan log secara teratur tanpa menyalahi batasan teknis IPFS, sekaligus mengoptimalkan frekuensi dan jumlah transaksi yang harus dikirim ke jaringan DLT.

III.1.5.9 Analisis Jaminan Integritas Berkas Log Lokal (*Hash Chaining*)

Meskipun sifat kekekalan (*immutability*) dan keterlacakan bukti digital dapat dijamin setelah berkas log diunggah ke IPFS dan CID-nya terkunci pada *smart contract* Move, berkas *audit record* yang masih berada dalam proses pengumpulan sementara

ra di tingkat lokal (*buffer/local log file*) belum memiliki perlindungan tersebut. Pada fase ini, data log lokal yang tersimpan secara sementara sebelum interval rotasi tercapai masih rentan terhadap ancaman modifikasi, penyisipan, maupun penghapusan entri oleh pihak yang memiliki akses ke lingkungan sistem pengumpul.

Untuk mengatasi kerentanan pada media penyimpanan sementara tersebut, diperlukan mekanisme perlindungan integritas di tingkat data log lokal yang mampu memberikan sifat *tamper-evident*. Mekanisme *hash chaining* dipilih sebagai solusi untuk menyambungkan setiap entri *audit record* yang masuk secara berurutan.

Dalam skema *hash chaining*, setiap kali sebuah entri *audit record* baru dibentuk, struktur data wajib menyertakan nilai *hash* dari entri *audit record* sebelumnya pada bidang *prev_record.hash*. Dengan pola keterikatan ini, setiap entri log secara kriptografis mengunci entri sebelumnya sehingga membentuk rantai data yang tidak dapat diubah sebagian. Apabila penyerang mencoba mengubah, menyisipkan, atau menghapus satu entri log pada berkas lokal, nilai *hash* dari entri terkait akan berubah dan merusak konsistensi rantai *hash* pada seluruh entri berikutnya.

Melalui penerapan *hash chaining* ini, berkas log sementara pada lingkungan lokal menjadi bersifat *tamper-evident*. Setiap upaya manipulasi data log selama masa pengumpulan lokal dapat langsung terdeteksi oleh komponen pengumpul sebelum berkas tersebut ditutup, di-rotasi, dan diunggah secara permanen ke jaringan IPFS.

III.1.5.10 Analisis Strategi Persistensi Data terhadap Risiko *Pruning*

Penyimpanan bukti audit pada infrastruktur terdesentralisasi menghadapi tantangan ketersediaan data (*availability*) dalam jangka panjang (M-TA-05). Risiko ini bersumber dari dua lini: proses *Garbage Collection* (GC) pada *node* IPFS yang berpotensi menghapus berkas *unpinned*, serta kebijakan *data pruning* pada *node* IOTA yang menghapus data riwayat transaksi lama demi efisiensi *storage*.

Untuk mengatasi risiko hilangnya bukti digital tersebut, strategi persistensi data dirancang melalui pendekatan ganda yang mengombinasikan mekanisme *IPFS Pinning* di tingkat *off-chain* dengan penyimpanan *state object* pada *smart contract* Move di tingkat *on-chain*. Strategi ini dijalankan melalui dua lapisan mekanisme:

1. **Persistensi Berkas Log (*Off-Chain IPFS Pinning*):** Setiap berkas *audit record* terstruktur yang diunggah ke IPFS wajib dieksekusi dengan perintah

pinning pada *dedicated IPFS node* (atau *IPFS cluster/pinning service*). Mekanisme ini memberi penanda eksplisit kepada *node* IPFS agar berkas log tersebut tidak diidentifikasi sebagai *junk* dan tidak terhapus saat proses *Garbage Collection* berjalan.

2. **Persistensi Metadata dan Bukti Integritas (*On-Chain Move State*):** *Content Identifier* (CID) IPFS beserta *hash* integritas berkas disimpan ke dalam *smart contract* Move sebagai *state object*. Berbeda dengan riwayat transaksi biasa (*transaction history*) pada IOTA yang rentan terkena *data pruning*, *state object* yang aktif pada *smart contract* Move akan terus dipertahankan oleh jaringan IOTA.

Melalui penerapan *IPFS Pinning* untuk mengamankan berkas log utama serta penyimpanan metadata pada *state object* Move Smart Contract, ketersediaan dan keteraksesan bukti digital DecMed dapat dijamin dalam jangka panjang tanpa terpengaruh oleh mekanisme *pruning* jaringan (M-TA-05).

III.1.5.11 Pemetaan Masalah terhadap Solusi

Setiap pendekatan solusi yang diusulkan diberi ID S-TA untuk menjaga ketertelusuran (*traceability*) terhadap masalah yang telah dipetakan pada Tabel III.1. Rangkuman pemetaan solusi berdasarkan hasil analisis yang telah dilakukan ditunjukkan pada Tabel III.9.

Tabel III.9 menunjukkan bahwa setiap permasalahan M-TA yang teridentifikasi telah memiliki pendekatan solusi S-TA yang secara spesifik menanganinya. Solusi S-TA-01 dan S-TA-02 menjawab permasalahan ketiadaan sistem terpusat dan sempitnya cakupan pencatatan. Sementara itu, S-TA-03, S-TA-04, dan S-TA-05 secara langsung menyelesaikan keterbatasan teknis terkait kerentanan modifikasi data log, inkonsistensi format, dan risiko hilangnya data akibat *pruning*. Seluruh daftar solusi ini menjadi acuan utama dalam perancangan arsitektur, implementasi, dan pengujian sistem pada bab selanjutnya.

III.2 Rancangan

Berdasarkan hasil analisis solusi pada Subbab III.1.5, rancangan solusi menjelaskan bentuk sistem yang dibangun. Fondasi DecMed, yaitu IOTA sebagai penyimpan-

Tabel III.9: Daftar Solusi yang Diusulkan Berdasarkan Hasil Analisis

ID	Nama Solusi	Masalah Terkait
S-TA-01	Penerapan <i>Centralized Audit Collector</i> dengan pengiriman <i>asynchronous push-based</i> untuk mengintegrasikan pengumpulan log dari seluruh <i>event source</i> .	M-TA-01
S-TA-02	Perluasan pemetaan cakupan <i>audit event</i> dari berbagai komponen sistem berdasarkan analisis ancaman STRIDE.	M-TA-02
S-TA-03	Penjaminan sifat <i>tamper-evident</i> dan <i>immutability</i> log bertingkat melalui <i>hash chaining</i> pada berkas lokal dan penguncian metadata di <i>smart contract Move</i> .	M-TA-03
S-TA-04	Standardisasi skema <i>audit record</i> bertingkat (Skema Umum dan Skema Detail) berbasis kontrol AU-3 NIST SP 800-53.	M-TA-04
S-TA-05	Penerapan strategi persistensi data ganda melalui <i>IPFS Pinning</i> dan penyimpanan <i>state object Move</i> untuk mencegah dampak <i>data pruning</i> .	M-TA-05

an *on-chain*, IPFS sebagai penyimpanan *off-chain*, dan PRE sebagai mekanisme kriptografis pendukung distribusi akses, tetap dipertahankan.

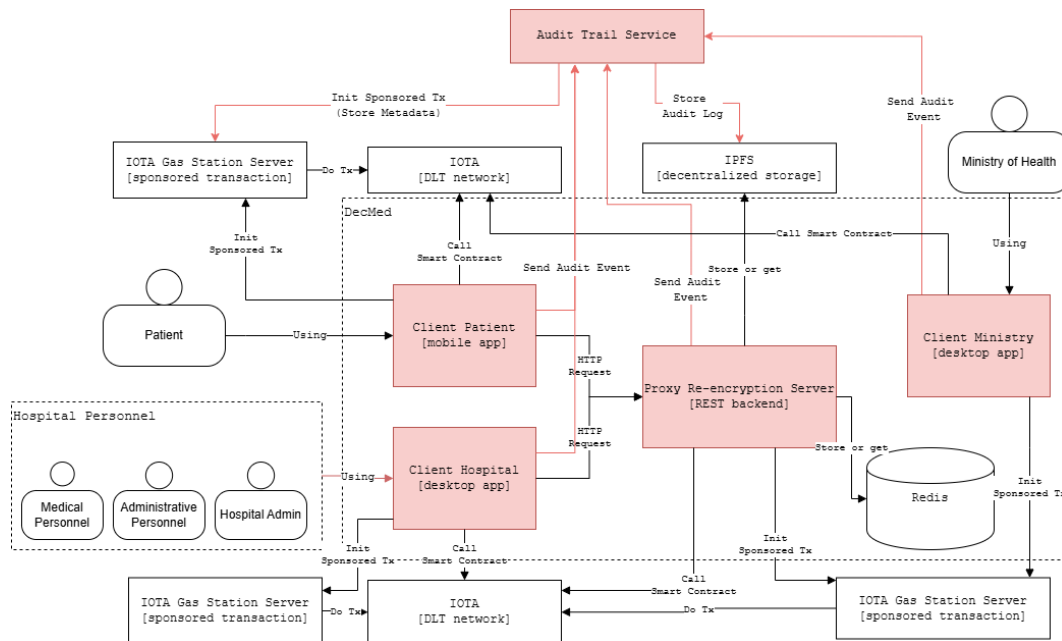
III.2.1 Rancangan Arsitektur Solusi

Keseluruhan arsitektur kerangka kerja DecMed dipertahankan pada arsitektur solusi. Tiga *client* utama DecMed, yaitu *Patient Client*, *Hospital Client*, dan *Ministry Client*, tetap ada namun dilengkapi dengan instrumentasi pengiriman *audit event*. PRE Server tetap berperan sebagai *middleware* keamanan, dengan tambahan instrumentasi serupa.

Perubahan utama berupa penambahan satu komponen baru, yaitu **Audit Log Service (ALS)**, sebagai layanan *backend* independen yang mengumpulkan, memvalidasi, dan menyimpan bukti digital dari seluruh komponen DecMed. Penambahan ALS tidak mengubah alur bisnis maupun fungsionalitas utama DecMed yang telah berjalan pada komponen eksisting. ALS bertindak sebagai pengumpul log terpusat yang memanfaatkan kembali (*reuse*) infrastruktur *off-chain* (IPFS Cluster) dan *on-chain* (jaringan IOTA melalui Gas Station Server) yang sudah tersedia pada DecMed.

Gambaran arsitektur sistem secara keseluruhan dapat dilihat pada Gambar III.2.

Warna merah merepresentasikan komponen baru dan penyesuaian alur interaksi yang ditambahkan pada arsitektur DecMed.



Gambar III.2: Rancangan Arsitektur Solusi Audit Log DecMed

Gambar III.2 menegaskan bahwa kerangka DecMed dipertahankan, sementara penambahan (ditandai merah) mencakup: (1) modul pengiriman *signed event* pada setiap *event source*, (2) ALS sebagai layanan pengumpul terpusat, (3) mekanisme pengarsipan berkas log ke IPFS, dan (4) publikasi metadata rotasi log ke jaringan IOTA melalui Gas Station Server yang telah ada. Dengan demikian, ALS tidak memerlukan infrastruktur penyimpanan baru, melainkan memanfaatkan ulang infrastruktur yang sudah ada pada DecMed.

Berdasarkan Gambar III.2, alur interaksi ALS dalam arsitektur DecMed dirancang sebagai berikut.

1. **Pengiriman *event* dari komponen DecMed.** Setiap kali terjadi aktivitas bernilai audit pada *event source* (yaitu *Patient Client*, *Hospital Client*, *Ministry Client*, dan PRE Server), komponen tersebut membungkus rekaman *audit event* dalam format *signed payload* Ed25519 dan mengirimkannya secara asinkron ke ALS.
2. **Pemrosesan dan pembentukan rantai integritas.** ALS menerima *event*, memverifikasi tanda tangan digital pengirim, membentuk *audit record* sesuai skema terstruktur, dan menerapkan mekanisme *hash-chaining* antar *record* yang masuk.

3. **Penyimpanan berkas *off-chain* di IPFS.** Secara periodik, ALS merotasi berkas log aktif, mengunggah berkas arsip ke IPFS Cluster, dan menerapkan *pinning* agar berkas tidak terhapus oleh proses *Garbage Collection*.
4. **Pencatatan metadata *on-chain* di IOTA.** ALS menerbitkan transaksi ke jaringan IOTA melalui Gas Station Server untuk mencatatkan metadata rotasi log ke dalam *smart contract* Move sebagai *immutable trust anchor*.

Dengan rancangan arsitektur ini, ALS berhasil memisahkan beban pemrosesan dan penyimpanan log dari transaksi data klinis utama, sekaligus menjamin bahwa seluruh aktivitas pada DecMed terekam secara utuh, bersifat *tamper-evident* pada level lokal, dan *immutable* setelah tersimpan pada IOTA dan IPFS.

III.2.2 Rancangan Skema Audit Record

III.2.2.1 Atribut Umum Audit Record

Berdasarkan pemetaan ancaman terhadap audit event pada Tabel III.2, setiap audit event memiliki kebutuhan bukti yang berbeda sesuai dengan aktivitas dan ancaman yang direpresentasikan. Meskipun demikian, terdapat beberapa informasi yang memiliki fungsi umum dan dapat digunakan pada seluruh jenis audit event. Informasi tersebut diperlukan agar setiap audit record dapat menjelaskan pihak yang terlibat, waktu terjadinya aktivitas, objek yang berkaitan dengan aktivitas, serta hasil dari aktivitas tersebut.

Empat kebutuhan bukti umum yang diidentifikasi adalah *actor*, *timestamp*, *object*, dan *result*. *Actor* digunakan untuk mengidentifikasi pihak atau komponen yang melakukan atau memicu aktivitas yang dicatat. Identitas tersebut tidak selalu berupa identitas pengguna, karena beberapa aktivitas dapat dilakukan oleh komponen sistem secara otomatis. Oleh karena itu, atribut ini dapat merepresentasikan pengguna, administrator, proses, maupun komponen sistem yang terkait dengan suatu aktivitas.

Timestamp digunakan untuk mencatat waktu terjadinya aktivitas. Informasi waktu diperlukan untuk mengetahui urutan kejadian, melakukan korelasi antar-event, serta menganalisis pola aktivitas yang berkaitan dengan suatu ancaman. Pencatatan waktu juga memungkinkan audit record dikaitkan dengan kejadian lain yang terjadi

pada periode yang sama.

Object digunakan untuk mengidentifikasi sumber daya atau entitas yang menjadi sasaran aktivitas. Bentuk objek bergantung pada jenis event, seperti rekam medis, *capability*, transaksi IOTA, objek IPFS, objek Redis, maupun metadata *on-chain*. Dengan menggunakan atribut umum *object*, perbedaan jenis sumber daya tersebut dapat direpresentasikan tanpa harus menjadikan setiap jenis sumber daya sebagai atribut pada seluruh audit record.

Sementara itu, *result* digunakan untuk mencatat hasil dari aktivitas yang dilakukan. Hasil dapat berupa keberhasilan, kegagalan, penolakan, maupun status lain yang relevan dengan operasi. Informasi ini diperlukan untuk membedakan aktivitas yang berhasil dilakukan dari percobaan aktivitas yang gagal atau ditolak, sehingga dapat digunakan dalam investigasi maupun analisis ancaman.

Keempat kebutuhan bukti tersebut kemudian dipetakan menjadi atribut umum pada struktur audit record, sebagaimana ditunjukkan pada Tabel III.10. Atribut lain yang hanya relevan terhadap jenis event tertentu tetap direpresentasikan sebagai atribut khusus pada masing-masing event.

Tabel III.10: Pemetaan Kebutuhan Bukti Umum ke Atribut Audit Record

Kebutuhan Bukti	Atribut Umum	Keterangan
Identitas pihak atau komponen yang melakukan atau memicu aktivitas	actor	Mengidentifikasi pengguna, administrator, proses, atau komponen sistem yang melakukan atau memicu aktivitas audit.
Waktu terjadinya aktivitas	timestamp	Mencatat waktu terjadinya aktivitas untuk menentukan urutan kejadian dan melakukan korelasi antar-event.
Objek atau sumber daya yang menjadi sasaran aktivitas	object	Mengidentifikasi entitas yang berkaitan dengan aktivitas, seperti rekam medis, <i>capability</i> , transaksi, objek IPFS, objek Redis, atau metadata <i>on-chain</i> .

Kebutuhan Bukti	Atribut Umum	Keterangan
Hasil aktivitas atau pemrosesan	<code>result</code>	Mencatat hasil aktivitas, seperti berhasil, gagal, ditolak, atau status lain yang relevan.

III.2.2.2 Atribut Khusus Audit Record

Selain atribut umum tersebut, setiap *audit event* juga memiliki atribut tambahan yang disesuaikan dengan kebutuhan bukti digital terhadap ancaman yang dipetakan pada Tabel III.2. Atribut tambahan tersebut disajikan pada Tabel C.1.

III.2.2.3 Rancangan Skema Terperinci AuditEvent dan Enum Atribut

Atribut umum yang telah diidentifikasi pada Subbab sebelumnya diturunkan menjadi skema data terperinci yang terdiri atas dua *struct* bersarang: *AuditEvent* sebagai pembungkus tingkat atas dan *Event* sebagai wadah seluruh atribut detail *event*. Pemisahan ini dilakukan agar atribut yang bersifat metadata pengiriman (`source_component` dan `source_timestamp`) dapat dipisahkan secara struktural dari atribut yang mendeskripsikan aktivitas (`actor_id`, `action_type`, `target_object`, `outcome`, dan `details`).

III.2.2.3.1 Struct AuditEvent *AuditEvent* memuat tiga bidang tingkat atas: `source_component` bertipe *AuditSourceComponent* yang mengidentifikasi komponen pemangkit *event*, `source_timestamp` bertipe *DateTime<Utc>* yang mencatat waktu kejadian menurut jam komponen pengirim, dan `event` bertipe *Event* yang memuat seluruh atribut aktivitas.

III.2.2.3.2 Dual Timestamp: `source_timestamp` dan `timestamp` Skema ini secara sengaja membedakan dua nilai waktu. `source_timestamp` dicatat oleh *event source* pada saat kejadian berlangsung dan disertakan di dalam *event* sebelum enkripsi, sehingga nilainya tidak dapat diubah oleh ALS. `timestamp` pada *AuditRecord* dicatat oleh ALS pada saat *record* dibentuk dari antrian internal. Kedua nilai dapat berbeda apabila terdapat latensi jaringan atau antrean yang signifikan. Penyertaan `source_timestamp` diperlukan agar urutan kejadian yang sesungguhnya dapat direkonstruksi selama investigasi, terlepas dari urutan kedatangan *event* di sisi ALS.

III.2.2.3.3 Struct Event dan Atribut Berketik Event memuat tujuh bidang: *actor_id* (identitas string aktor), *actor_type* bertipe *AuditActorType* (kategori aktor), *target_object* (identitas string objek target), *target_object_type* bertipe *AuditTargetObjectType* (kategori objek target), *outcome* bertipe *AuditOutcome*, *action_type* bertipe *AuditActionType*, dan *details* bertipe *AuditEventDetails*.

Penggunaan *enum* berketik pada *actor_type*, *target_object_type*, *outcome*, dan *action_type* alih-alih representasi String bebas merupakan keputusan desain yang disengaja. *Enum* berketik memindahkan validasi nilai ke tahap deserialisasi: nilai yang tidak dikenal langsung ditolak sebelum masuk ke antrian pemrosesan, sehingga tidak diperlukan validasi terpisah di dalam logika *handler*. Pendekatan ini juga menjamin konsistensi format di seluruh komponen karena hanya nilai yang telah didefinisikan pada *enum* yang dapat dikirimkan.

III.2.2.3.4 Enum AuditSourceComponent *AuditSourceComponent* mengkategorikan komponen pemangkit *event*. Keempat varian yang dirancang sesuai dengan *event source* yang diidentifikasi pada Subbab III.1.5.1 ditunjukkan pada Tabel III.11.

Tabel III.11: Varian *AuditSourceComponent*

Varian	Komponen
<i>HospitalClient</i>	Aplikasi <i>Hospital Client</i> berbasis Tauri.
<i>PatientClient</i>	Aplikasi <i>Patient Client</i> berbasis Tauri.
<i>MinistryClient</i>	Aplikasi <i>Ministry Client</i> berbasis Tauri.
<i>ProxyReencryption</i>	PRE Server.

III.2.2.3.5 Enum AuditActionType *AuditActionType* mengkategorikan jenis tindakan yang dilakukan. Varian dirancang berdasarkan operasi CRUD yang diperluas dengan tindakan khas sistem dekMed. Nilai serialisasi JSON menggunakan format *SCREAMING_SNAKE_CASE* agar mudah dibedakan dari nilai string bebas. Daftar varian yang dirancang merupakan superset dari nilai-nilai yang telah didefinisikan pada Tabel III.16, dengan penambahan varian *SIGN_IN* untuk mencakup peristiwa autentikasi sesi yang terjadi pada aplikasi *client*.

III.2.2.3.6 Enum AuditActorType *AuditActorType* mengkategorikan jenis aktor yang memicu *event*. Varian mencakup seluruh peran yang terdefinisi pada sistem

Tabel III.12: Varian AuditActionType

Varian	Makna
CREATE	Membuat atau menginisialisasi objek baru.
READ	Membaca data tanpa mengubah <i>state</i> .
UPDATE	Memperbarui objek yang sudah ada.
DELETE	Menghapus atau mencabut objek.
VALIDATE	Memverifikasi keabsahan tanpa mengubah <i>state</i> .
EXECUTE	Menjalankan proses yang telah disiapkan sebelumnya.
SIGN_IN	Autentikasi sesi pengguna; hanya digunakan pada aplikasi <i>client</i> .

DecMed, ditambah varian Unknown untuk skenario di mana identitas aktor belum dapat ditentukan pada saat *event* dibangkitkan (misalnya sebelum autentikasi selesai). Daftar varian ditunjukkan pada Tabel III.13.

Tabel III.13: Varian AuditActorType

Varian	Jenis Aktor
AdministrativePersonnel	Staf administrasi rumah sakit.
MedicalPersonnel	Tenaga medis (dokter, perawat, dan sejenisnya).
Patient	Pasien.
Admin	Administrator rumah sakit.
Ministry	Pengguna Kementerian Kesehatan.
PREServer	PRE Server yang bertindak sebagai aktor otomatis.
Unknown	Identitas aktor belum dapat ditentukan.

III.2.2.3.7 Enum AuditTargetObjectType AuditTargetObjectType mengategorikan jenis objek yang menjadi target aktivitas. Varian yang dirancang mencerminkan seluruh sumber daya yang dapat diakses atau dimanipulasi dalam sistem DecMed, sebagaimana ditunjukkan pada Tabel III.14.

III.2.3 Rancangan Spesifikasi *Audit Event*

Subbab ini merinci spesifikasi setiap *audit event* yang telah diidentifikasi pada Tabel III.2. Untuk setiap *event*, dijelaskan konteks aktivitas yang dicatat, kemungkinan nilai pada atribut umum, serta atribut tambahan beserta kemungkinan nilainya.

Tabel III.14: Varian AuditTargetObjectType

Varian	Objek yang Dikategorikan
ActivationKey	Kunci aktivasi akun pengguna.
AccessCapability	<i>Capability</i> akses pada <i>smart contract</i> .
MedicalRecord	Berkas rekam medis.
MedicalRecordMetadata	Metadata rekam medis yang tersimpan <i>on-chain</i> .
AccessDelegationQR	QR <i>code</i> delegasi akses.
AdministrativeData	Data administratif rumah sakit.
Nonce	<i>Nonce</i> sesi yang tersimpan sementara.
AccessKeys	Pasangan kunci akses (termasuk kunci PRE).
Transaction	Transaksi IOTA yang dikirimkan ke jaringan.
IPFSObject	Objek yang tersimpan pada IPFS.
GasReservation	Reservasi <i>gas</i> melalui Gas Station Server.
PREEndPoint	<i>Endpoint</i> layanan PRE Server.
OnChainData	Data yang tersimpan langsung pada <i>ledger</i> IOTA.

Atribut umum yang terdapat pada seluruh jenis *audit event* adalah *source_component*, *actor_id*, *target_object*, *outcome*, dan *action_type*. Kemungkinan nilai dari atribut *outcome* dan *action_type* berlaku seragam di seluruh *event* dan dijelaskan pada Tabel III.15 dan Tabel III.16.

Tabel III.15: Kemungkinan Nilai Atribut *outcome*

Nilai	Kondisi
Success	Aktivitas berhasil diselesaikan sesuai dengan yang dimaksudkan.
Failure	Aktivitas gagal diselesaikan akibat kesalahan teknis, kondisi yang tidak terpenuhi, atau kegagalan pada komponen yang dihubungi (mis. IOTA, IPFS, Redis tidak merespons).
Denied	Aktivitas ditolak secara eksplisit karena tidak memenuhi persyaratan otorisasi, misalnya <i>capability</i> tidak valid, JWT kadaluarsa, atau peran tidak sesuai.

III.2.3.1 EV1 — Authentication

EV1 mencatat setiap percobaan autentikasi pengguna pada *Patient Client* maupun *Hospital Client*, baik yang berhasil maupun yang gagal. Aktivitas yang tercakup

Tabel III.16: Kemungkinan Nilai Atribut `action_type`

Nilai	Kondisi
CREATE	Membuat atau menginisialisasi objek, entri, atau sumber daya baru. Contoh: mendaftarkan akun, menerbitkan <i>activation key</i> , membuat entri rekam medis, mengunggah konten ke IPFS.
READ	Membaca atau mengambil data dari objek yang sudah ada tanpa melakukan perubahan. Contoh: membaca rekam medis, mengambil data administratif, mengunduh konten dari IPFS.
UPDATE	Memperbarui atau memodifikasi objek yang sudah ada. Contoh: memperbarui entri rekam medis, memperbarui <i>activation key</i> , memperbarui metadata administratif.
DELETE	Menghapus atau mencabut objek, entri, atau izin yang sudah ada. Contoh: mencabut delegasi akses (<i>revoke access</i>), <i>sign out</i> , menghapus <i>nonce</i> dari Redis setelah digunakan.
VALIDATE	Memverifikasi keabsahan, keaslian, atau kelengkapan suatu objek atau klaim tanpa mengubah <i>state</i> . Contoh: memvalidasi QR <i>payload</i> , memvalidasi JWT, memvalidasi <i>capability on-chain</i> .
EXECUTE	Menjalankan proses yang telah dipersiapkan sebelumnya. Berbeda dari CREATE karena objeknya sudah ada dan tinggal dieksekusi. Contoh: mengeksekusi transaksi yang telah disponsori <i>gas</i> .

meliputi proses *sign-in* menggunakan *activation key*, *patient ID*, atau *seed words*, serta proses *sign-out*. Pencatatan *event* ini diperlukan untuk mendeteksi pola percobaan autentikasi yang mencurigakan, seperti percobaan berulang yang mengindikasikan serangan *brute force*.

Tabel III.17: Atribut Umum EV1 Authentication

Atribut	Kemungkinan Nilai
source_component	HospitalTauriBackend — PatientTauriBackend
actor_id	IOTA <i>address</i> pengguna yang melakukan autentikasi.
target_object	activation_key — patient_id — seed_words — menyatakan mekanisme autentikasi yang digunakan.
action_type	VALIDATE — CREATE — DELETE — VALIDATE untuk verifikasi kredensial, CREATE untuk inisialisasi sesi, DELETE untuk <i>sign-out</i> .
outcome	Success — Failure — Denied

Tabel III.18: Atribut Tambahan EV1 Authentication

Atribut	Kemungkinan Nilai dan Keterangan
auth_method	activation_key — patient_id — seed_words — metode spesifik yang digunakan dalam percobaan autentikasi.
role	Pasien — PersonnelMedis — PersonnelAdministratif — AdminFasyankes — peran pengguna yang sedang melakukan autentikasi.
attempt_count	Bilangan bulat positif — jumlah percobaan autentikasi yang telah dilakukan pada sesi atau dalam periode tertentu.
failure_reason	invalid_credentials — account_not_found — session_expired — null apabila outcome bukan Failure.
device_info	Informasi perangkat yang digunakan, misalnya <i>fingerprint</i> perangkat atau platform sistem operasi.

III.2.3.2 EV2 — QR Validation

EV2 mencatat proses validasi *payload* QR Access Delegation yang dipindai oleh pasien dari QR code yang ditampilkan oleh *hospital personnel*. *Payload* QR berisi

IOTA *address* dan PRE *public key personnel* dalam format <iota_address>@<hex_pre_pub_key>. Pencatatan *event* ini diperlukan untuk mendeteksi pemalsuan *payload* QR maupun kebocoran informasi delegasi akses.

Tabel III.19: Atribut Umum EV2 QR Validation

Atribut	Kemungkinan Nilai
source_component	PatientTauriBackend
actor_id	IOTA <i>address</i> pasien yang memindai QR.
target_object	IOTA <i>address</i> & PRE <i>public key hospital personnel</i> yang terdapat dalam QR.
action_type	READ — VALIDATE
outcome	Success — Failure

Tabel III.20: Atribut Tambahan EV2 QR Validation

Atribut	Kemungkinan Nilai dan Keterangan
qr_content_hash	Nilai <i>hash</i> SHA-256 dari <i>payload</i> QR yang dipindai — digunakan untuk membuktikan konten yang diterima tanpa menyimpan nilai aslinya.
hospital_personnel_iota_address	IOTA <i>address</i> <i>hospital personnel</i> yang terkandung dalam QR.
hospital_personnel_pre_public_key	PRE <i>public key</i> <i>hospital personnel</i> yang terkandung dalam QR.
hospital_name	Nama fasyankes tempat <i>personnel</i> bernaung, apabila tersedia dalam <i>payload</i> .
hospital_personnel_name	Nama <i>personnel</i> yang ditampilkan dalam QR, apabila tersedia.
validation_step	parse — format_check — address_lookup — tahapan validasi terakhir yang berhasil atau gagal.

III.2.3.3 EV3 — Capability Creation

EV3 mencatat proses pembuatan dan pencabutan *capability* akses yang diberikan pasien kepada *hospital personnel*. *Capability* ini tersimpan sebagai entri *on-chain* (HospitalPersonnelAccess) dan *kfrag* di Redis. Pencatatan *event* ini diperlukan untuk membuktikan bahwa pasien benar-benar telah memberikan atau mencabut delegasi akses kepada *personnel* tertentu, sehingga menangkal ancaman penyangkalan (T6).

Tabel III.21: Atribut Umum EV3 Capability Creation

Atribut	Kemungkinan Nilai
source_component	PatientTauriBackend — ProxyReencryption
actor_id	IOTA <i>address</i> pasien yang mendelegasikan akses.
target_object	IOTA <i>address hospital personnel</i> penerima akses.
action_type	CREATE — DELETE — CREATE untuk pemberian akses, DELETE untuk pencabutan akses.
outcome	Success — Failure

Tabel III.22: Atribut Tambahan EV3 Capability Creation

Atribut	Kemungkinan Nilai dan Keterangan
access_type	read — update — cakupan akses yang diberikan.
exp_duration_ms	Durasi kedaluwarsa <i>capability</i> dalam milidetik.
k_frag_fingerprint	<i>Fingerprint</i> (hash) dari <i>kfrag</i> yang disimpan di Redis, tanpa mengekspos nilai <i>kfrag</i> asli.
transaction_digest	<i>Digest</i> transaksi IOTA yang merekam pembuatan atau pencabutan <i>capability on-chain</i> .
hospital_name	Nama fasyankes tempat <i>personnel</i> penerima akses bernaung.
nonce_used	<i>Nonce</i> yang digunakan dalam proses pemberian akses untuk mencegah <i>replay attack</i> .

III.2.3.4 EV4 — Medical Record Access

EV4 mencatat setiap aktivitas pembacaan maupun penulisan data rekam medis yang dieksekusi oleh PRE Server atas permintaan *hospital personnel*. Aktivitas ini mencakup pengambilan data administratif, pengambilan data klinis beserta proses *re-encryption*, pembuatan entri rekam medis baru, dan pembaruan entri rekam medis. Pencatatan *event* ini diperlukan untuk membuktikan bahwa akses terhadap data sensitif pasien dilakukan dengan otorisasi yang sah (T7, T13).

Tabel III.23: Atribut Umum EV4 Medical Record Access

Atribut	Kemungkinan Nilai
source_component	ProxyReencryption
actor_id	IOTA <i>address hospital personnel</i> yang mengakses.
target_object	IPFS CID rekam medis (untuk data klinis) — IOTA <i>address</i> pasien (untuk data administratif).
action_type	CREATE — READ — UPDATE
outcome	Success — Failure — Denied

Tabel III.24: Atribut Tambahan EV4 Medical Record Access

Atribut	Kemungkinan Nilai dan Keterangan
patient_iota_address	IOTA <i>address</i> pasien pemilik data yang diakses.
record_index	Indeks rekam medis yang diakses atau dibuat, apabila relevan.
role_used	MedicalPersonnel — AdministrativePersonnel — peran <i>personnel</i> yang tercantum pada JWT saat akses dilakukan.
purpose_used	read — update — tujuan akses yang tercantum pada JWT.
jwt_sub	<i>Subject</i> JWT yang digunakan — biasanya IOTA <i>address personnel</i> .
capability_valid	true — false — hasil validasi <i>capability on-chain</i> sebelum akses diproses.
ipfs_cid	CID IPFS konten klinis yang diakses atau diunggah, null apabila operasi hanya menyangkut data administratif.
reencryption_performed	true — false — menyatakan apakah proses <i>re-encryption</i> PRE dilakukan pada operasi ini.

III.2.3.5 EV5 — Personnel Activation Key

EV5 mencatat aktivitas penerbitan dan pembaruan *activation key* yang dilakukan oleh Admin Fasyankes untuk *personnel* di bawahnya. *Activation key* ini memungkinkan *personnel* mendaftarkan diri ke dalam sistem. Pencatatan *event* ini diperlukan untuk membuktikan bahwa admin fasyankes benar-benar menerbitkan *activation key* kepada *personnel* tertentu (T8).

Tabel III.25: Atribut Umum EV5 Personnel Activation Key

Atribut	Kemungkinan Nilai
source_component	HospitalTauriBackend
actor_id	IOTA <i>address</i> Admin Fasyankes yang menerbitkan <i>key</i> .
target_object	Hash ID personel penerima <i>activation key</i> .
action_type	CREATE — UPDATE
outcome	Success — Failure

Tabel III.26: Atribut Tambahan EV5 Personnel Activation Key

Atribut	Kemungkinan Nilai dan Keterangan
admin_iota_address	IOTA <i>address</i> Admin Fasyankes yang menerbitkan <i>key</i> .
personnel_id_hash	Hash dari ID personel penerima <i>activation key</i> .
role_assigned	MedicalPersonnel — AdministrativePersonnel — Admin — peran yang ditetapkan pada <i>activation key</i> .
hospital_id_hash	Hash dari ID fasyankes tempat <i>personnel</i> bernaung.
transaction_digest	Digest transaksi IOTA yang merekam penerbitan atau pembaruan <i>activation key on-chain</i> .
activation_key_hash	Hash dari nilai <i>activation key</i> yang diterbitkan.

III.2.3.6 EV6 — Facility Registration

EV6 mencatat aktivitas registrasi fasyankes baru beserta Admin Fasyankes-nya yang dilakukan oleh Ministry Administrator. Pencatatan *event* ini diperlukan untuk membuktikan bahwa Kementerian benar-benar telah meregistrasi fasyankes tertentu be-

serta administratornya (T9).

Tabel III.27: Atribut Umum EV6 Facility Registration

Atribut	Kemungkinan Nilai
source_component	MinistryTauriBackend
actor_id	IOTA <i>address</i> Ministry Administrator.
target_object	<i>Hash</i> ID fasyankes yang diregistrasi.
action_type	CREATE — UPDATE
outcome	Success — Failure

Tabel III.28: Atribut Tambahan EV6 Facility Registration

Atribut	Kemungkinan Nilai dan Keterangan
facility_id	Identitas fasyankes yang diregistrasi.
facility_name	Nama fasyankes yang diregistrasi.
administrator_id	Identitas Admin Fasyankes yang ditetapkan saat registrasi.
transaction_digest	<i>Digest</i> transaksi IOTA yang merekam registrasi fasyankes.
activation_key_hash	<i>Hash</i> dari <i>facility activation key</i> yang diterbitkan saat registrasi.

III.2.3.7 EV7 — PRE Service Operation

EV7 mencatat setiap permintaan layanan yang diterima oleh PRE Server dari komponen *client* maupun dari PRE Server itu sendiri. Aktivitas yang tercakup meliputi seluruh *endpoint* REST PRE Server: /nonce, /keys, /medical-record, dan /administrative. Pencatatan *event* ini diperlukan untuk mendeteksi penyamaran sebagai PRE Server (*man-in-the-middle*), penyadapan komunikasi, dan pembanjiran *endpoint* (T10, T11, T12).

III.2.3.8 EV8 — IOTA Transaction Submission

EV8 mencatat setiap pengiriman transaksi Move ke jaringan IOTA, baik yang dilakukan oleh komponen *client* (Patient, Hospital, Ministry) maupun oleh PRE Server secara langsung menggunakan *proxy_iota_address*-nya sendiri. Pencatatan *event* ini diperlukan untuk mendeteksi modifikasi transaksi selama pengiriman (T14).

Tabel III.29: Atribut Umum EV7 PRE Service Operation

Atribut	Kemungkinan Nilai
source_component	ProxyReencryption
actor_id	IOTA <i>address</i> pemohon — proxy_iota_address apabila PRE Server bertindak sebagai aktor.
target_object	<i>Endpoint</i> PRE Server yang dipanggil, misalnya /api/v1/nonce, /api/v1/keys, /api/v1/medical-record.
action_type	CREATE — READ — VALIDATE
outcome	Success — Failure — Denied

Tabel III.30: Atribut Tambahan EV7 PRE Service Operation

Atribut	Kemungkinan Nilai dan Keterangan
endpoint_called	<i>Path</i> lengkap <i>endpoint</i> yang dipanggil, misalnya /api/v1/medical-record.
request_id	Pengenalan unik permintaan yang dapat digunakan untuk menghubungkan <i>event</i> ini dengan <i>event</i> lain pada siklus yang sama.
caller_component	PatientTauriBackend — HospitalTauriBackend — ProxyReencryption — komponen yang mengajukan permintaan.
channel_encryption	TLS — none — mekanisme pengamanan kanal komunikasi yang digunakan.
jwt_purpose	read — update — null — tujuan yang tercantum pada JWT yang disertakan dalam permintaan, apabila ada.
http_status_code	Kode status HTTP yang dikembalikan PRE Server sebagai respons, misalnya 200, 401, 403, 500.
latency_ms	Durasi pemrosesan permintaan oleh PRE Server dalam milidetik.

Tabel III.31: Atribut Umum EV8 IOTA Transaction Submission

Atribut	Kemungkinan Nilai
source_component	HospitalTauriBackend — PatientTauriBackend — MinistryTauriBackend — ProxyReencryption
actor_id	IOTA <i>address</i> penandatanganan transaksi — proxy_iota_address apabila PRE Server yang mengirim.
target_object	Nama fungsi Move yang dipanggil, misalnya create_access, signup, create_medical_record.
action_type	CREATE
outcome	Success — Failure

Tabel III.32: Atribut Tambahan EV8 IOTA Transaction Submission

Atribut	Kemungkinan Nilai dan Keterangan
transaction_digest	<i>Digest</i> transaksi IOTA yang dihasilkan setelah transaksi berhasil dikirim dan dikonfirmasi.
payload_hash	Nilai <i>hash</i> dari <i>payload</i> transaksi sebelum dikirim — digunakan untuk mendeteksi modifikasi selama pengiriman.
move_function	Nama fungsi Move yang dipanggil, misalnya create_access, revoke_access, create_medical_record.
move_module	Nama modul Move tempat fungsi berada, misalnya patient, proxy, admin.
network_confirmation	confirmed — pending — failed — status konfirmasi transaksi pada jaringan IOTA.
gas_used	Jumlah <i>gas</i> yang dikonsumsi oleh transaksi.
sponsor_address	IOTA <i>address</i> Gas Station yang mensponsori transaksi, apabila transaksi menggunakan mekanisme sponsorship.

III.2.3.9 EV9 — Gas Sponsorship Request

EV9 mencatat setiap permintaan *sponsorship* transaksi kepada Gas Station, baik dari komponen *client* maupun dari PRE Server. Permintaan ini mencakup dua tahap: reservasi *gas object* (/reserve_gas) dan eksekusi transaksi yang telah disponsori (/execute_tx). Pencatatan *event* ini diperlukan untuk mendeteksi penyamaran sebagai pemohon sah, penyangkalan atas permintaan *sponsorship*, dan pembanjiran *gas object* (T15, T16, T17).

Tabel III.33: Atribut Umum EV9 Gas Sponsorship Request

Atribut	Kemungkinan Nilai
source_component	HospitalTauriBackend — PatientTauriBackend — MinistryTauriBackend — ProxyReencryption
actor_id	IOTA <i>address</i> pemohon <i>sponsorship</i> — proxy_iota_address apabila PRE Server yang memohon.
target_object	<i>Endpoint</i> Gas Station yang dituju: /reserve_gas — /execute_tx.
action_type	CREATE — EXECUTE
outcome	Success — Failure

Tabel III.34: Atribut Tambahan EV9 Gas Sponsorship Request

Atribut	Kemungkinan Nilai dan Keterangan
gas_budget_requested	Jumlah <i>gas budget</i> yang diminta dalam permintaan <i>sponsorship</i> .
reserve_duration_secs	Durasi reservasi <i>gas object</i> dalam detik sebelum kadaluarsa.
reservation_id	Pengenal unik reservasi <i>gas</i> yang diterbitkan oleh Gas Station.
sponsor_address	IOTA <i>address</i> Gas Station yang bertindak sebagai <i>sponsor</i> .
transaction_digest	<i>Digest</i> transaksi yang disponsori, tersedia setelah tahap eksekusi.
gas_coin_object_ids	Daftar ID objek <i>gas coin</i> yang direservasi atau digunakan dalam transaksi.

III.2.3.10 EV10 — Redis Operation

EV10 mencatat setiap operasi penyimpanan, pembacaan, atau penghapusan objek sensitif pada Redis yang dilakukan oleh PRE Server. Objek yang dikelola mencakup *nonce* autentikasi (kunci Redis: `nonce:<iota_address>`) dan material PRE berupa *kfrag* serta *access keys* (kunci Redis: `keys:<hp_addr>@<patient_addr>`). Pencatatan *event* ini diperlukan untuk mendeteksi akses tidak sah atau modifikasi objek Redis (T18, T19, T20).

Tabel III.35: Atribut Umum EV10 Redis Operation

Atribut	Kemungkinan Nilai
source_component	ProxyReencryption
actor_id	proxy_iota_address PRE Server yang mengeksekusi operasi Redis.
target_object	Kunci Redis yang dioperasikan, misalnya <code>nonce:<iota_address></code> atau <code>keys:<hp_addr>@<patient_addr></code> .
action_type	CREATE — READ — DELETE
outcome	Success — Failure

Tabel III.36: Atribut Tambahan EV10 Redis Operation

Atribut	Kemungkinan Nilai dan Keterangan
redis_key_type	<code>nonce</code> — <code>access_keys</code> — kategori objek Redis yang dioperasikan.
operation_type	SET — GET — DEL — EXPIRE — operasi Redis spesifik yang dieksekusi.
ttd_remaining	Sisa TTL entri Redis dalam detik pada saat operasi dilakukan; -1 apabila tidak ada TTL, -2 apabila kunci tidak ditemukan.
key_pattern	Pola kunci Redis yang digunakan, misalnya <code>nonce:*</code> atau <code>keys:*@*</code> , untuk membantu korelasi tanpa mengekspos nilai kunci penuh.

III.2.3.11 EV11 — IPFS Operation

EV11 mencatat setiap operasi pengunggahan maupun pengunduhan objek pada IPFS Cluster yang dilakukan oleh PRE Server. Objek yang dikelola adalah konten klinis terenkripsi (AES-GCM) milik pasien. Pencatatan *event* ini diperlukan

untuk mendeteksi pengunduhan data klinis oleh pihak tidak berwenang maupun pembajakan permintaan ke IPFS (T21, T22).

Tabel III.37: Atribut Umum EV11 IPFS Operation

Atribut	Kemungkinan Nilai
source_component	ProxyReencryption
actor_id	proxy_iota_address PRE Server yang mengeksekusi operasi IPFS.
target_object	CID IPFS dari konten yang diunggah atau diunduh.
action_type	CREATE — READ
outcome	Success — Failure

Tabel III.38: Atribut Tambahan EV11 IPFS Operation

Atribut	Kemungkinan Nilai dan Keterangan
cid	<i>Content Identifier</i> IPFS dari objek yang dioperasikan.
operation_type	upload — download — arah operasi terhadap IPFS.
data_size	Ukuran data dalam <i>byte</i> yang diunggah atau diunduh.
patient_iota_address	IOTA <i>address</i> pasien pemilik data klinis yang dioperasikan.
ipfs_node_url	URL <i>node</i> IPFS Cluster yang digunakan dalam operasi.

III.2.3.12 EV12 — IOTA Metadata Operation

EV12 mencatat setiap aktivitas pembacaan atau pembaruan objek *on-chain* pada jaringan IOTA, yang dapat dilakukan oleh seluruh komponen sistem. Objek yang mencakup meliputi AddressId, PatientIdAccount, HospitalPersonnelIdAccount, HospitalIdMetadata, dan objek-objek *shared* lainnya. Pencatatan *event* ini diperlukan untuk mendeteksi potensi analisis pola transaksi dari metadata publik (T23).

III.2.3.13 EV13 — Capability Validation

EV13 mencatat setiap proses validasi *capability* yang dilakukan oleh PRE Server sebelum permintaan sensitif diproses lebih lanjut. Objek yang divalidasi meliputi ProxyCapEnum dan GlobalAdminCap (*on-chain*), serta JWT *token* yang diterbitkan

Tabel III.39: Atribut Umum EV12 IOTA Metadata Operation

Atribut	Kemungkinan Nilai
source_component	HospitalTauriBackend — PatientTauriBackend — MinistryTauriBackend — ProxyReencryption
actor_id	IOTA <i>address</i> pengguna — proxy_iota_address apabila PRE Server.
target_object	IOTA Object ID dari objek <i>on-chain</i> yang diakses.
action_type	READ — UPDATE
outcome	Success — Failure

Tabel III.40: Atribut Tambahan EV12 IOTA Metadata Operation

Atribut	Kemungkinan Nilai dan Keterangan
object_id	IOTA Object ID dari objek yang diakses atau diperbarui.
object_type	AddressId — PatientIdAccount — HospitalPersonnelIdAccount — HospitalIdMetadata — tipe objek lainnya.
move_function	Nama fungsi Move yang digunakan untuk mengakses objek, apabila akses dilakukan melalui dev_inspect atau transaksi.
is_mutable	true — false — menyatakan apakah operasi bersifat mutasi terhadap <i>state on-chain</i> atau hanya pembacaan.
transaction_digest	<i>Digest</i> transaksi IOTA apabila operasi bersifat mutasi; null apabila hanya dev_inspect.
dev_inspect_used	true — false — menyatakan apakah pembacaan dilakukan menggunakan mekanisme dev_inspect tanpa menerbitkan transaksi.

PRE Server untuk *hospital personnel*. Pencatatan *event* ini diperlukan untuk mendeteksi upaya eskalasi privilege melalui kelemahan validasi *capability* (T24, T25).

Tabel III.41: Atribut Umum EV13 Capability Validation

Atribut	Kemungkinan Nilai
source_component	ProxyReencryption
actor_id	IOTA <i>address hospital personnel</i> yang divalidasi — proxy_iota_address untuk validasi ProxyCap.
target_object	IOTA Object ID <i>capability on-chain</i> — JWT token identifier.
action_type	VALIDATE
outcome	Success — Denied — Failure

Tabel III.42: Atribut Tambahan EV13 Capability Validation

Atribut	Kemungkinan Nilai dan Keterangan
capability_id	IOTA Object ID <i>capability on-chain</i> atau pengenalan JWT yang divalidasi.
required_scope	Cakupan akses yang dipersyaratkan oleh operasi yang diminta, misalnya read atau update.
actual_scope	Cakupan akses aktual yang dimiliki oleh <i>capability</i> yang diperiksa.
rejection_reason	expired — scope_mismatch — not_found — invalid_signature — null apabila validasi berhasil.
middleware_layer	jwt_middleware — onchain_check — lapisan validasi yang melakukan pemeriksaan <i>capability</i> .

III.2.4 Rancangan Mekanisme Pembangkitan dan Penerimaan Audit Event

Berdasarkan analisis pada Subbab III.1.5.1 dan analisis mekanisme pengiriman log pada Subbab III.1.5.3, *audit event* dibangkitkan dari empat *event source* yang telah diidentifikasi, yaitu *Patient Client*, *Hospital Client*, *Ministry Client*, dan PRE Server. Rancangan mekanisme ini mencakup tiga aspek utama yang saling berkaitan: enkapsulasi *event* dalam format terenkripsi dan tertandatangan untuk menjamin kerahasiaan dan *non-repudiation*, mekanisme pengiriman dengan jaminan ketercapaian

ian untuk memastikan tidak ada *event* yang hilang, serta penerimaan dan verifikasi pada sisi ALS.

III.2.4.1 Rancangan Enkapsulasi Audit Event

Analisis pada Subbab III.1.5.4 menetapkan bahwa setiap *event source* harus membungkus *audit event* dalam format yang menjamin keaslian pengirim dan *non-repudiation*. Namun, terdapat kebutuhan keamanan tambahan yang perlu dipenuhi, yaitu kerahasiaan isi *event* selama transit. *Audit event* memuat atribut yang berpotensi sensitif, seperti identitas aktor, jenis tindakan yang dilakukan, dan informasi objek yang diakses. Meskipun kanal komunikasi umumnya dilindungi oleh TLS, perlindungan pada lapisan transport bersifat *hop-by-hop* dan tidak memberikan jaminan kerahasiaan yang melekat pada data itu sendiri setelah transit selesai.

Untuk memenuhi kedua kebutuhan tersebut secara bersamaan, rancangan mengadopsi pendekatan *Encrypt-then-Sign*, yaitu *payload event* dienkripsi terlebih dahulu kemudian *ciphertext* yang dihasilkan ditandatangani. Pola ini dipilih karena tanda tangan diberikan atas *ciphertext*, bukan *plaintext*, sehingga secara kriptografis membuktikan bahwa pengirim yang menciptakan *ciphertext* tersebut dan bertanggung jawab atas isinya, bukan sekadar memiliki akses atas informasi yang terkandung di dalamnya. Rancangan enkapsulasi terdiri atas dua lapisan mekanisme yang diuraikan pada subbab-subbab berikut.

III.2.4.1.1 Lapisan Enkripsi: AES-256-GCM dengan Distribusi Kunci ECIES *Payload audit event* yang telah diserialisasi ke JSON dienkripsi menggunakan algoritma AES-256-GCM dengan kunci sesi yang dibangkitkan secara acak pada setiap pengiriman. AES-256-GCM dipilih karena bersifat *authenticated encryption*, yaitu selain menjamin kerahasiaan, algoritma ini juga menghasilkan *authentication tag* yang memastikan integritas *ciphertext* tidak dapat dirusak tanpa terdeteksi.

Distribusi kunci sesi AES ke ALS dilakukan menggunakan mekanisme ECIES (*Elliptic Curve Integrated Encryption Scheme*) berbasis kurva P-256. ALS menerbitkan kunci publik P-256 yang digunakan oleh setiap *event source* untuk mengenkripsi kunci sesi AES. Hanya ALS yang memiliki kunci privat P-256 bersesuaian yang dapat mendekripsi kunci sesi, sehingga isi *audit event* tidak dapat dibaca oleh pihak mana pun yang tidak memiliki kunci privat tersebut, termasuk pihak yang

menyadap saluran komunikasi.

III.2.4.1.2 Lapisan Tanda Tangan: Ed25519 atas *Ciphertext* Setelah enkripsi selesai, *event source* menandatangani *ciphertext* menggunakan kunci privat IOTA (*IotaKeyPair*) milik aktor yang terlibat pada saat kejadian berlangsung. Penggunaan kunci IOTA sebagai kunci penandatanganan memiliki keunggulan krusial: kunci tersebut sudah terikat pada identitas aktor yang terdaftar pada *ledger* IOTA, sehingga tanda tangan yang dihasilkan dapat diverifikasi secara kriptografis terhadap identitas yang terregistrasi pada sistem. Dengan demikian, semua event dapat dipastikan berasal dari sumber yang terpercaya dan bukan event palsu dari pihak luar. Format *payload* yang dikirimkan ke ALS dibungkus dalam struktur *EncryptedSignedEvent* yang memuat lima bidang: *enc_aes_key* (kunci sesi AES yang telah dibungkus dengan ECIES P-256, dikodekan Base64), *ciphertext* (hasil enkripsi AES-256-GCM atas *payload event*, dikodekan Base64), *nonce* (*nonce* AES-GCM sepanjang 12 byte, dikodekan Base64), *signature* (*IotaSignature* atas *byte-byte ciphertext*, dikodekan Base64), dan *iota_address* (alamat IOTA pengirim yang digunakan ALS untuk memverifikasi tanda tangan).

Pemisahan *enc_aes_key* dan *nonce* sebagai bidang tersendiri dipilih karena keduanya dibutuhkan secara independen pada proses dekripsi di sisi ALS: *nonce* dibutuhkan oleh AES-GCM untuk dekripsi *ciphertext*, sedangkan *enc_aes_key* dibutuhkan oleh ECIES untuk memperoleh kunci sesi. Menggabungkan keduanya ke dalam satu *blob* Base64 akan mempersulit penanganan *error* dan membuat setiap sisi harus melakukan *parsing* tambahan.

Penggunaan *iota_address* alih-alih *public_key* heksadesimal sebagai referensi identitas pengirim dipilih karena *iota_address* merupakan representasi publik identitas IOTA yang lebih umum digunakan pada ekosistem IOTA, dan dapat digunakan langsung untuk pencarian *on-chain* apabila diperlukan verifikasi independen.

III.2.4.2 Rancangan Mekanisme Pengiriman

Berdasarkan analisis pada Subbab III.1.5.3, mekanisme pengiriman yang dipilih adalah *asynchronous push-based*. Setiap *event source* mengimplementasikan modul *ALSClnt* yang bertanggung jawab membungkus *event*, mengirimkannya ke ALS, dan menangani kegagalan pengiriman. Pengiriman dilakukan secara asinkron

menggunakan *fire-and-forget pattern*: komponen utama tidak menunggu konfirmasi dari ALS sebelum melanjutkan pemrosesan, sehingga tidak ada latensi tambahan yang dirasakan oleh pengguna.

Untuk mengakomodasi kemungkinan kegagalan jaringan sementara, `ALSCliant` menerapkan mekanisme *retry* dengan *exponential backoff*. Apabila pengiriman pertama gagal, komponen akan mencoba ulang dengan jeda yang semakin panjang hingga batas percobaan maksimum tercapai. Kegagalan pengiriman yang melampaui batas percobaan dicatat ke `stderr` komponen pengirim.

Format *payload* yang dikirimkan ke ALS dibungkus dalam struktur `Encrypted-SignedEvent` yang memuat lima bidang: `enc_aes_key` (kunci sesi AES yang telah dibungkus dengan ECIES P-256, dikodekan Base64), `ciphertext` (hasil enkripsi AES-256-GCM atas *payload event*, dikodekan Base64), `nonce` (*nonce* AES-GCM sepanjang 12 *byte*, dikodekan Base64), `signature` (*IotaSignature* atas *byte-byte ciphertext*, dikodekan Base64), dan `iota_address` (alamat IOTA pengirim yang digunakan ALS untuk memverifikasi tanda tangan).

Pemisahan `enc_aes_key` dan `nonce` sebagai bidang tersendiri dipilih karena keduanya dibutuhkan secara independen pada proses dekripsi di sisi ALS: `nonce` dibutuhkan oleh AES-GCM untuk dekripsi *ciphertext*, sedangkan `enc_aes_key` dibutuhkan oleh ECIES untuk memperoleh kunci sesi. Menggabungkan keduanya ke dalam satu *blob* Base64 akan mempersulit penanganan *error* dan membuat setiap sisi harus melakukan *parsing* tambahan.

Penggunaan `iota_address` alih-alih `public_key` heksadesimal sebagai referensi identitas pengirim dipilih karena `iota_address` merupakan representasi publik identitas IOTA yang lebih umum digunakan pada ekosistem IOTA, dan dapat digunakan langsung untuk pencarian *on-chain* apabila diperlukan verifikasi independen.

III.2.4.3 Rancangan Antrian Pengiriman Berbasis Berkas (AlsQueue)

Mekanisme *retry* dengan *exponential backoff* yang telah dirancang pada subbab sebelumnya memberikan jaminan pengiriman ulang apabila terjadi kegagalan sementara. Namun, jaminan tersebut bersifat *in-memory*: apabila aplikasi *event source* dimatikan atau mengalami *crash* sebelum pengiriman berhasil, *event* yang belum terkirim akan hilang sepenuhnya.

Untuk mengatasi celah ini, setiap *event source* dirancang dengan sebuah antrian pengiriman berbasis berkas (*file-based send queue*) yang disebut *AlsQueue*. *AlsQueue* menyimpan setiap *EncryptedSignedEvent* yang belum berhasil dikirimkan ke berkas lokal dalam format JSON-Lines, sehingga entri yang belum terkirim tetap bertahan meskipun aplikasi diulang.

III.2.4.3.1 Pola Store-then-Send Urutan operasi yang diterapkan adalah *store-then-send*: sebelum mencoba pengiriman langsung, *event source* terlebih dahulu menyimpan *EncryptedSignedEvent* ke *AlsQueue*. Apabila pengiriman langsung berhasil, entri dihapus dari antrian. Apabila gagal, entri tetap berada di antrian dan akan ditangani oleh *retry worker* pada iterasi berikutnya. Pola ini memastikan bahwa tidak ada *event* yang hilang akibat kegagalan jaringan sementara maupun *restart* aplikasi yang tidak terduga, sehingga memperkuat jaminan ketercapaian (*delivery guarantee*) yang disyaratkan oleh KNF-TA-04.

III.2.4.3.2 Struktur Entri Antrian Setiap entri antrian memuat informasi yang cukup untuk pengiriman ulang secara mandiri tanpa perlu melibatkan komponen lain: *id* (pengenal unik UUID v4), *signed_payload* (JSON *EncryptedSignedEvent* yang siap dikirim), *label* (nama *event* untuk pencatatan diagnostik), *created_at* (waktu pembuatan entri), dan *attempt_count* (jumlah percobaan pengiriman yang telah dilakukan).

III.2.4.3.3 Retry Worker dan Exponential Backoff *Retry worker* berjalan sebagai *background task* terpisah yang diinisialisasi satu kali saat aplikasi dimulai. Pada setiap iterasi, worker membaca seluruh entri dari berkas antrian, mencoba mengirimkan setiap entri yang belum melampaui batas percobaan maksimum, dan menerapkan jeda antar-iterasi menggunakan formula *exponential backoff*:

$$\text{jeda} = \min(\text{RETRY_BASE_SECS} \times 2^c, \text{RETRY_MAX_SECS})$$

di mana c adalah jumlah kegagalan berturut-turut. Jeda yang semakin panjang mencegah pemborosan sumber daya jaringan apabila ALS sedang tidak tersedia, sementara batas atas *RETRY_MAX_SECS* mencegah jeda yang terlalu panjang sehingga *event* tertunda secara berlebihan. Entri yang telah melampaui batas percobaan maksimum

tidak dikirimkan ulang, tetapi tetap disimpan dalam berkas antrian untuk keperluan diagnostik oleh operator sistem.

III.2.5 Rancangan Mekanisme Pembentukan dan Penyimpanan Audit Record

Setelah *event* yang telah terverifikasi diteruskan dari antrian internal, komponen Audit Logger membentuk sebuah *AuditRecord* yang terdiri atas komponen-komponen berikut.

- *record_id*, berupa pengenal unik berbasis UUID v7 yang mengandung informasi waktu sehingga bersifat monoton dan dapat diurutkan secara kronologis.
- *timestamp*, berupa waktu pembentukan *record* dalam format UTC yang diambil pada saat Audit Logger memproses *event*.
- *prev_record_hash*, berupa nilai *hash* SHA-256 dari *record* sebelumnya, atau *null* apabila merupakan *record* pertama dalam rantai.
- *record_hash*, berupa nilai *hash* SHA-256 yang dihitung dari kombinasi *record_id*, *timestamp*, *prev_record_hash*, dan isi *event*.
- Seluruh atribut *AuditEvent* yang telah terverifikasi.

Mekanisme *hash-chaining* pada *audit record* menjamin sifat *tamper-evident* pada berkas log lokal sebelum berkas tersebut diarsipkan ke IPFS. Nilai *record_hash* yang menyertakan *prev_record_hash* menjadikan setiap *record* terikat secara kriptografis pada seluruh *record* sebelumnya dalam rantai, sebagaimana dianalisis pada Subbab III.1.5.9. Penambahan, modifikasi, atau penghapusan *record* pada posisi mana pun akan menyebabkan nilai *record_hash* seluruh *record* sesudahnya menjadi tidak konsisten, sehingga kerusakan dapat terdeteksi melalui pemeriksaan konsistensi rantai.

Setiap *AuditRecord* yang terbentuk kemudian ditulis secara *append* ke berkas log lokal dalam format JSON-Lines (satu objek JSON per baris), menjamin bahwa berkas log tidak pernah menimpa entri yang sudah ada. Hal ini merealisasikan sifat *append-only* yang disyaratkan oleh kebutuhan nonfungsional KNF-TA-01 pada Tabel III.4.

III.2.6 Rancangan Mekanisme Rotasi Berkas Log

Berdasarkan analisis mekanisme penyimpanan berkas log pada Subbab III.1.5.8, IPFS menggunakan pengalamatan berbasis konten (*content-addressed storage*) sehingga setiap berkas diidentifikasi oleh *hash* dari isinya. Konsekuensinya, *audit record* tidak dapat diunggah satu per satu ke IPFS secara langsung, karena setiap unggahan terpisah menghasilkan CID yang berbeda-beda dan tidak membentuk satu arsip yang dapat diverifikasi secara utuh. Oleh karena itu, diperlukan mekanisme pengelompokan dan perputaran berkas log (*log rotation*) yang mengelola berkas log lokal sebagai *buffer* sementara sebelum pengarsipan ke IPFS.

Komponen Log Rotation & Archival Worker dirancang untuk berjalan sebagai *background task* yang berulang secara periodik pada interval yang dapat dikonfigurasi melalui konstanta `LOG_ROTATION_INTERVAL_SECS`. Pada setiap siklus rotasi, worker terlebih dahulu memeriksa apakah berkas log aktif ada dan tidak kosong. Apabila berkas kosong, siklus dilewati untuk menghindari pencatatan metadata rotasi yang tidak bermakna pada jaringan IOTA.

Apabila berkas log aktif berisi data, worker membentuk nama berkas arsip baru dengan format `logs/uploading_{unix_timestamp}.log` menggunakan *Unix timestamp* pada saat rotasi dilakukan, kemudian memindahkan berkas log aktif ke nama arsip baru. Pemindahan berkas dilakukan menggunakan operasi `rename` yang bersifat atomik pada tingkat sistem operasi, sehingga tidak ada jeda di mana berkas log aktif tidak tersedia untuk penulisan. Setelah pemindahan selesai, ALS langsung membuka berkas log aktif baru untuk menampung *audit record* yang masuk selama proses pengarsipan berkas lama berlangsung.

Berkas dikirimkan menggunakan mekanisme *streaming* berbasis `BytesCodec` sehingga isi berkas tidak perlu dimuat seluruhnya ke memori sebelum dikirim; hal ini penting untuk mengakomodasi berkas log yang berukuran besar tanpa membebani konsumsi memori ALS. IPFS Cluster mengembalikan respons yang memuat CID dari berkas yang berhasil diunggah. CID ini bersifat unik terhadap isi berkas dan menjadi penanda permanen yang digunakan pada tahap publikasi metadata ke IOTA. Apabila unggah gagal, siklus rotasi dihentikan pada tahap ini dan kesalahan dicatat ke `stderr`; berkas arsip tetap berada di penyimpanan lokal dengan awalan `uploading_` sehingga dapat diidentifikasi dan diproses ulang pada siklus rotasi ber-

ikutnya.

Berdasarkan analisis strategi persistensi data pada Subbab III.1.5.10, berkas yang diunggah ke IPFS rentan terhapus oleh mekanisme *Garbage Collection* apabila tidak ada penanda eksplisit yang menyatakan bahwa berkas tersebut perlu dipertahankan. Oleh karena itu, setiap berkas arsip log yang berhasil diunggah diterapkan *pinning* pada IPFS Cluster melalui *endpoint* /pins yang tersedia. *Pinning* memberi penanda eksplisit kepada *node* IPFS agar berkas log tersebut tidak diidentifikasi sebagai data tidak terpakai dan tidak terhapus saat proses *Garbage Collection* berjalan. Retensi jangka panjang berkas arsip log bergantung pada kebijakan *pinning* IPFS Cluster yang sama yang telah digunakan DecMed untuk data klinis pasien, sehingga rancangan ini tidak memperkenalkan infrastruktur persistensi baru.

III.2.7 Rancangan Mekanisme Publikasi Metadata *On-Chain* ke IOTA

Setelah berkas arsip log berhasil diunggah ke IPFS dan CID diperoleh, tahap terakhir adalah menerbitkan metadata rotasi log ke jaringan IOTA sebagai *immutable trust anchor*. Rancangan mekanisme ini mencakup tiga aspek: skema metadata, mekanisme penerbitan melalui *smart contract* Move, dan pembentukan rantai metadata *on-chain*.

III.2.7.1 Skema Metadata Rotasi Log

Objek metadata rotasi log dirancang untuk memuat seluruh informasi yang diperlukan agar verifikasi keutuhan berkas arsip log dapat dilakukan secara independen di masa mendatang. Rancangan skema metadata ditunjukkan pada Tabel III.43.

Kombinasi *cid* dan *file_hash* pada skema metadata memberikan dua lapis jaminan verifikasi yang saling melengkapi. Apabila *cid* digunakan untuk mengambil berkas dari IPFS, *file_hash* digunakan untuk memverifikasi bahwa berkas yang diperoleh identik dengan berkas yang tercatat pada metadata. Apabila keduanya cocok, keutuhan berkas arsip log tersebut dapat dinyatakan terjaga sejak saat rotasi dilakukan.

Tabel III.43: Skema Metadata Rotasi Log pada *Smart Contract* IOTA

Bidang	Tipe Data	Deskripsi dan Fungsi
cid	String	<i>Content Identifier</i> IPFS dari berkas arsip log yang diunggah. Digunakan untuk mengambil kembali berkas arsip dari IPFS saat verifikasi.
file.hash	String (hex)	Nilai <i>hash</i> SHA-256 dari berkas arsip, dihitung secara lokal sebelum unggah. Digunakan sebagai referensi integritas independen dari IPFS untuk mendeteksi apabila isi berkas yang diambil tidak sesuai.
log.sequence_number	u64	Nomor urut rotasi yang bersifat monoton dan bertambah satu setiap siklus rotasi. Digunakan untuk menentukan urutan kronologis arsip log tanpa bergantung pada <i>timestamp</i> semata.
rotation_time	String	Waktu pelaksanaan rotasi dalam format UTC. Digunakan untuk merekonstruksi garis waktu pengarsipan log.
prev_tx_digest	String (opsional)	<i>Transaction digest</i> publikasi metadata rotasi sebelumnya. Membentuk rantai antar entri metadata <i>on-chain</i> yang analog dengan mekanisme <i>hash-chaining</i> pada level <i>audit record</i> . Bernilai null pada rotasi pertama.

III.2.7.2 Penerbitan Metadata melalui *Smart Contract Move*

Metadata rotasi log diterbitkan ke jaringan IOTA melalui *smart contract Move* yang didefinisikan pada modul `als::audit_log`. Modul ini menyediakan fungsi `create_log` yang menerima representasi JSON dari metadata rotasi, membuat objek `LogRecord` pada *ledger* IOTA, kemudian langsung membekukannya melalui `transfer::freeze_object`. Pembekuan objek menjadikan `LogRecord` bersifat *immutable* secara penuh setelah transaksi final: tidak ada entitas mana pun, termasuk ALS itu sendiri, yang dapat mengubah isi `LogRecord` setelah diterbitkan. Rancangan ini secara langsung mengatasi masalah M-TA-03 yang mengidentifikasi bahwa mekanisme pencatatan eksisting pada DecMed masih menyediakan fungsi pembaruan pada *smart contract Move* yang ada. Penerbitan dilakukan melalui Gas Station Server yang sudah tersedia pada DecMed, sehingga ALS tidak perlu mengelola aset IOTA secara mandiri. Apabila transaksi berhasil dieksekusi, worker memperoleh *transaction digest* dan *object ID* dari `LogRecord` yang baru dibuat.

Bidang `prev_tx_digest` pada skema metadata dirancang untuk membentuk rantai antar entri metadata *on-chain* yang analog dengan mekanisme *hash-chaining* pada level *audit record* (Subbab III.2.5). Setiap `LogRecord` yang diterbitkan ke IOTA membawa referensi eksplisit terhadap `LogRecord` sebelumnya, sehingga membentuk urutan yang dapat ditelusuri dari entri terbaru ke entri paling awal. Melalui penelusuran rantai ini, pihak yang berwenang dapat memverifikasi kelengkapan arsip log secara kronologis dengan mencocokkan `log_sequence_number` untuk memastikan tidak ada rotasi yang terlewat. Setelah publikasi metadata ke IOTA berhasil, worker memperbarui nilai `prev_tx_digest` yang tersimpan dalam memori untuk digunakan pada siklus rotasi berikutnya, kemudian menghapus berkas arsip dari penyimpanan lokal. Penghapusan berkas lokal ini dilakukan karena bukti integritas jangka panjang sudah tersedia melalui dua lapis mekanisme: berkas arsip tersimpan secara permanen di IPFS melalui *pinning*, dan metadata dengan CID serta `file_hash` tersimpan secara *immutable* di IOTA.

III.2.7.3 Rantai Metadata *On-Chain*

Properti utama yang dihasilkan oleh rantai metadata *on-chain* adalah kemampuan untuk memverifikasi kelengkapan arsip log secara kronologis. Pihak yang berwenang dapat menelusuri seluruh rantai `LogRecord` dari yang terbaru mundur ke rotasi

pertama menggunakan `prev_tx_digest`, kemudian mencocokkan `log_sequence_number` untuk memastikan tidak ada rotasi yang terlewat dalam rangkaian arsip. Skema rantai metadata *on-chain* ditunjukkan pada Gambar III.3.

Gambar III.3: Skema Rantai Metadata Rotasi Log *On-Chain* pada IOTA

Setelah publikasi metadata ke IOTA berhasil, worker memperbarui nilai `prev_tx_digest` yang tersimpan dalam memori untuk digunakan pada siklus rotasi berikutnya, kemudian menghapus berkas arsip dari penyimpanan lokal menggunakan `fs::remove_file`. Penghapusan berkas lokal ini dilakukan karena bukti integritas jangka panjang sudah tersedia melalui dua lapis mekanisme: berkas arsip tersimpan secara permanen di IPFS melalui *pinning*, dan metadata dengan CID serta *file_hash* tersimpan secara *immutable* di IOTA. Penyimpanan berkas arsip secara lokal dalam jangka panjang tidak diperlukan dan justru akan menghabiskan kapasitas penyimpanan lokal ALS.

III.2.8 Pemetaan Rancangan terhadap Solusi

Setelah rancangan solusi diuraikan, setiap rancangan dipetakan kembali terhadap solusi yang telah dianalisis pada Subbab III.1.5. Melalui pemetaan ini, keterkaitan langsung setiap rancangan R-TA dengan solusi S-TA yang diusulkan ditunjukkan. Hasil pemetaan rancangan terhadap solusi ditunjukkan pada Tabel III.44.

Tabel III.44 memastikan setiap solusi S-TA memiliki rancangan R-TA yang merealisasikannya, sehingga ketertelusuran dari analisis solusi ke spesifikasi rancangan tetap terjaga. Selanjutnya, pemenuhan kebutuhan sistem terhadap rancangan yang telah ditetapkan ditunjukkan pada Tabel III.45.

Tabel III.45: Pemetaan Kebutuhan ke Mekanisme Rancangan ALS

ID	Kebutuhan	Mekanisme pada Rancangan
KF-TA-01	Mengagregasi data log dari berbagai komponen ke satu repositori terpusat.	<i>Endpoint</i> POST <code>/api/events</code> pada ALS sebagai titik masuk tunggal yang menerima <i>event</i> dari seluruh <i>event source</i> (R-TA-01, R-TA-03).

ID	Kebutuhan	Mekanisme pada Rancangan
KF-TA-02	Menangkap dan merekam keseluruhan jenis <i>event</i> EV1 hingga EV13.	Format <i>SignedEvent</i> yang seragam memungkinkan ALS menerima seluruh jenis <i>event</i> melalui <i>tagged enum AuditEventDetails</i> (R-TA-03).
KF-TA-03	Memformat dan menstandarisasi <i>audit record</i> ke dalam skema terstruktur.	Skema <i>AuditRecord</i> bertingkat dengan atribut umum wajib dan atribut tambahan spesifik per jenis <i>event</i> (R-TA-02).
KF-TA-04	Menyimpan <i>audit record</i> pada media penyimpanan yang bersifat <i>append-only</i> .	Penulisan ke berkas log lokal menggunakan mode <i>append</i> dikombinasikan dengan <i>hash-chaining</i> (R-TA-04); arsip jangka panjang pada IPFS (R-TA-05).
KF-TA-05	Menyediakan mekanisme retensi jangka panjang untuk mengantisipasi <i>pruning</i> pada jaringan DLT.	<i>Pinning</i> berkas arsip pada IPFS Cluster dan pencatatan CID beserta <i>file hash</i> pada <i>smart contract Move</i> di IOTA (R-TA-05).
KNF-TA-01	Integritas <i>audit record</i> yang <i>tamper-evident</i> .	<i>Hash-chaining</i> antar <i>record</i> (R-TA-04) dan penguncian metadata berkas arsip di IOTA (R-TA-05).
KNF-TA-02	Format data log yang konsisten dan mudah diproses.	Skema <i>AuditRecord</i> terstruktur dalam format <i>JSON-Lines</i> dengan anotasi <i>#[serde(flatten)]</i> (R-TA-02).
KNF-TA-03	Ketersediaan data audit log dalam jangka panjang.	<i>IPFS pinning</i> dan penyimpanan metadata sebagai <i>frozen object</i> di IOTA (R-TA-05).

ID	Kebutuhan	Mekanisme pada Rancangan
KNF-TA-04	Konsistensi pencatatan tanpa <i>event</i> bernilai audit tinggi yang terlewat.	Verifikasi tanda tangan Ed25519 pada ALS dan antrian asinkron berkapasitas 10.000 (R-TA-03).

Tabel III.45 menunjukkan bahwa seluruh kebutuhan fungsional (KF-TA-01 hingga KF-TA-05) dan kebutuhan nonfungsional (KNF-TA-01 hingga KNF-TA-04) terpenuhi oleh mekanisme rancangan yang telah diuraikan. Ketertelusuran dari kebutuhan ke rancangan ini menjadi dasar verifikasi pada tahap pengujian yang akan dipaparkan pada Bab IV.

Tabel III.44: Pemetaan Rancangan terhadap Solusi

ID	Nama Rancangan	Merealisasi Solusi
R-TA-01	Rancangan arsitektur solusi dengan penambahan ALS sebagai <i>centralized audit collector</i> yang terintegrasi dengan infrastruktur IPFS dan IOTA yang sudah ada pada DecMed.	S-TA-01
R-TA-02	Rancangan skema <i>audit record</i> bertingkat yang memisahkan atribut umum dan atribut tambahan spesifik per jenis <i>event</i> .	S-TA-04
R-TA-03	Rancangan mekanisme pembangkitan dan penerimaan <i>audit event</i> berbasis <i>signed payload</i> Ed25519 dengan pengiriman <i>asynchronous push-based</i> .	S-TA-01
R-TA-04	Rancangan mekanisme pembentukan <i>audit record</i> dengan <i>hash-chaining</i> antar <i>record</i> untuk menjamin sifat <i>tamper-evident</i> pada berkas log lokal.	S-TA-03
R-TA-05	Rancangan mekanisme rotasi log, pengarsipan berkas ke IPFS dengan <i>pinning</i> , dan publikasi metadata <i>immutable</i> ke IOTA melalui <i>smart contract</i> Move.	S-TA-03, S-TA-05

BAB IV

IMPLEMENTASI DAN PENGUJIAN

Bab ini menjelaskan proses implementasi dari rancangan solusi yang telah dipaparkan pada Bab III, khususnya penambahan layanan *Audit Log Service* (ALS) dan integrasinya ke dalam arsitektur sistem DecMed yang telah ada.

IV.1 Lingkungan Implementasi

Implementasi sistem dilakukan pada lingkungan prototipe yang merepresentasikan komponen utama DecMed, yaitu *Patient Client*, *Hospital Client*, *Ministry Client*, PRE Server, penyimpanan *off-chain* IPFS Cluster, IOTA Gas Station, *smart contract* IOTA Move, dan layanan Audit Log Service (ALS) yang berjalan pada jaringan IOTA Localnet. Lingkungan ini dipilih agar rancangan pada Bab III dapat diuji tanpa bergantung pada integrasi langsung dengan sistem informasi rumah sakit yang sesungguhnya.

Komponen ALS dan PRE Server dijalankan bersama pada lingkungan lokal bersama ketiga aplikasi *client*. Komponen pendukung yang membutuhkan ketersediaan jaringan, yaitu IPFS Cluster dan IOTA Gas Station, dijalankan pada Virtual Private Server (VPS). *Smart contract* IOTA Move di-*deploy* pada jaringan IOTA Localnet yang juga berjalan di VPS. IOTA Localnet digunakan karena sudah memadai untuk mensimulasikan interaksi dengan jaringan IOTA.

Spesifikasi perangkat lokal yang digunakan adalah sebagai berikut.

1. Sistem Operasi: Ubuntu 24.04.4, berjalan di lingkungan WSL2.
2. CPU: Intel Core i7-12700H (6 *core* fisik dan 12 *logical thread* dialokasikan untuk WSL2).
3. Memori RAM: 32 GB (16 GB teralokasi untuk WSL2).
4. Perangkat bantu pengembangan: Rust (`rustc 1.85.0`, `cargo 1.85.0`), Node.js v22.14.0, dan Docker 27.4.0.

Sementara itu, spesifikasi VPS yang digunakan adalah sebagai berikut.

1. Sistem Operasi: Ubuntu 24.04.4 LTS.
2. Jumlah vCPU: 4 vCPU.
3. Memori RAM: 8 GB.
4. Perangkat bantu: Docker 27.4.0.

IV.2 Implementasi Sistem

Bagian ini menjabarkan realisasi teknis dari rancangan pada Bab III. Narasi konsep dan alur yang sudah ditetapkan di Bab III dirujuk ulang; fokus di sini adalah komponen kode, antarmuka, dan perilaku konkret sistem.

Daftar komponen dan keterkaitannya dengan rancangan ditunjukkan pada Tabel IV.1. ID I-ALS dipakai sebagai rujukan. R-TA-01 (rancangan arsitektur solusi) mencakup keseluruhan sistem sehingga tidak muncul sebagai baris tersendiri.

Tabel IV.1 menjadi acuan urutan pembahasan implementasi berikut, dari skema tipe data, mekanisme enkapsulasi *event*, penerimaan dan verifikasi oleh ALS, pembentukan *audit record* dengan *hash-chaining*, rotasi log dan pengarsipan, hingga integrasi ke seluruh *event source* DecMed.

IV.2.1 Batasan Implementasi

Implementasi mempertahankan arsitektur kerangka kerja DecMed sebagaimana dijelaskan pada Subbab III.2.1. Komponen perangkat lunak ALS diimplementasikan sebagai *crate* Rust yang berdiri sendiri. Dependensi utama yang digunakan ditunjukkan pada Tabel IV.2. Batasan cakupan implementasi, konfigurasi IPFS *pinning*, dan nilai `LOG_ROTATION_INTERVAL_SECS` diuraikan pada Subbab IV.3.

IV.2.2 Implementasi Skema Tipe Data Audit

Implementasi skema tipe data audit merealisasikan katalog *audit event* dan skema atribut yang telah ditentukan pada Subbab III.2.2. Tipe-tipe data pada sisi ALS didefinisikan pada berkas `audit-trail/src/types.rs`, sedangkan tipe-tipe

Tabel IV.1: Daftar Implementasi Sistem Audit Log

ID	Komponen	Merealisasikan Rancangan
I-ALS-01	Skema tipe data AuditEvent, Event, AuditEventDetails, AuditRecord, dan AuditBatch beserta enum pendukungnya.	Katalog <i>audit event</i> (EV1–EV13) dan skema atribut audit pada Subbab III.2.2.
I-ALS-02	Tipe data EncryptedSignedEvent dan mekanisme enkripsi-tanda tangan berlapis pada sisi <i>event source</i> .	Subbab III.2.4 (lapisan enkapsulasi <i>event</i>).
I-ALS-03	<i>Endpoint</i> POST /api/events dan mekanisme dekripsi serta verifikasi <i>encrypted signed event</i> pada ALS.	Subbab III.2.4 (komponen Audit Receiver).
I-ALS-04	Komponen AuditLogger dengan antrian asinkron internal.	Subbab III.2.5 (komponen Audit Logger dan mekanisme <i>hash-chaining</i>).
I-ALS-05	<i>Worker</i> rotasi log, unggah IPFS, dan publikasi metadata ke IOTA.	Subbab III.2.6–III.2.7 (Log Rotation & Archival Worker).
I-ALS-06	Modul als dengan ALSClient, AlsQueue, dan <i>retry worker</i> pada setiap <i>event source</i> .	Subbab III.2.4 dan III.2.4.2 (integrasi ALS ke dalam arsitektur DecMed).

Tabel IV.2: Dependensi Utama Audit Log Service

Dependensi	Versi	Fungsi
axum	0.8.4	<i>Web framework</i> HTTP untuk <i>endpoint</i> penerimaan <i>audit event</i> .
tokio	1.44.2	<i>Async runtime</i> Rust untuk pemrosesan konkuren.
serde / serde_json	1.x	Serialisasi dan deserialisasi data JSON.
aes-gcm	0.10	Enkripsi dan dekripsi AES-256-GCM pada lapisan transport <i>event</i> .
p256	0.13	ECDH P-256 untuk skema ECIES pembungkusan kunci AES.
iota-sdk / iota-types	1.x	Tipe kriptografi IOTA (<i>IotaKeyPair</i> , <i>IotaSignature</i>) dan interaksi jaringan IOTA.
shared-crypto	1.x	Pembungkusan <i>intent message</i> IOTA untuk penandatanganan <i>event</i> .
sha2	0.10	Komputasi <i>hash</i> SHA-256 untuk mekanisme <i>hash-chaining</i> dan KDF ECIES.
uuid	1.x	Pembangkitan <i>record_id</i> berbasis UUID v7 yang bersifat monoton.
chrono	0.4.39	Penanganan waktu dan <i>timestamp</i> dalam format UTC.
request	0.12	Klien HTTP untuk unggah berkas ke IPFS dan pengiriman <i>event</i> ke ALS.
tokio-util	0.7.11	<i>Streaming</i> berkas saat unggah ke IPFS menggunakan <i>BytesCodec</i> .

setara yang digunakan pada setiap *event source* didefinisikan masing-masing pada berkas `als/types.rs` milik komponen bersangkutan. Tipe utama yang menyusun skema ini adalah `EncryptedSignedEvent`, `AuditEvent`, `Event`, empat *enum* pendukung (`AuditOutcome`, `AuditSourceComponent`, `AuditActionType`, `AuditActorType`, `AuditTargetObjectType`), `AuditEventDetails`, `AuditRecord`, dan `AuditBatch`.

IV.2.2.1 Tipe Data `EncryptedSignedEvent`

`EncryptedSignedEvent` adalah struktur yang dikirimkan oleh setiap *event source* ke ALS melalui *endpoint* HTTP. Tipe ini mewakili *event* yang telah dienkripsi dan ditandatangani, sehingga isi *event* tidak dapat dibaca oleh pihak ketiga yang menyadap saluran komunikasi. Tipe ini memuat lima bidang: `enc_aes_key` sebagai kunci sesi AES yang telah dibungkus menggunakan ECIES P-256 dalam representasi Base64, `ciphertext` sebagai *payload audit event* yang telah dienkripsi AES-256-GCM dalam representasi Base64, `nonce` sebagai *nonce* AES-GCM sepanjang 12 *byte* dalam representasi Base64, `signature` sebagai tanda tangan `IotaSignature` atas `ciphertext` dalam representasi Base64, dan `iota_address` sebagai alamat IOTA pengirim yang digunakan ALS untuk memverifikasi tanda tangan. Implementasi `EncryptedSignedEvent` ditunjukkan pada Kode 1.

```

1  #[derive(Debug, Clone, Serialize, Deserialize)]
2  pub struct EncryptedSignedEvent {
3      pub enc_aes_key: String, // base64:
4          ↪ ECIES(ephemeral_pubkey.wrap_nonce.enc_key)
5      pub ciphertext: String, // base64: AuditEvent terenkripsi
6          ↪ AES-256-GCM
7      pub nonce: String, // base64: nonce AES-GCM (12 bytes)
8      pub signature: String, // base64: IotaSignature atas
9          ↪ ciphertext bytes
10     pub iota_address: String, // untuk verifikasi signature di ALS
11 }

```

Kode 1. Implementasi `EncryptedSignedEvent`

Perlu dicatat bahwa pada sisi ALS, terdapat juga tipe `SignedEvent` yang digunakan secara internal untuk keperluan pengujian langsung ke *endpoint*, memuat bidang `payload` (JSON *plaintext*), `signature` (heksadesimal Ed25519), dan `public_key` (heksadesimal). Pada alur produksi, *event source* selalu mengirimkan `EncryptedSignedEvent`, bukan `SignedEvent`.

IV.2.2.2 Tipe Data AuditEvent dan Event

AuditEvent mewakili satu peristiwa audit yang telah diekstrak dan terdekripsi dari EncryptedSignedEvent. Tipe ini memuat tiga bidang: `source_component` bertipe AuditSourceComponent (komponen sumber pembangkit *event*), `source_timestamp` bertipe DateTime<Utc> (cap waktu dari sisi *event source* pada saat kejadian berlangsung), dan `event` bertipe Event yang memuat seluruh atribut detail *event*.

Pemisahan `source_timestamp` dari `timestamp` pada AuditRecord dilakukan secara sengaja: `source_timestamp` mencatat waktu kejadian menurut jam *event source*, sedangkan `timestamp` pada AuditRecord mencatat waktu pembentukan *record* menurut jam ALS. Kedua nilai ini dapat berbeda apabila terdapat latensi jaringan atau antrian, sehingga keduanya diperlukan untuk keperluan rekonstruksi urutan kejadian.

Event memuat tujuh bidang yang merealisasikan skema atribut umum dan atribut detail: `actor_id` (identitas aktor yang memicu aktivitas), `actor_type` bertipe AuditActorType (kategori aktor), `target_object_type` bertipe AuditTargetObjectType (kategori objek target), `target_object` (identitas spesifik objek target), `outcome` bertipe AuditOutcome (hasil aktivitas), `action_type` bertipe AuditActionType (jenis tindakan), dan `details` bertipe AuditEventDetails yang diserialisasi sejajar menggunakan anotasi `#[serde(flatten)]`. Implementasi kedua struct ditunjukkan pada Kode 2.

IV.2.2.3 Enum AuditOutcome dan AuditSourceComponent

AuditOutcome diimplementasikan sebagai *enum* dengan empat varian: Success, Failure, Denied, dan Unknown, merealisasikan atribut outcome pada skema atribut umum. Anotasi `#[serde(rename_all = "PascalCase")]` memastikan nilai JSON berupa "Success", "Failure", dan seterusnya.

AuditSourceComponent diimplementasikan sebagai *enum* dengan empat varian yang merealisasikan seluruh *event source* DecMed: HospitalClient, PatientClient, MinistryClient, dan ProxyReencryption. Anotasi `#[serde(rename_all = "PascalCase")]` juga diterapkan pada *enum* ini. Penggunaan *enum* berketik (*typed enum*) menggantikan representasi String memastikan bahwa hanya nilai yang valid secara semantik yang dapat dimasukkan ke bidang `source_component`; nilai

```

1  #[derive(Clone, Debug, Deserialize, Serialize)]
2  pub struct AuditEvent {
3      pub source_component: AuditSourceComponent,
4      pub source_timestamp: DateTime<Utc>,
5      pub event: Event,
6  }
7
8  #[derive(Clone, Debug, Deserialize, Serialize)]
9  pub struct Event {
10     pub actor_id: String,
11     pub actor_type: AuditActorType,
12     pub target_object_type: AuditTargetObjectType,
13     pub target_object: String,
14     pub outcome: AuditOutcome,
15     pub action_type: AuditActionType,
16
17     #[serde(flatten)]
18     pub details: AuditEventDetails,
19 }

```

Kode 2. Implementasi AuditEvent dan Event

yang tidak dikenal akan ditolak pada tahap deserialisasi. Implementasi kedua *enum* ini ditunjukkan pada Kode 3.

IV.2.2.4 Enum AuditActionType

AuditActionType diimplementasikan sebagai *enum* yang mengkategorikan jenis tindakan yang dilakukan oleh aktor. Varian yang didefinisikan adalah Create, Read, Update, Delete, Validate, dan Execute. Pada *Hospital Client* dan *Patient Client*, varian tambahan SignIn ditambahkan untuk mencatat peristiwa autentikasi sesi.

Anotasi `#[serde(rename_all = "SCREAMING_SNAKE_CASE")]` diterapkan sehingga nilai JSON berupa "CREATE", "READ", "SIGN_IN", dan seterusnya. Konsistensi format ini mempermudah pemrosesan log secara otomatis karena setiap nilai *action_type* dapat dicocokkan secara tepat terhadap daftar yang telah ditentukan. Implementasinya ditunjukkan pada Kode 4.

IV.2.2.5 Enum AuditActorType dan AuditTargetObjectType

AuditActorType mengkategorikan jenis aktor yang memicu *event*. Varian yang didefinisikan mencakup AdministrativePersonnel, MedicalPersonnel, Patient, Admin, Ministry, dan PREServer. Pada *Hospital Client* dan *Patient Client*, dua

```

1  #[derive(Clone, Debug, Deserialize, Serialize)]
2  #[serde(rename_all = "PascalCase")]
3  pub enum AuditOutcome {
4      Success,
5      Failure,
6      Denied,
7      Unknown,
8  }
9
10 #[derive(Clone, Debug, Deserialize, Serialize)]
11 #[serde(rename_all = "PascalCase")]
12 pub enum AuditSourceComponent {
13     HospitalClient,
14     PatientClient,
15     MinistryClient,
16     ProxyReencryption,
17 }

```

Kode 3. Implementasi AuditOutcome dan AuditSourceComponent

```

1  // Versi Hospital Client dan Patient Client (dengan SignIn)
2  #[derive(Clone, Debug, Deserialize, Serialize)]
3  #[serde(rename_all = "SCREAMING_SNAKE_CASE")]
4  pub enum AuditActionType {
5      Create,
6      Read,
7      Update,
8      Delete,
9      Validate,
10     Execute,
11     SignIn,
12 }

```

Kode 4. Implementasi AuditActionType

varian tambahan Unknown dan Client disertakan untuk menangani skenario di mana identitas aktor belum dapat ditentukan saat *event* dibangkitkan.

Untuk memudahkan integrasi dengan sistem otorisasi yang sudah ada, setiap komponen mengimplementasikan *trait* konversi From dari tipe peran lokal ke AuditActorType. Pada PRE Server, From<AuthRole> mengonversi AuthRole::AdministrativePersonnel, AuthRole::MedicalPersonnel, dan AuthRole::Patient ke varian AuditActorType yang bersesuaian. Pada *Hospital Client*, From<HospitalPersonnelRole> mengonversi HospitalPersonnelRole::Admin, HospitalPersonnelRole::AdministrativePersonnel, dan HospitalPersonnelRole::MedicalPersonnel. Selain itu, *trait* FromStr juga diimplementasikan pada *Hospital Client* dan *Patient Client* untuk mengonversi nilai *string* (misalnya dari sesi yang tersimpan) ke AuditActorType. Implementasi

konversi pada PRE Server ditunjukkan pada Kode 5.

```
1  #[derive(Clone, Debug, Deserialize, Serialize)]
2  pub enum AuditActorType {
3      AdministrativePersonnel,
4      MedicalPersonnel,
5      Patient,
6      Admin,
7      Ministry,
8      PREServer,
9  }
10
11 impl From<AuthRole> for AuditActorType {
12     fn from(role: AuthRole) -> Self {
13         match role {
14             AuthRole::AdministrativePersonnel =>
15                 AuditActorType::AdministrativePersonnel,
16             AuthRole::MedicalPersonnel =>
17                 AuditActorType::MedicalPersonnel,
18             AuthRole::Patient =>
19                 AuditActorType::Patient,
20         }
21     }
22 }
```

Kode 5. Implementasi AuditActorType dan konversi From<AuthRole> pada PRE Server

AuditTargetObjectType mengkategorikan jenis objek yang menjadi target aktivitas. Varian yang digunakan bervariasi antar *event source* sesuai objek yang dapat diakses oleh masing-masing komponen. PRE Server mendefinisikan varian yang lebih lengkap, mencakup ActivationKey, AccessCapability, MedicalRecord, MedicalRecordMetadata, AccessDelegationQR, AdministrativeData, Nonce, AccessKeys, Transaction, IPFSObject, GasReservation, PREEndPoint, dan OnChainData. *Client* aplikasi mendefinisikan subset dari varian tersebut yang relevan dengan operasi yang dapat dipicu dari antarmuka pengguna.

IV.2.2.6 Tipe Data AuditEventDetails

AuditEventDetails diimplementasikan sebagai *tagged enum* dengan anotasi `#[serde(tag = "event_type")]` sehingga nilai `event_type` pada JSON yang masuk menentukan varian *enum* yang akan di-*parse*. Terdapat 13 varian yang masing-masing merealisasikan satu jenis *audit event* (EV1–EV13) sesuai Tabel III.2.

Perbedaan antarjenis *event* terletak seluruhnya pada bidang dalam varian masing-

masing. Sebagai contoh, varian EV7 (*PRE Service Operation*) memuat `endpoint_called`, `request_id`, `caller_component`, `channel_encryption`, dan `jwt_purpose`. Pada *client* aplikasi, varian EV7 ditambahkan dua bidang observabilitas yaitu `http_status_code` (bertipe `u16`) dan `latency_ms` (bertipe `u64`) yang tidak didefinisikan pada sisi PRE Server karena PRE Server mengukur latensi dari perspektif yang berbeda. Varian EV13 (validasi *capability*) memuat `capability_id`, `requester_id`, `validation_result` (bertipe `bool`), dan `rejection_reason` (bertipe `Option<String>` yang bernilai `None` apabila validasi berhasil). Implementasi seluruh varian `AuditEventDetails` ditunjukkan pada Lampiran 9.

IV.2.2.7 Tipe Data `AuditRecord`

`AuditRecord` mewakili catatan audit yang telah terbentuk dan siap disimpan ke berkas log. Tipe ini memuat empat bidang tambahan di luar konten *event*: `record_id` bertipe `Uuid` (UUID v7 yang mengandung informasi waktu sehingga bersifat monoton dan dapat diurutkan secara kronologis), `timestamp` bertipe `DateTime<Utc>` (waktu pembentukan *record* menurut jam ALS), `prev_record_hash` bertipe `Option<String>` (nilai *hash* SHA-256 dari *record* sebelumnya, atau `None` apabila merupakan *record* pertama dalam rantai), dan `record_hash` bertipe `String` (nilai *hash* SHA-256 yang dihitung dari kombinasi keempat bidang tersebut bersama isi *event*). Seluruh atribut `AuditEvent` diserialisasi sejajar ke dalam `AuditRecord` menggunakan `#[serde(flatten)]`, sehingga berkas JSON-Lines yang dihasilkan memuat seluruh atribut pada satu tingkat yang sama. Implementasi `AuditRecord` ditunjukkan pada Kode 6.

```
1  #[derive(Debug, Clone, Serialize, Deserialize)]
2  pub struct AuditRecord {
3      pub record_id: Uuid,
4      pub timestamp: DateTime<Utc>,
5      pub prev_record_hash: Option<String>,
6      pub record_hash: String,
7
8      #[serde(flatten)]
9      pub event: AuditEvent,
10 }
```

Kode 6. Implementasi AuditRecord

IV.2.2.8 Tipe Data AuditBatch

AuditBatch merepresentasikan ringkasan satu berkas log yang telah dirotasi dan diarsipkan ke IPFS. Tipe ini tidak disimpan dalam berkas log itu sendiri, melainkan dibentuk oleh *worker* setelah proses rotasi selesai sebagai metadata yang merangkum isi berkas arsip. Bidang-bidang pada AuditBatch mencakup identitas dan rentang waktu *batch* (*batch_id*, *start_time*, *end_time*), ringkasan isi (*record_count*, *first_record_id*, *last_record_id*), nilai *hash* rekam pertama dan terakhir untuk verifikasi ujung rantai (*first_record_hash*, *final_record_hash*), serta informasi persistensi (*file_hash* sebagai *hash* SHA-256 berkas arsip secara keseluruhan, dan *ipfs_cid* sebagai CID yang dihasilkan setelah unggah berhasil). Implementasi AuditBatch ditunjukkan pada Kode 7.

```
1  #[derive(Debug, Clone, Serialize, Deserialize)]
2  pub struct AuditBatch {
3      pub batch_id: Uuid,
4      pub start_time: DateTime<Utc>,
5      pub end_time: DateTime<Utc>,
6      pub record_count: u64,
7      pub first_record_id: Uuid,
8      pub last_record_id: Uuid,
9      pub first_record_hash: String,
10     pub final_record_hash: String,
11     pub file_hash: String,
12     pub ipfs_cid: String,
13 }
```

Kode 7. Implementasi AuditBatch

IV.2.3 Implementasi Mekanisme Enkapsulasi Event pada Event Source

Sebelum *event* dikirimkan ke ALS, setiap *event source* mengenkapsulasi *event* melalui tujuh langkah berurutan yang diimplementasikan pada metode privat

`build_and_send` milik `ALSCClient`. Langkah-langkah tersebut ditunjukkan pada Kode 8.

Langkah 1 hingga 5 merealisasikan mekanisme enkapsulasi berlapis yang dirancang pada Subbab III.2.4.1. Langkah 2 dan 3 bersama-sama merealisasikan lapisan enkripsi: *payload* dienkripsi dengan AES-256-GCM menggunakan kunci sesi acak, dan kunci sesi tersebut kemudian dibungkus dengan kunci publik P-256 ALS melalui skema ECIES. Implementasi ECIES pada fungsi `ecies_encrypt_key` (berkas `als/crypto.rs`) menggunakan *ephemeral keypair* P-256 yang dibangkitkan baru pada setiap pemanggilan, melakukan ECDH terhadap kunci publik ALS, menerapkan SHA-256 sebagai KDF (*key derivation function*) atas *shared secret*, kemudian mengenkripsi kunci AES dengan kunci turunan tersebut menggunakan AES-256-GCM. Hasilnya dikodekan sebagai satu *string* Base64 yang menggabungkan *ephemeral public key*, *wrap nonce*, dan *encrypted AES key*.

Langkah 4 merealisasikan lapisan tanda tangan menggunakan pola *Encrypt-then-Sign*: yang ditandatangani adalah *ciphertext*, bukan *plaintext*. Penandatanganan menggunakan `IotaKeyPair` milik aktor yang terlibat pada saat kejadian berlangsung, yang diperoleh dari `AppState` komponen. Penggunaan kunci IOTA alih-alih kunci Ed25519 terpisah memiliki implikasi penting: kunci tersebut sudah terikat pada identitas aktor yang terdaftar pada *ledger* IOTA, sehingga *non-repudiation* dapat ditegakkan secara kriptografis terhadap identitas yang terregistrasi pada sistem.

Langkah 6 merupakan mekanisme durabilitas yang dibahas lebih lanjut pada Subbab IV.2.4.

IV.2.4 Implementasi `AlsQueue` dan `Retry Worker`

Untuk menjamin tidak ada *event* yang hilang akibat kegagalan jaringan sementara, setiap *event source* mengimplementasikan `AlsQueue`, sebuah antrian berbasis berkas (*file-based queue*) yang menyimpan `EncryptedSignedEvent` yang belum berhasil dikirimkan. Implementasi `AlsQueue` terdapat pada berkas `als/queue.rs` di setiap *event source*.

Setiap entri antrian direpresentasikan sebagai `QueueEntry` yang memuat `id` (UUID v4), `signed_payload` (JSON `EncryptedSignedEvent`), `label` (nama *event* untuk pencatatan diagnostik), `created_at` (waktu pembuatan entri), dan `attempt_count` (jumlah percobaan pengiriman yang telah dilakukan). Antrian disimpan sebagai

berkas JSON-Lines pada direktori lokal sehingga entri yang belum terkirim tetap bertahan meskipun aplikasi diulang.

Pola pengiriman yang diterapkan adalah *store-then-send*: entri disimpan ke antrian terlebih dahulu (Langkah 6 pada Kode 8), kemudian pengiriman langsung dicoba (Langkah 7). Apabila pengiriman langsung berhasil, entri dihapus dari antrian. Apabila gagal, entri tetap berada di antrian dan akan ditangani oleh *retry worker*.

Retry worker dijalankan sebagai *task* Tokio terpisah melalui fungsi `spawn_retry_worker` yang dipanggil saat aplikasi diinisialisasi. Worker berjalan dalam *loop* tak terbatas yang setiap iterasinya membaca seluruh entri dari antrian, mencoba mengirim ulang setiap entri yang belum melampaui batas percobaan maksimum (`MAX_ATTEMPTS_BEFORE_SKIP`), dan menerapkan *exponential backoff* pada jeda antar-iterasi. Jeda dihitung sebagai $\min(\text{RETRY_BASE_SECS} \times 2^{\text{consecutive_failures}}, \text{RETRY_MAX_SECS})$, sehingga semakin banyak kegagalan berturut-turut, semakin panjang jeda sebelum percobaan berikutnya, tanpa tumbuh tak terbatas. Entri yang telah melampaui batas percobaan tidak dikirim ulang tetapi tetap disimpan dalam antrian untuk keperluan diagnostik.

IV.2.5 Implementasi Audit Receiver

Komponen Audit Receiver diimplementasikan melalui *handler* `handle_event` pada berkas `audit-trail/src/handlers.rs` dan *routing* pada `audit-trail/src/main.rs`. *Endpoint* `POST /api/events` menerima *request* berisi objek `EncryptedSignedEvent` dalam format JSON.

Proses pada *handler* `handle_event` terdiri atas tiga fase. Fase pertama adalah verifikasi tanda tangan: ALS memverifikasi `IotaSignature` yang ada di bidang *signature* terhadap *byte-byte ciphertext*, menggunakan `iota_address` yang disertakan sebagai referensi kunci publik. Apabila verifikasi gagal, *handler* mengembalikan respons *error* tanpa memproses *event* lebih lanjut, sehingga *ciphertext* yang dimanipulasi pihak luar tidak pernah masuk ke tahap dekripsi.

Fase kedua adalah dekripsi *payload*: ALS mendekripsi `enc_aes_key` menggunakan kunci privat P-256 miliknya untuk memperoleh kunci sesi AES, kemudian mendekripsi *ciphertext* menggunakan kunci sesi dan *nonce* tersebut untuk memperoleh JSON *plaintext* `AuditEvent`. AES-256-GCM secara otomatis memverifikasi *authentication tag* selama dekripsi; apabila *ciphertext* telah dirusak, dekripsi gagal

dan *event* ditolak.

Fase ketiga adalah pengiriman ke antrian internal: *event* yang telah terverifikasi dan terdekripsi diurai menjadi objek `AuditEvent` dan diteruskan ke antrian asinkron `tokio::sync::mpsc` berkapasitas 10.000 *event* melalui `state.audit_tx.send(audit_event).await`. Desain ini memisahkan tanggung jawab penerimaan HTTP dari pemrosesan dan penulisan log, sehingga ALS dapat menerima *event* berikutnya tanpa menunggu penulisan ke berkas log selesai. Status implementasi komponen Audit Receiver ditunjukkan pada Tabel IV.3.

Tabel IV.3. Status Implementasi Komponen Audit Receiver

Bagian	Status
<i>Endpoint</i> POST <code>/api/events</code>	Terimplementasi.
Verifikasi <i>IotaSignature</i> atas <i>ciphertext</i>	Terimplementasi.
Dekripsi ECIES untuk memperoleh kunci sesi AES	Terimplementasi.
Dekripsi AES-256-GCM untuk memperoleh <i>plaintext event</i>	Terimplementasi.
<i>Parsing</i> <code>AuditEvent</code> dari <i>plaintext</i> yang valid	Terimplementasi.
Pengiriman ke antrian asinkron internal (kapasitas 10.000)	Terimplementasi.
Penolakan <i>event</i> dengan tanda tangan tidak valid	Terimplementasi.

IV.2.6 Implementasi Audit Logger

Komponen Audit Logger diimplementasikan sebagai struct `AuditLogger` pada berkas `audit-trail/src/audit.rs`. Komponen ini berjalan sebagai *task* Tokio terpisah yang di-*spawn* pada saat aplikasi dimulai.

`AuditLogger` menyimpan dua bidang: `rx` bertipe `Receiver<AuditEvent>` sebagai sisi penerima antrian asinkron, dan `prev_record_hash` bertipe `Option<String>` sebagai *hash* dari *record* sebelumnya untuk keperluan *hash-chaining*.

Metode `run` menjalankan *loop* utama menggunakan pola `while let Some(event) = self.rx.recv().await`. Untuk setiap *event*, *loop* melakukan tiga langkah berurutan: (1) memanggil fungsi `create_audit_record` untuk membentuk `AuditRecord` baru, (2) memanggil `write_audit_record` untuk

menulis *record* ke berkas log, dan (3) memperbarui `self.prev_record_hash` dengan *hash* dari *record* yang baru saja ditulis.

Fungsi `create_audit_record` merealisasikan mekanisme *hash-chaining* dengan membangkitkan `record_id` baru menggunakan `Uuid::now_v7()`, mengambil waktu saat ini menggunakan `Utc::now()`, kemudian mendelegasikan komputasi *hash* ke `Utils::calculate_record_hash`. Fungsi `calculate_record_hash` menggabungkan `record_id`, `timestamp`, `prev_record_hash`, dan isi *event* menjadi satu objek JSON, kemudian menghitung SHA-256 dari representasi JSON tersebut dan mengembalikannya sebagai *string* heksadesimal.

Fungsi `write_audit_record` membuka berkas log dengan mode *append* menggunakan `OpenOptions::new().create(true).append(true)`, kemudian menulis representasi JSON dari *record* diikuti karakter baris baru. Penggunaan mode *append* menjamin bahwa entri yang sudah ada tidak pernah tertimpa, merealisasikan karakteristik *append-only* yang disyaratkan oleh KNF-TA-01. Konstanta `LOG_FILE_PATH` yang merujuk ke `./logs/audit_logs.log` didefinisikan pada `audit-trail/src/constants.rs`.

IV.2.7 Implementasi Log Rotation dan Archival Worker

Komponen Log Rotation & Archival Worker diimplementasikan sebagai fungsi `Utils::spawn_log_rotation_worker` pada berkas `audit-trail/src/utils.rs`.

Worker berjalan sebagai *task* Tokio terpisah yang berulang secara periodik sesuai `LOG_ROTATION_INTERVAL_SECS`. Pada setiap siklus, worker melakukan langkah-langkah berikut secara berurutan.

1. **Pembacaan dan validasi berkas log aktif.** Worker memeriksa apakah berkas `audit_logs.log` ada dan tidak kosong. Apabila kosong, siklus dilewati.
2. **Pembacaan seluruh *record* untuk pembentukan `AuditBatch`.** Worker membaca seluruh baris berkas log untuk mengekstrak `first_record_id`, `last_record_id`, `first_record_hash`, `final_record_hash`, `record_count`, `start_time`, dan `end_time`.
3. **Penghitungan *hash* berkas arsip.** Worker menghitung SHA-256 dari seluruh isi berkas log untuk menghasilkan `file_hash` yang akan disimpan pada metadata *on-chain*.

4. **Penggantian berkas aktif.** Berkas log aktif diganti namanya menjadi berkas arsip bertanda waktu, dan berkas `audit_logs.log` baru dibuat kosong untuk melanjutkan pencatatan.
5. **Pengunggahan berkas arsip ke IPFS.** Berkas arsip diunggah ke IPFS Cluster menggunakan *multipart upload* melalui request dengan `BytesCodec`. CID yang dikembalikan oleh IPFS disimpan sebagai `ipfs_cid` pada `AuditBatch`.
6. **Publikasi metadata ke IOTA.** Worker memanggil fungsi `create_log` pada *smart contract* Move melalui Gas Station Server, menerbitkan `AuditBatch` yang telah terbentuk sebagai *frozen object* pada jaringan IOTA.
7. **Penghapusan berkas arsip lokal.** Setelah unggah IPFS dan publikasi IOTA berhasil, berkas arsip lokal dihapus untuk menghemat kapasitas penyimpanan.

IV.2.8 Implementasi Smart Contract Move

Smart contract Move diimplementasikan pada modul `als::audit_log` dalam *package* Move yang di-*deploy* ke jaringan IOTA Localnet. Modul ini menyediakan satu fungsi utama, yaitu `create_log`, yang menerima representasi JSON dari metadata rotasi dan membuat objek `LogRecord` baru pada *ledger* IOTA.

Aspek terpenting dari rancangan *smart contract* ini adalah bahwa modul `als::audit_log` tidak menyediakan fungsi apa pun untuk mengubah atau menghapus objek `LogRecord` yang telah dibuat. Satu-satunya operasi yang tersedia adalah `create_log`. Setiap objek `LogRecord` yang dibuat langsung dibekukan melalui `transfer::freeze_object`, sehingga tidak ada entitas mana pun — termasuk ALS itu sendiri — yang dapat mengubah isinya setelah transaksi pembuatannya *final*. Rancangan ini secara langsung mengatasi masalah M-TA-03 yang mengidentifikasi bahwa mekanisme pencatatan eksisting pada DecMed masih menyediakan fungsi pembaruan pada *smart contract* Move yang ada.

IV.2.9 Implementasi Integrasi ALS ke DecMed

Integrasi ALS ke dalam arsitektur DecMed dilakukan secara seragam pada seluruh *event source* yang telah diidentifikasi: *PRE Server*, *Patient Client*, *Hospital Client*, dan *Ministry Client*. Setiap *event source* diberikan penambahan modul `als` dengan

pola struktur yang konsisten di setiap komponen: berkas `als.rs` yang memuat implementasi `ALSCClient` dan `AlsQueue`, `types.rs` yang mendefinisikan tipe-tipe data bersama, `crypto.rs` yang memuat fungsi kriptografi, dan `queue.rs` yang mengimplementasikan `AlsQueue` dan *retry worker*.

IV.2.9.1 Tipe Data Bersama

Pada setiap *event source*, berkas `als/types.rs` mendefinisikan ulang tipe-tipe data yang strukturnya setara dengan yang ada di ALS. Pendekatan ini dipilih agar setiap komponen sumber *event* tidak bergantung pada *crate* ALS secara langsung, mempertahankan independensi antarkomponen. Varian `AuditEventDetails` yang diimplementasikan pada masing-masing *event source* dibatasi hanya pada jenis *event* yang relevan dengan aktivitas komponen tersebut; varian untuk *event* yang berasal dari komponen lain tidak didefinisikan.

IV.2.9.2 Implementasi ALSCClient

Setiap modul `als` menyediakan *struct* `ALSCClient` yang mengelola seluruh alur enkapsulasi dan pengiriman *event*. Berbeda dari rancangan awal yang menggunakan kunci Ed25519 yang dibangkitkan secara acak, implementasi akhir menggunakan `IotaKeyPair` yang sudah dimiliki oleh aktor yang sedang aktif pada komponen. Kunci ini diambil dari `AppState` komponen saat *event* dibangkitkan, sehingga tanda tangan yang dihasilkan secara langsung mencerminkan identitas aktor pada *ledger* IOTA.

`ALSCClient` menyediakan satu metode publik utama: `send_event_from_state`, yang menerima referensi `AppState`, objek `Event`, dan label *event* bertipe `&'static str`. Metode ini mengambil `IotaKeyPair` dari `AppState`, membangun `AuditEvent` dengan `source_component` yang sesuai komponen dan `source_timestamp` berupa `Utc::now()`, kemudian men-*spawn task* Tokio yang memanggil `build_and_send` secara asinkron. Pola *spawn-and-forget* ini memastikan bahwa operasi audit tidak menambah latensi pada alur layanan utama.

Retry worker dijalankan sekali saat *startup* melalui `ALSCClient::start_retry_worker()`, yang pada aplikasi Tauri menggunakan `tauri::async_runtime::spawn` dan pada PRE Server menggunakan `tokio::spawn`.

IV.2.9.3 Integrasi ke State Komponen

Pada PRE Server, ALSClient tidak disimpan sebagai bidang tersendiri pada AppState karena semua dependensi yang dibutuhkan (IotaKeyPair dan iota_address) sudah tersedia pada AppState. Pada aplikasi *client* berbasis Tauri, AppState berisi keys_entry terenkripsi yang dibuka menggunakan PIN sesi aktif pada saat *event* dibangkitkan.

IV.2.9.4 Distribusi Event per Event Source

Patient Client, *Hospital Client*, dan *Ministry Client* merupakan aplikasi *client* berbasis Tauri. Pada ketiga aplikasi tersebut, modul als diimplementasikan pada sisi *backend* Rust dari aplikasi Tauri, sehingga enkripsi, penandatanganan, dan pengiriman *event* tidak terekspos ke lapisan antarmuka SvelteKit/TypeScript. Distribusi *event* berdasarkan *event source* ditunjukkan pada Tabel ??.

Perlu dicatat bahwa *event* EV8 (IOTA Transaction Submission) dan EV9 (Gas Sponsorship Request) tercatat pada sisi pemohon, bukan pada Gas Station Server.

IV.3 Pengujian

Pengujian sistem dilakukan untuk memastikan implementasi memenuhi kebutuhan yang sudah didefinisikan pada Subbab IV.2.2 hingga III.2.7.

IV.3.1 Strategi Pengujian

Strategi pengujian disusun berdasarkan kebutuhan sistem pada Tabel III.3 dan Tabel III.4. Pengujian dibagi menjadi pengujian kebutuhan fungsional dan pengujian kebutuhan nonfungsional.

Pengujian kebutuhan fungsional diberi ID P-KF dan dijalankan melalui skenario manual untuk memeriksa perilaku sistem dari sudut pandang komponen DecMed sebagai pengirim *audit event* dan ALS sebagai penerima. Pengujian kebutuhan nonfungsional diberi ID P-KNF dan dijalankan melalui pengujian unit atau integrasi terhadap komponen verifikasi tanda tangan digital, mekanisme *hash-chaining*, penulisan berkas log, rotasi dan pengarsipan ke IPFS, serta publikasi metadata ke IOTA. Pengujian otomatis dijalankan dengan cargo test pada komponen yang relevan.

Kriteria keberhasilan pengujian ditentukan berdasarkan kesesuaian hasil skenario manual terhadap kebutuhan fungsional dan kelulusan pengujian otomatis terhadap kebutuhan nonfungsional. Setiap pengujian dikaitkan langsung dengan ID kebutuhan yang diuji untuk memastikan ketertelusuran.

IV.3.2 Pengujian Kebutuhan Fungsional

Pengujian kebutuhan fungsional dijalankan secara manual untuk memastikan perilaku sistem sesuai dengan rancangan pada Subbab III.2.1 hingga III.2.7. Terdapat lima skenario pengujian dengan ID P-KF-01 sampai P-KF-05 yang dijalankan untuk memverifikasi kemampuan ALS menerima, memformat, menyimpan, dan menyajikan kembali *audit event* dari seluruh *event source*.

IV.3.2.1 P-KF-01: Penerimaan dan Agregasi *Audit Event* dari Berbagai *Event Source*

Pengujian P-KF-01 memverifikasi KF-TA-01, yaitu kemampuan ALS mengagregasi data log dari berbagai komponen DecMed ke dalam satu repositori terpusat. Skenario pengujian dilakukan dengan mengirimkan *signed event* secara berturut-turut dari empat *event source* yang berbeda, yaitu *Patient Client*, *Hospital Client*, *Ministry Client*, dan PRE Server, ke *endpoint* POST /api/events pada ALS.

Setiap *event source* mengirimkan dua event: satu event dengan hasil Success dan satu event dengan hasil Failure, untuk memastikan ALS dapat menangani seluruh varian AuditOutcome. Setelah seluruh pengiriman selesai, berkas log ./logs/audit_logs.log diperiksa untuk memastikan seluruh delapan event berhasil ditulis sebagai baris JSON-Lines yang terpisah tanpa ada yang terlewat.

Hasil pengujian menunjukkan seluruh delapan *audit record* berhasil tersimpan pada berkas log. Setiap *record* memuat atribut source_component yang mencerminkan komponen pengirimnya masing-masing, sehingga asal setiap entri dapat ditelusuri tanpa ambiguitas.

IV.3.2.2 P-KF-02: Pencatatan Keseluruhan Jenis *Audit Event* EV1–EV13

Pengujian P-KF-02 memverifikasi KF-TA-02, yaitu kemampuan ALS menangkap dan merekam keseluruhan 13 jenis *audit event* yang telah didefinisikan. Skenario pengujian dilakukan dengan mengirimkan satu *signed event* untuk setiap varian

AuditEventDetails (EV1 hingga EV13) ke ALS.

Setiap *event* dikonstruksi dengan nilai bidang yang valid sesuai skema masing-masing varian pada Lampiran 9. ALS diharapkan berhasil mem-*parse* seluruh 13 varian tersebut melalui mekanisme *tagged enum* `#[serde(tag = "event_type")]` dan menuliskannya ke berkas log.

Hasil pengujian menunjukkan seluruh 13 jenis *event* berhasil diterima dan disimpan. Setiap *record* memuat bidang `event_type` dengan nilai yang sesuai (misalnya "EV1" untuk *event* autentikasi dan "EV13" untuk validasi *capability*), beserta atribut tambahan spesifik yang sesuai dengan varian masing-masing.

IV.3.2.3 P-KF-03: Standardisasi Format *Audit Record*

Pengujian P-KF-03 memverifikasi KF-TA-03, yaitu kemampuan ALS memformat dan menstandarisasi *audit record* ke dalam skema terstruktur sebelum disimpan. Skenario pengujian dilakukan dengan mengirimkan satu *signed event* bertipe EV4 (*Medical Record Access*) ke ALS, kemudian memeriksa struktur JSON dari *audit record* yang dihasilkan.

Audit record yang dihasilkan diverifikasi terhadap skema AuditRecord pada Kode 6. Pemeriksaan mencakup keberadaan seluruh bidang wajib, yaitu `record_id` (berformat UUID v7), `timestamp` (berformat UTC ISO 8601), `prev_record_hash` (bernilai null untuk *record* pertama), `record_hash` (berupa *string* heksadesimal SHA-256 sepanjang 64 karakter), dan seluruh atribut umum AuditEvent beserta atribut tambahan EV4.

Hasil pengujian menunjukkan *audit record* yang dihasilkan memiliki struktur yang sesuai dengan skema yang ditentukan. Seluruh bidang wajib tersedia dan berformat benar, serta atribut tambahan EV4 berupa `access_type`, `medical_record_id`, `capability_id`, dan `authorization_token_id` terserialisasi pada tingkat yang sama dengan atribut umum berkat anotasi `#[serde(flatten)]`.

IV.3.2.4 P-KF-04: Penyimpanan *Audit Record* secara *Append-Only*

Pengujian P-KF-04 memverifikasi KF-TA-04, yaitu kemampuan ALS menyimpan *audit record* pada media penyimpanan yang bersifat *append-only*. Skenario pengujian dilakukan dengan mengirimkan lima *event* secara berurutan ke ALS, kemudian memeriksa berkas log untuk memastikan setiap *event* tersimpan sebagai baris baru

tanpa menimpa baris sebelumnya.

Pemeriksaan dilakukan dengan menghitung jumlah baris pada berkas log sebelum dan sesudah pengiriman lima *event*. Jumlah baris setelah pengiriman harus bertambah tepat lima baris dari jumlah sebelumnya. Selain itu, isi baris-baris lama diverifikasi untuk memastikan tidak ada perubahan.

Hasil pengujian menunjukkan jumlah baris bertambah tepat lima setelah pengiriman, dan isi baris-baris lama tetap tidak berubah. Penggunaan `OpenOptions::new().create(true).append(true)` pada `write_audit_record` terbukti merealisasikan karakteristik *append-only* yang disyaratkan oleh KNF-TA-01.

IV.3.2.5 P-KF-05: Retensi Jangka Panjang melalui Rotasi Log, Pengarsipan IPFS, dan Publikasi Metadata IOTA

Pengujian P-KF-05 memverifikasi KF-TA-05, yaitu kemampuan ALS menyediakan mekanisme retensi jangka panjang. Skenario pengujian dilakukan dengan mengonfigurasi `LOG_ROTATION_INTERVAL_SECS` ke nilai kecil (60 detik) untuk mempercepat siklus rotasi, kemudian mengisi berkas log dengan sejumlah *event* dan menunggu siklus rotasi berjalan.

Keberhasilan pengujian diverifikasi melalui tiga indikator: (1) berkas arsip `logs/uploading-{timestamp}.log` terbentuk selama proses pengarsipan kemudian terhapus setelah selesai, (2) log konsol menampilkan pesan `[rotation] upload IPFS berhasil. CID: {cid}` dengan nilai CID yang valid, dan (3) log konsol menampilkan pesan `[iota] LogRecord dibuat. Object ID: {id} | TX: {digest}` yang menandakan metadata berhasil diterbitkan pada jaringan IOTA.

Hasil pengujian menunjukkan seluruh tiga indikator terpenuhi. CID yang dihasilkan oleh IPFS dapat digunakan untuk mengambil kembali berkas arsip melalui *gateway* IPFS, dan *object ID* LogRecord pada IOTA dapat diverifikasi melalui IOTA Explorer pada jaringan Localnet.

IV.3.3 Pengujian End-to-end Audit Log

Pengujian end-to-end (P-INT-01) merupakan pengujian integrasi yang memverifikasi keseluruhan *pipeline* audit log dari pembangkitan *event* hingga verifikasi inte-

gritas rantai *hash*, menggunakan satu skenario konkret yang dapat direproduksi. Pengujian ini tidak menggantikan P-KF atau P-KNF yang menguji komponen secara atomik, melainkan melengkapinya dengan membuktikan bahwa seluruh komponen bekerja benar secara terpadu dalam satu alur nyata.

Skenario yang digunakan adalah akses rekam medis elektronik (RME) pasien oleh tenaga medis dari *Hospital Client*, yang menghasilkan *event* jenis EV4 (*Medical Record Access*).

IV.3.3.1 Langkah 1: Event yang Terbentuk

Ketika tenaga medis mengakses RME pasien, *Hospital Client* membangkitkan objek Event berikut dan membungkusnya ke dalam *AuditEvent* sebelum dikirimkan ke ALS.

```
1  {
2    "source_component": "HospitalClient",
3    "source_timestamp": "2025-06-10T08:14:32.104Z",
4    "event": {
5      "actor_id": "0xabc123...tenaga_medis_iota_address",
6      "actor_type": "MedicalPersonnel",
7      "target_object_type": "MedicalRecord",
8      "target_object": "ipfs://bafybeig...cid_rekam_medis",
9      "outcome": "Success",
10     "action_type": "READ",
11     "event_type": "EV4",
12     "access_type": "direct",
13     "medical_record_id": "RME-2025-00471",
14     "capability_id": "cap:0xdef456...capability_on_chain",
15     "authorization_token_id": "jwt:eyJ...token_id"
16   }
17 }
```

Atribut *actor_id* berisi alamat IOTA aktor yang juga digunakan untuk menandatangani *ciphertext* *EncryptedSignedEvent*. Atribut *source_timestamp* dicatat di sisi *Hospital Client* sebelum enkripsi, sehingga nilainya mencerminkan waktu kejadian menurut jam komponen pengirim dan tidak dapat diubah oleh ALS.

Sebelum dikirimkan, *AuditEvent* di atas dienkripsi menggunakan AES-256-GCM

dengan kunci sesi acak, kunci sesi dibungkus dengan kunci publik P-256 ALS melalui ECIES, dan *ciphertext* ditandatangani menggunakan IotaKeyPair aktor. Hasilnya dikirimkan ke ALS dalam format EncryptedSignedEvent.

IV.3.3.2 Langkah 2: Record yang Tersimpan

Setelah ALS memverifikasi tanda tangan, mendekripsi *payload*, dan memproses *event* melalui AuditLogger, satu baris JSON-Lines berikut ditulis secara *append* ke berkas `./logs/audit_logs.log`. Contoh *record* kedua dalam rantai ditunjukkan di bawah (nilai `prev_record_hash` mengacu pada *record* sebelumnya).

```
1  {
2    "record_id": "01971b3e-...-uuid-v7",
3    "timestamp": "2025-06-10T08:14:32.381Z",
4    "prev_record_hash": "a3f8c2...hash-record-sebelumnya",
5    "record_hash": "7d4e91...hash-record-ini",
6    "source_component": "HospitalClient",
7    "source_timestamp": "2025-06-10T08:14:32.104Z",
8    "event": {
9      "actor_id": "0xabc123...tenaga_medis_iota_address",
10     "actor_type": "MedicalPersonnel",
11     "target_object_type": "MedicalRecord",
12     "target_object": "ipfs://bafybeig...cid_rekam_medis",
13     "outcome": "Success",
14     "action_type": "READ",
15     "event_type": "EV4",
16     "access_type": "direct",
17     "medical_record_id": "RME-2025-00471",
18     "capability_id": "cap:0xdef456...capability_on_chain",
19     "authorization_token_id": "jwt:ey...token_id"
20   }
21 }
```

Selisih antara `source_timestamp` (08:14:32.104Z) dan `timestamp` (08:14:32.381Z) sebesar ± 277 ms merupakan latensi dari enkripsi, pengiriman jaringan, dan pemrosesan antrian internal ALS. Kedua nilai diperlukan: `source_timestamp` untuk merekonstruksi urutan kejadian yang sesungguhnya,

timestamp untuk menentukan urutan pemrosesan di sisi ALS.

Nilai `record_hash` dihitung oleh `Utils::calculate_record_hash` sebagai SHA-256 dari gabungan `record_id`, `timestamp`, `prev_record_hash`, dan seluruh isi *event* yang diserialisasi ke JSON. Proses ini dapat direproduksi: menjalankan fungsi yang sama terhadap data yang sama selalu menghasilkan nilai yang identik.

IV.3.3.3 Langkah 3: Hubungan Antar-Record dan Rantai Hash

Properti rantai *hash* dapat diverifikasi dengan membandingkan nilai `prev_record_hash` pada setiap *record* dengan nilai `record_hash` pada *record* sebelumnya. Untuk tiga *record* berurutan pada berkas log, hubungan yang harus terbentuk adalah:

N	prev_record_hash	record_hash
1	null	a3f8c2...
2	a3f8c2...	7d4e91...
3	7d4e91...	b91d4f...

Nilai `prev_record_hash` pada *record* ke- N harus sama persis dengan `record_hash` pada *record* ke- $(N - 1)$. Properti ini diverifikasi melalui fungsi `verify_chain` yang membaca seluruh baris berkas log secara berurutan dan memeriksa konsistensi setiap pasangan *record* yang bersebelahan.

IV.3.3.4 Langkah 4: Verifikasi Integritas dan Deteksi Perusakan

Verifikasi integritas dijalankan menggunakan *unit test* `verify_chain_valid` dan `verify_chain_tampered` dengan perintah:

```
1 cargo test verify_chain -- --nocapture
```

Skenario a — Rantai valid. Tiga *record* dibuat secara berurutan melalui `create_audit_record`. Fungsi `verify_chain` dipanggil terhadap ketiga *record* tersebut. Hasil yang diharapkan adalah seluruh *record* dinyatakan valid dan fungsi mengembalikan `true`.

Skenario b — Perusakan pada posisi tengah. Setelah rantai tiga *record* dibuat, nilai bidang `actor_id` pada *record* kedua diubah secara langsung di memori. Fungsi `verify_chain` dipanggil kembali. Hasil yang diharapkan adalah *record* kedua gagal verifikasi karena nilai `record_hash` yang tersimpan tidak lagi sesuai dengan *hash* yang dihitung ulang dari konten yang telah dimodifikasi. Selain itu, *record*

ketiga juga gagal verifikasi karena nilai `prev_record_hash`-nya tidak lagi cocok dengan `record_hash record` kedua yang seharusnya.

Hasil pengujian yang terekam adalah: skenario a mengembalikan `chain_valid: true` untuk seluruh 3 *record*, sedangkan skenario b mengembalikan `chain_valid: false` dengan pesan record 2: `hash mismatch` dan record 3: `prev_hash mismatch`. Hasil ini membuktikan bahwa kerusakan pada posisi mana pun dalam rantai terdeteksi secara otomatis melalui efek domino pada seluruh *record* sesudahnya.

IV.3.3.5 Langkah 5: Kesimpulan dari Satu Record

Dari *record* EV4 yang tersimpan, dapat disimpulkan secara langsung:

- **Siapa** yang mengakses: `actor_id 0xabc123...` dengan peran `MedicalPersonnel`.
- **Apa** yang diakses: RME dengan ID `RME-2025-00471` yang tersimpan di IPFS pada CID `bafybeig...`
- **Kapan** akses terjadi: `2025-06-10T08:14:32.104Z` menurut jam komponen pengirim.
- **Dengan otorisasi apa**: `capability cap:0xdef456...` yang dapat diverifikasi *on-chain* di IOTA, dan JWT dengan ID `jwt:ey...`
- **Hasilnya**: akses berhasil (`outcome: Success`).

Tanda tangan `IotaSignature` yang valid atas *ciphertext* membuktikan bahwa *event* ini benar-benar dibangkitkan oleh pemegang kunci privat IOTA yang bersesuaian dengan `actor_id` tersebut. Karena kunci IOTA terikat pada identitas yang terdaftar di *ledger*, aktor tidak dapat menyangkal bahwa akses ini dilakukan darinya selama kunci privat tidak dikompromikan.

IV.3.3.6 Langkah 6: Batas Kesimpulan

Terdapat tiga batas yang perlu dinyatakan secara eksplisit agar kesimpulan di atas tidak melampaui apa yang sesungguhnya dibuktikan oleh sistem.

(a) Isi *event* tersimpan dalam bentuk terenkripsi. Berkas `audit_logs.log` menyimpan `AuditRecord` dalam *plaintext* JSON-Lines karena dekripsi dilakukan oleh ALS sebelum pembentukan *record*. Namun, *event* yang telah diarsipkan ke IPFS dan berkas arsip yang sudah dirotasi tersimpan setelah melalui proses ALS; ins-

peksi berkas arsip memerlukan akses langsung ke ALS atau rekonstruksi dari log. Untuk keperluan investigasi mendalam, dekripsi *ciphertext* yang masih tersimpan di antrian memerlukan kunci privat P-256 ALS.

(b) Fasilitas pencarian dan filter lintas-record belum tersedia. Implementasi saat ini hanya menyediakan *endpoint* GET /api/logs untuk membaca metadata rotasi *on-chain*. Analisis lintas-record — misalnya menemukan semua akses oleh aktor tertentu dalam rentang waktu tertentu — harus dilakukan secara manual terhadap berkas JSON-Lines atau menggunakan alat bantu eksternal seperti jq. Fasilitas ini belum terintegrasi ke dalam ALS.

(c) Non-repudiation antar-sesi belum terjamin penuh. Tanda tangan menggunakan IotaKeyPair yang diambil dari AppState aktif. Karena kunci IOTA terikat pada identitas aktor di *ledger*, *non-repudiation* berlaku selama kunci privat tidak dikompromikan dan aktor masih memiliki kendali atas kunci tersebut. Namun, apabila terjadi pergantian kunci IOTA aktor antara waktu kejadian dan waktu investigasi, verifikasi tanda tangan terhadap kunci publik saat ini dapat gagal meskipun *event* tersebut sah. Mekanisme rotasi kunci dengan riwayat kunci publik yang terekam berada di luar cakupan implementasi saat ini.

Dengan demikian, P-INT-01 membuktikan bahwa mekanisme inti sistem berfungsi benar: *record* terbentuk sesuai skema, rantai *hash* valid, kerusakan terdeteksi, dan dari satu *record* sudah dapat disimpulkan bukti minimal yang relevan untuk keperluan akuntabilitas. Klaim ini tidak melampaui batas ketiga yang telah disebutkan di atas.

IV.3.4 Pengujian Kebutuhan Nonfungsional

Pengujian kebutuhan nonfungsional dijalankan untuk mengevaluasi pemenuhan aspek integritas, interoperabilitas, ketersediaan, dan konsistensi pencatatan pada sistem ALS. Setiap pengujian dikaitkan dengan kebutuhan nonfungsional yang telah didefinisikan pada Tabel III.4. Unit testing menyimulasikan skenario ancaman STRIDE yang relevan, khususnya ancaman kategori *Tampering* (T14, T18) dan *Repudiation* (T6, T7, T8, T9, T16) yang dipetakan ke kebutuhan nonfungsional.

IV.3.4.1 Pengujian KNF-TA-01: Integritas *Audit Record* yang *Tamper-Proof*

Pengujian KNF-TA-01 memastikan bahwa *audit record* yang telah tersimpan bersifat *tamper-evident* dan kerusakan dapat terdeteksi melalui mekanisme *hash-chaining*. Pengujian ini menguji mitigasi terhadap ancaman T7 dan T8 (kategori *Repudiation*) yang berkaitan dengan kemampuan tenaga medis dan admin fasyankes menyangkal tindakan yang telah dilakukan.

Pengujian pertama, yaitu P-KNF-01, dijalankan melalui unit testing pada fungsi `Utils::verify_record`. Skenario pengujian menciptakan sebuah `AuditRecord` yang valid menggunakan `create_audit_record`, kemudian memverifikasi *record* tersebut dengan `verify_record`. Hasil verifikasi harus mengembalikan `true`. Selanjutnya, satu karakter pada bidang `source_component` dalam *record* dimodifikasi secara langsung, dan `verify_record` dipanggil kembali. Hasil verifikasi harus mengembalikan `false` karena nilai `record_hash` yang tersimpan tidak lagi sesuai dengan *hash* yang dihitung ulang dari konten *record* yang telah dimodifikasi.

Pengujian kedua, yaitu P-KNF-02, dijalankan melalui unit testing untuk mensimulasikan kerusakan pada posisi tengah rantai. Tiga `AuditRecord` dibuat secara berurutan sehingga membentuk rantai (setiap *record* menyertakan `prev_record_hash` dari *record* sebelumnya). *Record* kedua kemudian dimodifikasi, dan seluruh tiga *record* diverifikasi. *Record* pertama harus tetap valid, sedangkan *record* kedua dan ketiga harus gagal verifikasi karena efek domino kerusakan rantai.

Hasil pengujian menunjukkan bahwa modifikasi pada posisi mana pun dalam rantai *audit record* terdeteksi secara akurat. Implementasi `calculate_record_hash` yang menggabungkan `record_id`, `timestamp`, `prev_record_hash`, dan isi *event* terbukti efektif menjamin sifat *tamper-evident* pada berkas log lokal.

IV.3.4.2 Pengujian KNF-TA-02: Interoperabilitas dan Keterbacaan Format Data Log

Pengujian KNF-TA-02 memastikan format data log yang dihasilkan konsisten, terstruktur, dan mudah dibaca oleh komponen sistem. Pengujian ini berkaitan dengan kemampuan format JSON-Lines untuk diproses lintas komponen tanpa konversi tambahan.

Pengujian P-KNF-03 dijalankan melalui unit testing yang membaca kembali berkas log yang telah terisi beberapa *audit record* dan melakukan deserialisasi setiap ba-

ris JSON menggunakan `serde_json::from_str::<AuditRecord>`. Seluruh baris harus berhasil dideserialisasi tanpa *error* menjadi objek `AuditRecord` yang valid, dan atribut-atributnya harus cocok dengan nilai yang dikirimkan saat pengiriman *event*.

Hasil pengujian menunjukkan seluruh baris berkas log berhasil dideserialisasi. Penggunaan `#[serde(flatten)]` pada `AuditEvent` dan `AuditEventDetails` terbukti menghasilkan format JSON yang datar (*flat*) dan mudah diproses, tanpa hierarki objek yang bersarang secara berlebihan.

IV.3.4.3 Pengujian KNF-TA-03: Ketersediaan Data *Audit Log* dalam Jangka Panjang

Pengujian KNF-TA-03 memastikan ketersediaan dan keteraksesan data *audit log* dalam jangka panjang melalui kombinasi *IPFS pinning* dan penyimpanan metadata *on-chain* pada IOTA.

Pengujian P-KNF-04 dijalankan dengan mengunggah sebuah berkas arsip log ke IPFS melalui fungsi `Utils::add_file_to_ipfs`, kemudian mengambil kembali berkas tersebut menggunakan CID yang diperoleh melalui *endpoint* GET `{IPFS_GATEWAY_BASE_URL}/ipfs/{cid}`. Konten yang diambil kembali harus identik dengan konten berkas yang diunggah, diverifikasi dengan membandingkan nilai *hash* SHA-256 dari keduanya.

Pengujian P-KNF-05 dijalankan dengan memverifikasi bahwa objek `LogRecord` yang diterbitkan ke IOTA melalui `IotaLogClient::publish_metadata`. Pemeriksaan dilakukan melalui *endpoint* GET `/object/{object_id}` pada IOTA Localnet RPC. Respons harus memuat field `"owner": "Immutable"` yang menandakan objek tidak dapat dimodifikasi oleh siapa pun setelah transaksi pembuatannya final.

Hasil pengujian P-KNF-04 menunjukkan berkas arsip yang diambil kembali dari IPFS memiliki nilai *hash* SHA-256 yang identik dengan berkas aslinya, membuktikan integritas data selama penyimpanan dan pengambilan. Hasil pengujian P-KNF-05 menunjukkan objek `LogRecord` pada IOTA Localnet berstatus *immutable*, membuktikan bahwa mekanisme `transfer::freeze_object` pada *smart contract* `Move als::audit_log` berhasil menegakkan sifat *append-only* pada level *on-chain*.

IV.3.4.4 Pengujian KNF-TA-04: Konsistensi Pencatatan melalui Verifikasi Tanda Tangan Digital

Pengujian KNF-TA-04 memastikan konsistensi pencatatan dengan memverifikasi bahwa mekanisme *signed payload* Ed25519 mencegah *event* palsu masuk ke rantai *audit record*. Pengujian ini menguji mitigasi terhadap ancaman T6 (pasien menyangkal pemberian akses) dan T16 (PRE Server menyangkal pengajuan sponsorship), karena *event* yang masuk ke ALS harus dapat dibuktikan berasal dari komponen pengirim yang sah.

Pengujian pertama, yaitu P-KNF-06, dijalankan melalui unit testing pada fungsi `Utils::verify_and_extract_event`. Skenario pengujian membuat sebuah `SignedEvent` yang valid dengan *payload*, *signature*, dan *public key* yang sesuai, kemudian memanggil `verify_and_extract_event`. Fungsi harus mengembalikan `AuditEvent` yang valid tanpa *error*.

Pengujian kedua, yaitu P-KNF-07, dijalankan dengan skenario serupa tetapi *payload* dimodifikasi setelah penandatanganan. Satu karakter pada *payload string* diubah sebelum konstruksi `SignedEvent`. Fungsi `verify_and_extract_event` harus mengembalikan *error* dengan pesan yang mengindikasikan kegagalan verifikasi tanda tangan digital.

Pengujian ketiga, yaitu P-KNF-08, dijalankan dengan skenario di mana *signature* diganti dengan *signature* yang dihasilkan oleh pasangan kunci Ed25519 yang berbeda dari yang digunakan untuk menandatangani *payload*. Fungsi `verify_and_extract_event` harus mengembalikan *error* yang mengindikasikan tanda tangan tidak valid atau *public key* tidak cocok.

Hasil pengujian menunjukkan P-KNF-06 berhasil mengekstrak `AuditEvent` yang valid, sedangkan P-KNF-07 dan P-KNF-08 keduanya menghasilkan *error* dengan pesan “Digital signature tidak valid!”. Mekanisme verifikasi Ed25519 menggunakan ed25519-dalek terbukti efektif mencegah *event* yang dimanipulasi atau berasal dari sumber tidak sah masuk ke sistem pencatatan.

IV.4 Evaluasi

Evaluasi dilakukan dengan membandingkan hasil implementasi dan pengujian terhadap kebutuhan sistem. Hasil evaluasi digunakan untuk menilai apakah pende-

katan *Centralized Audit Collector* dengan *signed payload* Ed25519, *hash-chaining*, pengarsipan IPFS, dan publikasi metadata *on-chain* pada IOTA mampu menjawab keterbatasan mekanisme pencatatan pada DecMed awal yang telah diidentifikasi pada Subbab III.1.1.

Berdasarkan hasil pengujian kebutuhan fungsional dan nonfungsional, kebutuhan sistem yang diuji pada Tugas Akhir ini dinyatakan terpenuhi dalam batas skenario uji dan batasan prototipe yang ditetapkan.

IV.4.1 Evaluasi terhadap Kebutuhan Sistem

Berdasarkan kebutuhan sistem pada Subbab IV.2.2 hingga III.2.7, implementasi dievaluasi terhadap kebutuhan fungsional dan nonfungsional yang menjadi dasar pengembangan ALS. Setiap kebutuhan diperiksa berdasarkan komponen yang merealisasikannya dan skenario pengujian yang memverifikasi perilakunya. Pemetaan ini menunjukkan keterhubungan antara kebutuhan, implementasi, dan pengujian.

Ringkasan pemetaan kebutuhan sistem terhadap implementasi dan pengujian disajikan pada Tabel IV.4.

Tabel IV.4. Evaluasi Pemenuhan Kebutuhan Sistem *Audit Log*

ID Kebutuhan	Komponen Implementasi	Skenario Pengujian	Status
KF-TA-01	<i>Endpoint</i> POST <i>/api/events</i> , AuditLogger, antrian <i>tokio::sync::mpsc</i>	P-KF-01	Terpenuhi
KF-TA-02	AuditEventDetails <i>tagged enum</i> (EV1–EV13)	P-KF-02	Terpenuhi
KF-TA-03	AuditRecord dengan <i>#[serde(flatten)]</i>	P-KF-03	Terpenuhi

ID Kebutuhan	Komponen Implementasi	Skenario Pengujian	Status
KF-TA-04	write_audit_record dengan mode <i>append</i>	P-KF-04	Terpenuhi
KF-TA-05	Log Rotation Worker, add_file_to_ipfs, publish_metadata, freeze_object	P-KF-05, P-KNF-04, P-KNF-05	Terpenuhi
KNF-TA-01	create_audit_record (<i>hash-chaining</i>), verify_record, object on-chain	P-KNF-01, P-KNF-02, P-KNF-05	Terpenuhi
KNF-TA-02	Format JSON-Lines, #[serde(flatten)], skema AuditRecord	P-KNF-03	Terpenuhi
KNF-TA-03	IPFS <i>pinning</i> , LogRecord sebagai <i>object immutable</i> di IOTA	P-KNF-04, P-KNF-05	Terpenuhi
KNF-TA-04	Verifikasi Ed25519 pada verify_and_extract_event, antrian asinkron berkapasitas 10.000	P-KNF-06, P-KNF-07, P-KNF-08	Terpenuhi

Tabel IV.4 menunjukkan bahwa seluruh kebutuhan fungsional (KF-TA-01 hingga KF-TA-05) dan kebutuhan nonfungsional (KNF-TA-01 hingga KNF-TA-04) telah terpenuhi berdasarkan skenario pengujian yang dijalankan. Implementasi *Centralized Audit Collector* dengan pengiriman *asynchronous push-based*, standarisasi

skema `AuditRecord`, *hash-chaining* antar *record*, rotasi dan pengarsipan berkas log ke IPFS, serta publikasi metadata *immutable* ke IOTA melalui *smart contract* Move memenuhi kebutuhan yang diuji.

Kebutuhan fungsional diverifikasi melalui skenario P-KF-01 sampai P-KF-05, sedangkan kebutuhan nonfungsional diverifikasi melalui pengujian P-KNF-01 sampai P-KNF-08.

Evaluasi terhadap masalah-masalah yang diidentifikasi pada Tabel III.1 menghasilkan temuan sebagai berikut. Masalah M-TA-01 (belum adanya sistem *audit log* terintegrasi) diselesaikan melalui penambahan ALS sebagai *centralized audit collector* dengan titik masuk tunggal `POST /api/events`, diverifikasi oleh P-KF-01. Masalah M-TA-02 (cakupan *event* yang sempit) diselesaikan melalui katalog 13 jenis *event* (EV1–EV13) yang diturunkan dari analisis ancaman STRIDE, diverifikasi oleh P-KF-02. Masalah M-TA-03 (*audit record* yang masih dapat dimodifikasi) diselesaikan melalui mekanisme *hash-chaining* pada berkas log lokal dan penyimpanan pada *on-chain* melalui *smart contract* Move, diverifikasi oleh P-KNF-01, P-KNF-02, dan P-KNF-05. Masalah M-TA-04 (format *audit record* yang tidak terstandarisasi) diselesaikan melalui skema `AuditRecord` yang seragam dengan format JSON-Lines, diverifikasi oleh P-KNF-03. Masalah M-TA-05 (kerentanan terhadap *data pruning*) diselesaikan melalui strategi persistensi ganda berbasis *IPFS pinning* dan penyimpanan *state object* Move, diverifikasi oleh P-KNF-04 dan P-KNF-05.

IV.4.2 Keterbatasan Implementasi

Implementasi pada Tugas Akhir ini memiliki beberapa keterbatasan sebagai berikut.

1. Implementasi dilakukan dalam lingkungan prototipe yang terdiri atas komputer lokal, VPS, dan jaringan IOTA Localnet. Seluruh komponen ALS dan PRE Server dijalankan pada komputer lokal dengan sistem operasi Ubuntu 24.04.4 di lingkungan WSL2. VPS menjalankan IPFS Cluster dan IOTA Gas Station. *Smart contract* `als::audit_log` diunggah ke jaringan IOTA Localnet, bukan jaringan IOTA Testnet maupun Mainnet, sehingga perilaku sistem pada jaringan publik yang sesungguhnya belum dapat diverifikasi.
2. Mekanisme *IPFS pinning* yang diimplementasikan bergantung pada konfigurasi IPFS Cluster yang diwarisi dari implementasi DecMed sebelumnya dengan tiga *peer*. Ketahanan *pinning* terhadap kegagalan *node* pada kluster

berskala produksi belum diuji.

3. Nilai LOG_ROTATION_INTERVAL_SECS yang digunakan dalam pengujian (60 detik) berbeda dari nilai yang direkomendasikan untuk produksi (300 detik sesuai konstanta). Implikasi rotasi yang terlalu sering terhadap konsumsi *gas* IOTA dan kuota transaksi Gas Station belum dievaluasi secara menyeluruh.
4. Kunci privat Ed25519 yang digunakan oleh setiap *event source* untuk menandatangani *audit event* dibangkitkan secara acak pada saat komponen diinisialisasi dan tidak dipersistensikan. Apabila komponen di-*restart*, kunci baru akan dibangkitkan sehingga tidak ada mekanisme untuk menghubungkan *event* dari sesi sebelumnya dengan sesi baru secara kriptografis. Pengelolaan kunci jangka panjang untuk keperluan *non-repudiation* yang sesungguhnya memerlukan mekanisme penyimpanan kunci yang aman dan persisten.
5. Pengujian verifikasi integritas rantai *audit record* dilakukan pada skala kecil (puluhan *record*). Performa verifikasi rantai pada volume berkas log yang besar (ribuan hingga jutaan *record*) belum dievaluasi.

Keterbatasan tersebut menunjukkan bahwa implementasi masih berada pada tingkat pembuktian konsep (*proof of concept*). Pengembangan berikutnya perlu menguji sistem pada jaringan IOTA Testnet, memperkuat mekanisme pengelolaan kunci Ed25519 untuk mendukung *non-repudiation* jangka panjang, serta mengevaluasi skalabilitas *hash-chain* verification pada volume log produksi.

```

1  // Step 1: Serialize AuditEvent ke bytes JSON
2  let payload_bytes = serde_json::to_vec(&event)?;
3
4  // Step 2: Enkripsi payload dengan AES-256-GCM (kunci sesi acak)
5  let encrypted = aes_encrypt(&payload_bytes)?;
6
7  // Step 3: Bungkus kunci AES dengan public key ALS menggunakan
   ↳ ECIES P-256
8  let enc_aes_key = ecies_encrypt_key(&encrypted.key,
   ↳ ALS_SERVER_PUBLIC_KEY)?;
9
10 // Step 4: Tandatangani ciphertext dengan IotaKeyPair
   ↳ (Encrypt-then-Sign)
11 let intent_msg = IntentMessage::new(
12     Intent::personal_message(),
13     encrypted.ciphertext.clone(),
14 );
15 let signature = IotaSignature::new_secure(&intent_msg,
   ↳ iota_key_pair);
16
17 // Step 5: Bangun EncryptedSignedEvent
18 let signed_event = EncryptedSignedEvent {
19     enc_aes_key,
20     ciphertext: STANDARD.encode(&encrypted.ciphertext),
21     nonce: STANDARD.encode(&encrypted.nonce),
22     signature: signature.encode_base64(),
23     iota_address: iota_address.clone(),
24 };
25
26 // Step 6: Simpan ke AlsQueue terlebih dahulu (durabilitas)
27 AlsQueue::push(new_queue_entry(signed_json.clone(), label)).await?;
28
29 // Step 7: Coba kirim langsung; jika gagal, retry worker akan
   ↳ menangani
30 request::Client::new()
31     .post(ALS_ENDPOINT)
32     .header("Content-Type", "application/json")
33     .body(signed_json)
34     .send()
35     .await;

```

Kode 8. Alur enkapsulasi *event* pada `ALSClient::build_and_send`

BAB V

PENUTUP

V.1 Kesimpulan

Penelitian ini menyimpulkan dua hal berikut.

1. Kebutuhan *audit event* pada sistem DecMed ditentukan melalui analisis ancaman menggunakan metode STRIDE terhadap arsitektur sistem yang telah didekomposisi. Dari dekomposisi sistem yang mencakup identifikasi dependensi eksternal, titik masuk, aset, dan level kepercayaan, dihasilkan *Data Flow Diagram* yang menjadi basis identifikasi ancaman. Analisis STRIDE menghasilkan 25 ancaman (T1–T25) yang mencakup seluruh enam kategori STRIDE: *Spoofing*, *Tampering*, *Repudiation*, *Information Disclosure*, *Denial of Service*, dan *Elevation of Privilege*. Pemetaan ancaman tersebut menghasilkan 13 jenis *audit event* (EV1–EV13) yang perlu dicatat beserta atribut bukti spesifik yang dibutuhkan untuk keperluan investigasi dan mitigasi, sehingga cakupan pencatatan ditentukan berdasarkan risiko yang terukur.
2. Layanan *audit log* yang terintegrasi dengan arsitektur DecMed dirancang dan diimplementasikan sebagai komponen *Audit Log Service* (ALS) yang berdiri sendiri. ALS mengadopsi pola *centralized audit collector* dengan pengiriman *event* secara *asynchronous push-based* dari empat *event source*, yaitu *Patient Client*, *Hospital Client*, *Ministry Client*, dan PRE Server. Setiap *event* dibungkus dalam format *signed payload* Ed25519 untuk menjamin keaslian dan *non-repudiation*. Integritas berkas log lokal dijamin melalui mekanisme *hash-chaining* antar *audit record*, di mana setiap *record* menyertakan nilai *hash* SHA-256 dari *record* sebelumnya. Retensi jangka panjang diwujudkan melalui rotasi log periodik, pengarsipan berkas log ke IPFS Cluster dengan *pinning*, dan publikasi metadata rotasi pada *smart contract* Move di jaringan IOTA melalui Gas Station Server yang sudah tersedia pada DecMed. ALS tidak mengubah alur bisnis maupun fungsionalitas utama DecMed yang telah berjalan, melainkan memanfaatkan ulang infrastruktur IPFS dan IOTA yang sudah ada.

V.2 Saran

Berdasarkan batasan dan hasil evaluasi Tugas Akhir ini, beberapa saran untuk pengembangan berikutnya adalah sebagai berikut.

1. Mekanisme pengelolaan kunci Ed25519 pada setiap *event source* perlu diperkuat. Pada implementasi saat ini, pasangan kunci dibangkitkan secara acak setiap kali komponen diinisialisasi tanpa persistensi, sehingga tidak ada kontinuitas kriptografis antar-sesi. Pengembangan berikutnya dapat mengintegrasikan penyimpanan kunci yang aman dan persisten, misalnya berbasis *secure enclave* atau modul HSM, agar *non-repudiation* dapat ditegakkan secara penuh dalam jangka panjang.
2. Pembacaan dan analisis *audit log* perlu dikembangkan sebagai komponen tersendiri. Implementasi saat ini hanya menyediakan *endpoint* GET `/api/logs` untuk membaca metadata rotasi log *on-chain*, tetapi belum menyediakan fasilitas pencarian, penyaringan, maupun korelasi *audit record* berdasarkan atribut seperti `actor_id`, `target_object`, rentang waktu, atau jenis *event*. Fasilitas tersebut dibutuhkan agar investigasi insiden keamanan dapat dilakukan secara efektif.
3. Cakupan *audit event* dapat diperluas seiring berkembangnya fungsionalitas DecMed, misalnya dengan menambahkan jenis *event* untuk skenario akses darurat (*break-glass access*), integrasi dengan standar interoperabilitas data kesehatan seperti SATUSEHAT atau FHIR, maupun pencatatan aktivitas pada komponen Gas Station Server yang saat ini diperlakukan sebagai infrastruktur eksternal di luar cakupan ALS.
4. Pengujian keamanan lebih lanjut terhadap ALS itu sendiri, termasuk pengujian penetrasi terhadap *endpoint* penerimaan *event* dan analisis ketahanan mekanisme *hash-chaining* terhadap serangan yang terkoordinasi, dapat dijadikan penelitian lanjutan untuk meningkatkan kepercayaan terhadap sistem *audit log* yang dihasilkan.

DAFTAR PUSTAKA

- Benet, J. (2014). IPFS - Content Addressed, Versioned, P2P File System. <https://arxiv.org/abs/1407.3561>
- International Organization for Standardization. (2021). ISO 27789:2021 – Health Informatics – Audit Trails for Electronic Health Records (Second edition). <https://www.iso.org/standard/76028.html>
- International Organization for Standardization. (2022). ISO/IEC 27002:2022 – Information Security, Cybersecurity and Privacy Protection – Information Security Controls [Control 8.15: Logging]. <https://www.iso.org/standard/75652.html>
- Peraturan Menteri Kesehatan Republik Indonesia Nomor 24 Tahun 2022 tentang Rekam Medis (2022). <https://peraturan.bpk.go.id/Details/245544/permenkes-no-24-tahun-2022>
- Kent, K., & Souppaya, M. (2006). *Guide to Computer Security Log Management* (techreport **number** NIST Special Publication 800-92). National Institute of Standards dan Technology. <https://doi.org/10.6028/NIST.SP.800-92>
- Laksono, A. C., & Prayudi, Y. (2021). Threat Modeling Menggunakan Pendekatan STRIDE dan DREAD untuk Mengetahui Risiko dan Mitigasi Keamanan pada Sistem Informasi Akademik. *JUSTINDO (Jurnal Sistem dan Teknologi Informasi Indonesia)*, 6(1), 9–20. <https://doi.org/10.32528/justindo.v6i1.3944>
- National Institute of Standards and Technology. (2020). *Security and Privacy Controls for Information Systems and Organizations* (techreport **number** NIST Special Publication 800-53, Revision 5). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-53r5>
- OWASP Foundation. (2026). *Threat Modeling Process*. Diakses pada Juni 7, 2026, dari https://owasp.org/www-community/Threat_Modeling_Process
- Undang-Undang Republik Indonesia Nomor 19 Tahun 2016 tentang Perubahan atas Undang-Undang Nomor 11 Tahun 2008 tentang Informasi dan Transaksi Elektronik (2016). <https://peraturan.bpk.go.id/Details/37582/uu-no-19-tahun-2016>

- Pramesti, D. P. A., Ayuningtyas, D., & Verdi, R. (2024). Keamanan dan Kerahasiaan Data Medis Pasien dalam Implementasi Rekam Medis Elektronik: Tinjauan Sistematis. *PREPOTIF: Jurnal Kesehatan Masyarakat*, 8(3), 7691–7702. <https://doi.org/10.31004/prepotif.v8i3.38445>
- Purnama, H., Sudewa, I. P. B. H., Nizami, T., Rosyada, B. S., Mahesa, P. R., & Ahmadi, N. (2026). Access Control Development Within the Framework of an IOTA-Based Electronic Medical Record Management System. *Sensors*, 26(5), 1422.
- Shostack, A. (2014). *Threat modeling: Designing for security*. John wiley & sons.
- Theis, M., Trzeciak, R. F., Costa, D. L., Moore, A. P., Miller, S., Cassidy, T., & Clacycomb, W. R. (2022). *Common Sense Guide to Mitigating Insider Threats* (techreport) (7th edition). CERT Division, Software Engineering Institute, Carnegie Mellon University.

Lampiran A. Dekomposisi Aplikasi DecMed

Tabel A.1 sampai Tabel A.4 menjabarkan dekomposisi aplikasi decmed.

Tabel A.1. Dependensi Eksternal Sistem DecMed

ID	Komponen	Deskripsi
DE-01	IOTA Rebased	DLT berbasis Directed Acyclic Graph (DAG) dengan konsensus Mystici (Byzantine Fault Tolerant, Delegated Proof-of-Stake), finalitas transaksi ~500 ms, mendukung lebih dari 50.000 TPS.
DE-02	Smart Contract Move	<i>package</i> decmed yang dipublikasikan pada IOTA Layer 1, terdiri atas modul <code>patient</code> , <code>admin</code> , <code>hospital_personnel</code> , <code>proxy</code> , <code>shared</code> , serta modul struktur data <code>std_struct_*</code> dan <code>std_enum_*</code> .
DE-03	PRE Server	diimplementasikan dengan Rust dan <i>framework</i> Axum, menyediakan endpoint REST untuk proses autentikasi, pertukaran kunci, dan operasi RME.
DE-04	IOTA Gas Station	terdiri atas Gas Station Server (endpoint reservasi dan eksekusi <i>gas object</i>), Gas Station Storage (berbasis Redis dengan skrip LUA untuk akses konkuren), dan Key Store Manager/KSM (KMS eksternal atau penyimpanan kunci <i>in-memory</i>).
DE-05	Redis	digunakan oleh PRE Server untuk menyimpan data sementara seperti <i>nonce</i> autentikasi (TTL 3 menit) dan material PRE seperti <i>kfrag</i> (TTL 2 jam); juga digunakan oleh Gas Station sebagai <i>storage gas object</i> .

ID	Komponen	Deskripsi
DE-06	IPFS Cluster	penyimpanan <i>off-chain</i> terdesentralisasi berbasis <i>content-addressing</i> (CID) untuk data klinis pasien yang telah dienkripsi, digunakan karena ukuran data klinis dapat melebihi batas objek IOTA (250 KB).
DE-07	Client Application Framework	Tauri (Rust + SvelteKit/TypeScript), dengan tiga varian: Patient Client (mobile), Hospital Client (desktop), dan Ministry Client (desktop).
DE-08	Skema BIP39	standar <i>mnemonic</i> 12 kata untuk pembuatan dan pemulihan kunci kriptografis pasien dan tenaga fasyankes secara <i>non-custodial</i> .
DE-09	Mekanisme kriptografis Proxy Re-Encryption (PRE) dan AES-GCM	digunakan untuk enkripsi data administratif dan klinis serta proses re-enkripsi <i>ciphertext</i> bagi penerima yang berwenang.

Tabel A.2. Titik Masuk Sistem DecMed

ID	Nama	Deskripsi	Level Kepercayaan
TM-01	POST /nonce	Endpoint PRE Server untuk memperoleh <i>nonce</i> autentikasi 64-byte, divalidasi melalui <i>Move call is_account_registered</i> .	LK1–LK2

ID	Nama	Deskripsi	Level Kepercayaan
TM-02	POST /keys	Endpoint PRE Server untuk menyimpan material PRE (<i>kfrag</i> , kunci publik) dan menerbitkan token akses JWT bagi tenaga fasyan-kes.	LK2
TM-03	GET /medical-record	Endpoint PRE Server untuk mengambil metadata administrasi dan klinis pasien beserta proses re-enkripsi, memerlukan token akses <i>read</i> yang valid.	LK4
TM-04	POST /medical-record	Endpoint PRE Server untuk membuat entri RME klinis baru, memerlukan token akses <i>update</i> .	LK4
TM-05	PUT /medical-record	Endpoint PRE Server untuk mengubah entri RME klinis yang sudah ada pada kunjungan yang sama.	LK4
TM-06	GET /medical-record/update	Endpoint PRE Server untuk mengambil metadata RME sebelum proses pengubahan (edit).	LK4
TM-07	GET /administrative	Endpoint PRE Server untuk mengambil metadata administrasi pasien, dapat diakses oleh admin fasyankes maupun tenaga administratif.	LK3, LK5

ID	Nama	Deskripsi	Level Kepercayaan
TM-08	Modul decmed::patient	Kumpulan <i>method on-chain</i> yang dapat dieksekusi pasien: signup, create_access, revoke_access, update_administrative_metadata, get_medical_record(s), get_access_log, dan sejenisnya.	LK1–LK2
TM-09	Modul decmed::hospital_personnel	Kumpulan <i>method on-chain</i> bagi tenaga fasyankes: signup, use_activation_key, create_activation_key, cleanup_read/update_access, get_read/update_access, dan sejenisnya.	LK3–LK5
TM-10	Modul decmed::admin	Kumpulan <i>method on-chain</i> bagi Kementerian Kesehatan: create_activation_key (registrasi fasyankes), update_activation_key, get_hospitals.	LK6

ID	Nama	Deskripsi	Level Kepercayaan
TM-11	Modul decmed::proxy	Kumpulan <i>method on-chain</i> yang dieksekusi oleh PRE Server: <i>create_capability</i> , <i>create/update_medical_record</i> , <i>get_administrative_data</i> , <i>get_hospital_personnel_role</i> , dan sejenisnya.	LK7
TM-12	Gas Station endpoint (<i>reserve & execute</i>)	Dua endpoint utama Gas Station Server untuk men-cadangkan objek <i>gas</i> dan mengeksekusi transaksi ter-sponsori atas nama <i>client</i> .	LK7, LK8
TM-13	QR Code Delegasi Akses	Antarmuka fisik/visual berisi <i>iota_address</i> dan <i>pre_public_key</i> tenaga medis, dipindai oleh Patient Client untuk memulai proses delegasi akses.	LK2, LK4
TM-14	Sign-in lokal (PIN/biometrik)	Mekanisme autentikasi harian pasien/tenaga fasyan-kes menggunakan PIN atau biometrik terhadap kunci privat yang tersimpan ter-enkripsi di perangkat.	LK1–LK5

Tabel A.3. Aset Sistem DecMed

ID	Nama	Deskripsi
A-01	Data administratif RME	Identitas dan data sosial pasien (NIK, nama, dsb.), terenkripsi AES-GCM sebelum disimpan <i>on-chain</i> .
A-02	Data klinis RME	Data pemeriksaan, diagnosis, dan tindakan medis, terenkripsi AES-GCM dan disimpan pada IPFS sebagai <code>enc_medical_data</code> .
A-03	Entri <i>capability</i> akses (<i>on-chain</i>)	Struktur data <code>hospital_personnel_access_data</code> berisi <code>access_data_types</code> , <code>exp</code> (durasi kedaluwarsa), <code>metadata</code> , dan <code>medical_metadata_index</code> – fondasi model CapBAC.
A-04	<i>Recovery phrase</i> (BIP39) & kunci privat pasien/personel	<i>Mnemonic</i> 12 kata dan kunci privat turunannya, disimpan terenkripsi pada perangkat menggunakan <i>secure hardware</i> .
A-05	Material PRE (<code>kfrag</code> , <code>cfrag</code> , <i>pre_secret/public_key</i>)	Komponen kriptografis yang memungkinkan proses re-enkripsi data pasien kepada penerima yang berwenang.
A-06	<i>Nonce</i> autentikasi	Nilai acak 64-byte, TTL 3 menit, disimpan di Redis untuk mencegah serangan <i>replay</i> pada proses autentikasi pasien.
A-07	Token akses JWT	Token dengan klaim <i>role</i> dan <i>access type</i> (Read/Update); durasi berbeda antara tenaga medis (15 menit baca, 2 jam tulis) dan tenaga administratif.
A-08	<i>Activation key</i>	Kredensial sekali pakai yang diterbitkan Kementerian Kesehatan (untuk admin fasyankes) dan admin fasyankes (untuk personel), berfungsi sebagai <i>cryptographic witness</i> atas tindakan yang diotorisasi.

ID	Nama	Deskripsi
A-09	GlobalAdminCap & ProxyCapEnum	Representasi <i>capability</i> tertinggi pada sistem – masing-masing untuk Kementerian Kesehatan dan PRE Server; kompromi terhadap aset ini setara dengan eskalasi privilese total.
A-10	CID data klinis (IPFS)	<i>Content Identifier</i> yang menjadi referensi lokasi dan <i>fingerprint</i> integritas data klinis terenkripsi pada IPFS.
A-11	<i>Gas object</i> & kunci penandatanganan Gas Station	Token IOTA yang dipecah menjadi objek <i>gas</i> untuk mensponsori transaksi, beserta kunci penandatanganan yang dikelola KSM.
A-12	Log akses RME	Catatan riwayat akses terhadap data RME pasien, dapat dilihat pasien melalui <code>get_access_log</code> .

Tabel A.4. Level Kepercayaan Sistem DecMed

ID	Nama	Deskripsi
LK1	Aktor belum teraktivasi/terregistrasi	Pengguna yang belum menyelesaikan proses <i>signup</i> atau aktivasi <i>client</i> ; tidak memiliki <i>capability</i> apa pun pada sistem.
LK2	Pasien (teraktivasi)	Pasien yang telah teregistrasi; memiliki <i>capability</i> membaca RME sendiri, memberi dan mencabut akses, serta melihat log akses.
LK3	Administrative Personnel (dengan <i>capability</i> aktif)	Tenaga administratif fasyankes yang telah diberi akses oleh pasien; <i>scope</i> akses terbatas pada data administratif saja.
LK4	Medical Personnel (dengan <i>capability</i> aktif)	Tenaga medis fasyankes yang telah diberi akses oleh pasien; <i>scope</i> akses mencakup data administratif dan klinis, termasuk hak tulis (tambah/ubah) data klinis.

ID	Nama	Deskripsi
LK5	Hospital Admin	Administrator fasyankes; berwenang menambah personel dan menerbitkan <i>activation key</i> pada lingkup fasyankesnya.
LK6	Ministry of Health	Administrator Kementerian Kesehatan; berwenang meregistrasi fasyankes baru beserta adminnya, otoritas tertinggi di antara aktor manusia.
LK7	PRE Server (Proxy)	Proses <i>backend</i> dengan <i>capability</i> ProxyCapEnum; berwenang memverifikasi entri <i>capability on-chain</i> dan melakukan re-enkripsi data.
LK8	Gas Station Server	Proses <i>backend</i> yang mensponsori dan mengeksekusi transaksi atas nama <i>client</i> , memiliki akses terhadap kunci penandatanganan (KSM).
LK9	IOTA Network Validator	Node jaringan yang berpartisipasi dalam konsensus Mysticeti untuk memvalidasi dan memfinalisasi transaksi pada ledger IOTA.

Catatan: LK3 dan LK4 bersifat *parallel*, bukan hierarkis – keduanya diberikan *capability* oleh pasien secara independen dengan cakupan data yang berbeda (administratif saja vs. administratif dan klinis), sehingga tidak dapat diasumsikan LK4 selalu memiliki privilese lebih tinggi dari LK3 di seluruh konteks ancaman.

Lampiran B. Hasil Analisis Ancaman STRIDE

Tabel B.1. Identifikasi Ancaman Sistem DecMed

ID	Deskripsi Ancaman	STRIDE
T1	Penyerang mendapatkan kredensial atau membajak sesi lokal milik pengguna untuk berpura-pura menjadi pemilik identitas sah saat <i>sign-in</i> .	S
T2	Kredensial <i>sign-in</i> (PIN) tersadap atau terekam melalui <i>shoulder surfing</i> , <i>keylogger</i> , atau malware pada perangkat saat dimasukkan ke Client.	I
T3	Percobaan <i>sign-in</i> berulang secara masif menghabiskan sumber daya Client (<i>local resource exhaustion</i>).	D
T4	Payload QR Access Delegation dipalsukan sebelum dipindai oleh Patient Client.	S
T5	Payload QR Access Delegation disadap sehingga informasi delegasi akses bocor kepada pihak tidak berwenang.	I
T6	Pasien menyangkal pernah memberikan akses (<i>create_access</i>) kepada tenaga medis tertentu.	R
T7	Tenaga medis atau tenaga administratif menyangkal pernah mengakses maupun mengubah data pasien tertentu.	R
T8	Admin fasyankes menyangkal pernah menerbitkan <i>activation key</i> kepada personel tertentu.	R
T9	Ministry Administrator menyangkal pernah meregistrasi fasyankes tertentu beserta admin-nya.	R
T10	Penyerang menyamar sebagai Proxy Re-encryption API (P4) yang sah (<i>man-in-the-middle</i>) saat Client mengirim permintaan.	S
T11	Penyerang menyadap komunikasi Client dengan PRE Server, mengungkap data sensitif seperti token JWT maupun data administratif.	I
T12	Pihak eksternal membanjiri <i>endpoint</i> dengan permintaan palsu untuk menghabiskan sumber daya PRE Server.	D

ID	Deskripsi Ancaman	STRIDE
T13	Token JWT dipalsukan atau diputar ulang (<i>replay</i>) pada permintaan dari Client ke PRE Server.	S
T14	Penyerang memodifikasi <i>Move transaction</i> pada perjalanan dari IOTA Transaction Client menuju IOTA Network.	T
T15	Penyerang menyamar sebagai IOTA Transaction Client (P8) yang sah saat berinteraksi dengan Gas Station untuk mengajukan sponsorship transaksi palsu.	S
T16	PRE Server (P8) menyangkal telah mengajukan atau menerima keputusan sponsorship transaksi tertentu dari Gas Station.	R
T17	Penyerang mengajukan permintaan sponsorship <i>gas</i> secara berulang sehingga <i>gas object</i> habis dan transaksi sah pengguna lain gagal tersponsori.	D
T18	Pihak dengan akses tidak sah ke Redis memodifikasi nilai <i>nonce</i> autentikasi atau <i>kfrag</i> sebelum digunakan.	T
T19	Pihak dengan akses tidak sah ke Redis membaca <i>kfrag</i> atau <i>nonce</i> sebelum masa berlakunya (TTL) berakhir.	I
T20	Redis dipenuhi (<i>memory exhaustion</i>) melalui permintaan <i>nonce</i> baru secara berulang sehingga proses autentikasi pengguna sah terganggu.	D
T21	Data klinis terenkripsi diunduh dari IPFS Cluster Storage oleh pihak tidak berwenang.	I
T22	IPFS Cluster dibanjiri permintaan berukuran besar sehingga mengganggu ketersediaan data klinis bagi pengguna sah.	D
T23	Metadata <i>on-chain</i> yang tersimpan pada IOTA On-Chain State bersifat publik sehingga pola transaksi berpotensi dianalisis untuk mengungkap informasi tidak langsung mengenai pasien.	I
T24	Kelemahan validasi <i>capability on-chain</i> memungkinkan pembacaan atau pembuatan ProxyCapEnum/GlobalAdminCap tanpa otorisasi sah, setara privilese tertinggi PRE Server/Kementerian Kesehatan.	E

ID	Deskripsi Ancaman	STRIDE
T25	Kegagalan validasi otorisasi pada Authentication & Access Control memungkinkan Medical Record Management memproses permintaan dengan <i>scope</i> lebih tinggi dari yang seharusnya.	E

Lampiran C. Atribut Tambahan Audit Event

Tabel C.1. Atribut Audit Tambahan untuk Setiap Audit Event

ID Event	Atribut Tambahan	Alasan
EV1	auth_method, failed_attempt_count, device_fingerprint.	Mendukung identifikasi metode autentikasi, pola kegagalan autentikasi, serta perangkat yang digunakan untuk mendeteksi percobaan autentikasi yang mencurigakan.
EV2	qr_payload_id, qr_payload, signature_valid, recipient_identity.	Mendokumentasikan payload dan keaslian QR Access Delegation serta identitas pihak yang menerima atau menggunakan delegasi akses.
EV3	capability_id, access_scope, expiry_duration, recipient_identity, transaction_digest.	Mendokumentasikan capability yang dibuat, ruang lingkup akses yang diberikan, masa berlaku, pihak penerima akses, serta transaksi yang merekam pembuatan capability.
EV4	access_type, medical_record_id, capability_id, authorization_token_id, authorization_result.	Mengidentifikasi jenis operasi terhadap rekam medis serta mekanisme otorisasi yang digunakan dan hasil pemeriksaan otorisasi.

ID Event	Atribut Tambahan	Alasan
EV5	activation_key_id, personnel_id, facility_id, transaction_digest.	Mendokumentasikan activation key yang dihasilkan, personel yang menjadi penerima, fasilitas pelayanan kesehatan terkait, serta transaksi yang merekam aktivitas tersebut.
EV6	facility_id, facility_name, facility_identifier, transaction_digest.	Mendokumentasikan identitas fasilitas pelayanan kesehatan yang diregistrasikan serta transaksi yang merekam proses registrasi.
EV7	endpoint_called, request_id, service_type, caller_component, channel_encryption.	Mengidentifikasi layanan PRE yang diminta, endpoint tujuan, komponen pemanggil, serta mekanisme pengamanan kanal komunikasi untuk mendukung investigasi terhadap penyalahgunaan layanan PRE.
EV8	transaction_digest, payload_hash, network_confirmation_status.	Menyediakan bukti transaksi yang dikirim, identitas ringkas payload yang dikirim, serta status konfirmasi transaksi pada jaringan IOTA.
EV9	transaction_digest, requested_gas_budget, sponsorship_status.	Mendokumentasikan transaksi yang meminta sponsorship, jumlah gas yang diminta, serta status pemrosesan sponsorship untuk mendukung investigasi terhadap penyalahgunaan layanan gas sponsorship.

ID Event	Atribut Tambahan	Alasan
EV10	redis_key_type, operation_type, ttl_remaining, request_origin.	Mendokumentasikan jenis objek Redis yang diakses, operasi yang dilakukan, masa berlaku objek, serta asal permintaan untuk mendeteksi perubahan, pembacaan, maupun pola akses Redis yang mencurigakan.
EV11	cid, operation_type, data_size, request_origin.	Mengidentifikasi objek IPFS yang diakses, jenis operasi, ukuran data, serta asal permintaan untuk mendukung investigasi terhadap akses tidak sah maupun permintaan dalam jumlah besar.
EV12	metadata_type, object_id, query_parameters.	Mendokumentasikan jenis metadata atau objek <i>on-chain</i> yang diakses beserta parameter yang digunakan dalam permintaan pembacaan.
EV13	capability_id, required_scope, actual_scope, validation_result, rejection_reason.	Mendokumentasikan capability yang diperiksa, ruang lingkup akses yang dipersyaratkan dan dimiliki, serta alasan penolakan apabila hasil validasi tidak memenuhi persyaratan otorisasi.

Lampiran D. Pemetaan Audit Event ke Event Source

Tabel D.1. Daftar Identifikasi *Audit Event* Sistem DecMed

ID	<i>Audit Event</i>	Deskripsi Aktivitas	<i>Source</i>
EV1	Authentication	Pencatatan setiap percobaan autentikasi pengguna ke Patient Client maupun Hospital Client, baik berhasil maupun gagal.	Patient Client, Hospital Client
EV2	QR Delegation	Pencatatan proses validasi <i>payload</i> QR Access Delegation sebelum akses diberikan kepada tenaga fasyankes.	Patient Client
EV3	Capability Issuance	Pencatatan aktivitas penerbitan dan pembuatan <i>capability</i> akses oleh pasien kepada tenaga fasyankes.	Patient Client
EV4	Medical Record Access	Pencatatan aktivitas pembacaan maupun perubahan data rekam medis.	Hospital Client
EV5	Hospital Personnel Key Generation	Pencatatan aktivitas pembuatan <i>activation key</i> untuk personel/tenaga fasyankes.	Hospital Client
EV6	Healthcare Facility Registration	Pencatatan aktivitas registrasi fasyankes beserta administratornya.	Ministry Client
EV7	PRE Service Request	Pencatatan setiap permintaan layanan yang diterima PRE Server.	PRE Server

ID	<i>Audit Event</i>	Deskripsi Aktivitas	<i>Source</i>
EV8	IOTA Transaction Submission	Pencatatan aktivitas pembuatan, penandatanganan, dan pengiriman transaksi ke jaringan IOTA.	Patient Client, Hospital Client, Ministry Client, PRE Server
EV9	Gas Sponsorship Request	Pencatatan aktivitas permintaan <i>sponsorship</i> transaksi kepada Gas Station.	Patient Client, Hospital Client, Ministry Client, PRE Server
EV10	Redis Operation	Pencatatan aktivitas penyimpanan, pembaruan, maupun operasi objek sensitif pada Redis.	PRE Server
EV11	IPFS Object Access	Pencatatan aktivitas pengunggahan maupun pengunduhan objek pada IPFS.	PRE Server
EV12	On-chain Metadata Access	Pencatatan aktivitas pembacaan metadata maupun objek pada <i>ledger</i> IOTA.	PRE Server, Patient Client
EV13	Capability Validation	Pencatatan proses validasi <i>capability</i> sebelum permintaan diproses lebih lanjut.	PRE Server

Lampiran E. Implementasi Tipe AuditEventDetails

```
1  #[derive(Debug, Clone, Serialize, Deserialize)]
2  #[serde(tag = "event_type")]
3  pub enum AuditEventDetails {
4      #[serde(rename = "EV1")]
5      Authentication {
6          auth_method: String,
7          authentication_result: String,
8          failed_attempt_count: u32,
9          device_fingerprint: String,
10     },
11     #[serde(rename = "EV2")]
12     QRDelegation {
13         qr_payload_id: String,
14         recipient_identity: String,
15         signature_valid: bool,
16     },
17     #[serde(rename = "EV3")]
18     CapabilityIssuance {
19         capability_id: String,
20         access_scope: String,
21         expiry_duration: u64,
22         transaction_digest: String,
23     },
24     #[serde(rename = "EV4")]
25     MedicalRecordAccess {
26         access_type: String,
27         medical_record_id: String,
28         capability_id: String,
29         authorization_token_id: String,
30     },
31     #[serde(rename = "EV5")]
32     HospitalPersonnelKeyGeneration {
33         facility_id: String,
34         personnel_id: String,
```

```

35         activation_key_id: String,
36     },
37     #[serde(rename = "EV6")]
38     HealthcareFacilityRegistration {
39         facility_id: String,
40         facility_name: String,
41         administrator_id: String,
42     },
43     #[serde(rename = "EV7")]
44     PREServiceRequest {
45         endpoint_called: String,
46         request_id: String,
47         caller_component: String,
48         channel_encryption: String,
49     },
50     #[serde(rename = "EV8")]
51     IotaTransactionSubmission {
52         transaction_digest: String,
53         signer_identity: String,
54         payload_hash: String,
55         network_confirmation_status: String,
56     },
57     #[serde(rename = "EV9")]
58     GasSponsorshipRequest {
59         requested_gas_budget: u64,
60         requester_id: String,
61         transaction_digest: String,
62     },
63     #[serde(rename = "EV10")]
64     RedisOperation {
65         redis_key_type: String,
66         operation_type: String,
67         process_identifier: String,
68         ttl_remaining: i64,
69     },
70     #[serde(rename = "EV11")]

```

```

71     IPFSObjectAccess {
72         cid: String,
73         operation_type: String,
74         requester_id: String,
75     },
76     #[serde(rename = "EV12")]
77     OnChainMetadataAccess {
78         onchain_object_id: String,
79         requester_id: String,
80     },
81     #[serde(rename = "EV13")]
82     CapabilityValidation {
83         capability_id: String,
84         requester_id: String,
85         validation_result: bool,
86         rejection_reason: Option<String>,
87     },
88 }

```

Kode 9. Implementasi AuditEventDetails

Lampiran F. Kode Pengujian

Lampiran ini memuat seluruh kode yang digunakan dalam pengujian nonfungsional (P-KNF-01 hingga P-KNF-08) dan pengujian integrasi (P-INT-01). Kode diorganisasi mulai dari fungsi pendukung, implementasi fungsi verifikasi yang diuji, kemudian setiap kasus pengujian secara berurutan. Seluruh kode berada pada berkas `audit-trail/src/tests.rs`, kecuali `verify_record` dan `verify_chain` yang berada pada `audit-trail/src/utils.rs`.

F.1 Fungsi Pendukung Pengujian

```

1  fn make_ev4_event() -> AuditEvent {
2      AuditEvent {
3          source_component: AuditSourceComponent::HospitalClient,
4          source_timestamp: Utc::now(),
5          event: Event {
6              actor_id: "0xabc123_tenaga_medis".to_string(),
7              actor_type: AuditActorType::MedicalPersonnel,
8              target_object_type:
9                  ↪ AuditTargetObjectType::MedicalRecord,
10             target_object: "ipfs://bafybeig_cid_rme".to_string(),
11             outcome: AuditOutcome::Success,
12             action_type: AuditActionType::Read,
13             details: AuditEventDetails::MedicalRecordAccess {
14                 patient_iota_address:
15                     ↪ "0xpatient_iota_addr".to_string(),
16                 record_index: Some(0),
17                 role_used: "MedicalPersonnel".to_string(),
18             },
19         },
20     }
21 }
22
23 fn make_ev1_event(actor: &str, outcome: AuditOutcome) -> AuditEvent
24 ↪ {
25     AuditEvent {
26         source_component: AuditSourceComponent::HospitalClient,
27         source_timestamp: Utc::now(),
28         event: Event {
29             actor_id: actor.to_string(),
30             actor_type: AuditActorType::MedicalPersonnel,
31             target_object_type: AuditTargetObjectType::AccessKeys,
32             target_object: "session".to_string(),
33             outcome,
34             action_type: AuditActionType::SignIn,
35             details: AuditEventDetails::Authentication {
36                 auth_method: "iota_signature".to_string(),
37                 role: "MedicalPersonnel".to_string(),
38                 failure_reason: None,
39             },
40         },
41     }
42 }

```

Kode 10. Fungsi pembantu pembangkit AuditEvent uji

```

1 fn build_chain(n: usize) -> Vec<AuditRecord> {
2     let mut records = Vec::new();
3     let mut prev_hash: Option<String> = None;
4
5     for i in 0..n {
6         let event = make_ev1_event(
7             &format!("actor_{i}"),
8             AuditOutcome::Success,
9         );
10        let record = create_audit_record(event, prev_hash.clone());
11        prev_hash = Some(record.record_hash.clone());
12        records.push(record);
13    }
14
15    records
16 }
17
18 fn make_signed_event(event: &AuditEvent) -> (SignedEvent,
19     ↳ SigningKey) {
20     let signing_key = SigningKey::generate(&mut OsRng);
21     let payload = serde_json::to_string(event).expect("serialize
22     ↳ event");
23     let sig = signing_key.sign(payload.as_bytes());
24
25     let signed = SignedEvent {
26         payload,
27         signature: hex::encode(sig.to_bytes()),
28         public_key:
29             ↳ hex::encode(signing_key.verifying_key().as_bytes()),
30     };
31
32     (signed, signing_key)
33 }

```

Kode 11. Fungsi pembantu pembangkit rantai *record* dan SignedEvent

```

1 // Ditambahkan ke dalam impl Utils di audit-trail/src/utils.rs
2
3 pub fn verify_record(record: &AuditRecord) -> bool {
4     let expected = Self::calculate_record_hash(
5         record.record_id,
6         record.timestamp,
7         record.prev_record_hash.clone(),
8         &record.event,
9     );
10    record.record_hash == expected
11 }
12
13 pub fn verify_chain(records: &[AuditRecord]) -> Result<(), String>
14     ↪ {
15     let mut prev_hash: Option<String> = None;
16
17     for (i, record) in records.iter().enumerate() {
18         if record.prev_record_hash != prev_hash {
19             return Err(format!(
20                 "record {}: prev_hash mismatch (expected {:?}, got
21                 ↪ {:?})",
22                 i + 1,
23                 prev_hash,
24                 record.prev_record_hash
25             ));
26         }
27
28         if !Self::verify_record(record) {
29             return Err(format!("record {}: hash mismatch", i +
30             ↪ 1));
31         }
32
33         prev_hash = Some(record.record_hash.clone());
34     }
35
36     Ok(())
37 }

```

Kode 12. Implementasi `verify_record` dan `verify_chain` pada `Utils`

```

1 #[test]
2 fn verify_record_valid() {
3     let event = make_ev4_event();
4     let record = create_audit_record(event, None);
5     assert!(
6         Utils::verify_record(&record),
7         "Record yang baru dibuat harus valid"
8     );
9 }

```

Kode 13. Pengujian P-KNF-01a: *record* valid diterima

```

1  #[test]
2  fn verify_record_tampered_actor_id() {
3      let event = make_ev4_event();
4      let mut record = create_audit_record(event, None);
5
6      // Modifikasi: ganti actor_id dengan nilai palsu
7      record.event.event.actor_id = "Oxpenyerang_palsu".to_string();
8
9      assert!(
10         !Utils::verify_record(&record),
11         "Record yang dimodifikasi harus gagal verifikasi"
12     );
13 }
14
15 #[test]
16 fn verify_record_tampered_outcome() {
17     let event = make_ev4_event();
18     let mut record = create_audit_record(event, None);
19
20     // Modifikasi: ubah outcome dari Success menjadi Failure
21     record.event.event.outcome = AuditOutcome::Failure;
22
23     assert!(
24         !Utils::verify_record(&record),
25         "Perubahan outcome harus terdeteksi"
26     );
27 }

```

Kode 14. Pengujian P-KNF-01b: modifikasi *record* terdeteksi

```

1  #[test]
2  fn verify_chain_three_records_valid() {
3      let records = build_chain(3);
4      assert!(
5         Utils::verify_chain(&records).is_ok(),
6         "Rantai tiga record yang tidak dimodifikasi harus valid"
7     );
8  }
9
10 #[test]
11 fn verify_chain_single_record_valid() {
12     let records = build_chain(1);
13     assert!(Utils::verify_chain(&records).is_ok());
14 }
15
16 #[test]
17 fn verify_chain_empty() {
18     let records: Vec<AuditRecord> = vec![];
19     assert!(Utils::verify_chain(&records).is_ok());
20 }

```

Kode 15. Pengujian P-KNF-02a: rantai *hash* valid

```

1  #[test]
2  fn verify_chain_tamper_middle_record() {
3      let mut records = build_chain(3);
4
5      // Modifikasi record ke-2: ubah actor_id
6      records[1].event.event.actor_id = "0xpenyerang".to_string();
7
8      let result = Utils::verify_chain(&records);
9      assert!(
10         result.is_err(),
11         "Perusakan pada record ke-2 harus terdeteksi"
12     );
13
14     let msg = result.unwrap_err();
15     assert!(
16         msg.contains("record 2"),
17         "Pesan error harus menyebut record 2, dapat: {msg}"
18     );
19 }
20
21 #[test]
22 fn verify_chain_tamper_first_record() {
23     let mut records = build_chain(3);
24
25     // Modifikasi record pertama
26     records[0].event.event.actor_id = "0xpenyerang".to_string();
27
28     let result = Utils::verify_chain(&records);
29     assert!(result.is_err(), "Perusakan record ke-1 harus
        ↳ terdeteksi");
30
31     let msg = result.unwrap_err();
32     assert!(
33         msg.contains("record 1") || msg.contains("record 2"),
34         "Efek domino harus terdeteksi: {msg}"
35     );
36 }

```

Kode 16. Pengujian P-KNF-02b: perusakan rantai terdeteksi

```

1  #[test]
2  fn format_roundtrip_single_record() {
3      let event = make_ev4_event();
4      let record = create_audit_record(event, None);
5
6      // Serialisasi ke JSON (simulasi satu baris JSON-Lines)
7      let json_line = serde_json::to_string(&record)
8          .expect("serialize record");
9
10     // Deserialisasi kembali ke AuditRecord
11     let decoded: AuditRecord = serde_json::from_str(&json_line)
12         .expect("deserialize record");
13
14     assert_eq!(record.record_id, decoded.record_id);
15     assert_eq!(record.record_hash, decoded.record_hash);
16     assert_eq!(
17         record.event.event.actor_id,
18         decoded.event.event.actor_id
19     );
20 }

```

Kode 17. Pengujian P-KNF-03a: *round-trip* satu record JSON-Lines

```

1  #[test]
2  fn format_roundtrip_chain_five_records() {
3      let records = build_chain(5);
4
5      // Simulasi penulisan berkas log (JSON-Lines)
6      let jsonl: String = records
7          .iter()
8          .map(|r| serde_json::to_string(r).unwrap())
9          .collect::<Vec<_>>()
10         .join("\n");
11
12     // Pembacaan kembali baris per baris
13     let decoded: Vec<AuditRecord> = jsonl
14         .lines()
15         .map(|line| serde_json::from_str(line)
16             .expect("tiap baris harus valid JSON"))
17         .collect();
18
19     assert_eq!(records.len(), decoded.len(),
20         "Jumlah record harus sama");
21
22     for (original, parsed) in records.iter().zip(decoded.iter()) {
23         assert_eq!(original.record_id, parsed.record_id);
24         assert_eq!(original.record_hash, parsed.record_hash);
25         assert_eq!(original.prev_record_hash,
26             ↪ parsed.prev_record_hash);
27     }
28
29     // Rantai yang dibaca kembali harus tetap valid
30     assert!(Utils::verify_chain(&decoded).is_ok());
31 }

```

Kode 18. Pengujian P-KNF-03b: *round-trip* lima record dan verifikasi rantai

```

1  #[test]
2  fn signed_event_valid_accepted() {
3      let event = make_ev1_event("0xactor_sah",
4          ↪ AuditOutcome::Success);
5      let (signed, _key) = make_signed_event(&event);
6
7      let result = Utils::verify_and_extract_event(&signed);
8      assert!(
9          result.is_ok(),
10         "Event dengan tanda tangan valid harus diterima: {:?}" ,
11         result.err()
12     );
13
14     let extracted = result.unwrap();
15     assert_eq!(extracted.event.actor_id, "0xactor_sah");
16 }

```

Kode 19. Pengujian P-KNF-06: SignedEvent valid diterima

```

1  #[test]
2  fn signed_event_tampered_payload_rejected() {
3      let event = make_ev1_event("Oxactor_sah",
4          ↪ AuditOutcome::Success);
5
6      let (mut signed, _key) = make_signed_event(&event);
7
8      // Modifikasi payload setelah penandatanganan
9      signed.payload.push_str("x");
10
11     let result = Utils::verify_and_extract_event(&signed);
12     assert!(
13         result.is_err(),
14         "Payload yang dimodifikasi harus ditolak"
15     );
16
17     let err = result.unwrap_err().to_string();
18     assert!(
19         err.to_lowercase().contains("signature")
20         || err.to_lowercase().contains("tanda tangan")
21         || err.to_lowercase().contains("invalid"),
22         "Pesannya error harus mengindikasikan kegagalan verifikasi:"
23         ↪ {err}"
24     );
25 }

```

Kode 20. Pengujian P-KNF-07: *payload* yang dimodifikasi ditolak

```

1  #[test]
2  fn signed_event_wrong_key_rejected() {
3      let event = make_ev1_event("0xactor_sah",
4          ↳ AuditOutcome::Success);
5      let (mut signed, _original_key) = make_signed_event(&event);
6
7      // Ganti signature dengan tanda tangan dari keypair yang
8      ↳ berbeda
9      let different_key = SigningKey::generate(&mut OsRng);
10     let wrong_sig = different_key.sign(signed.payload.as_bytes());
11     signed.signature = hex::encode(wrong_sig.to_bytes());
12     // public_key masih milik original_key → mismatch
13
14     let result = Utils::verify_and_extract_event(&signed);
15     assert!(
16         result.is_err(),
17         "Tanda tangan dari kunci berbeda harus ditolak"
18     );
19 }
20
21 #[test]
22 fn signed_event_wrong_public_key_rejected() {
23     let event = make_ev1_event("0xactor_sah",
24         ↳ AuditOutcome::Success);
25     let (mut signed, _key) = make_signed_event(&event);
26
27     // Ganti public_key dengan kunci publik acak yang tidak
28     ↳ bersesuaian
29     let impostor_key = SigningKey::generate(&mut OsRng);
30     signed.public_key =
31         hex::encode(impostor_key.verifying_key().as_bytes());
32
33     let result = Utils::verify_and_extract_event(&signed);
34     assert!(
35         result.is_err(),
36         "Public key yang tidak bersesuaian harus ditolak"
37     );
38 }

```

Kode 21. Pengujian P-KNF-08: tanda tangan dari kunci yang tidak bersesuaian ditolak

```

1  #[test]
2  fn full_cycle_ev4_chain_valid() {
3      // Langkah 1: Bangkitkan event EV4 (Medical Record Access)
4      let ev4 = make_ev4_event();
5
6      // Langkah 2: Bangun tiga record berurutan; record ke-2 adalah
7      ↪ EV4
8      let rec1 = create_audit_record(
9          make_ev1_event("actor_1", AuditOutcome::Success),
10         None,
11     );
12     let rec2 = create_audit_record(
13         ev4,
14         Some(rec1.record_hash.clone()),
15     );
16     let rec3 = create_audit_record(
17         make_ev1_event("actor_3", AuditOutcome::Success),
18         Some(rec2.record_hash.clone()),
19     );
20     // Langkah 3: Verifikasi hubungan prev_record_hash
21     ↪ antar-record
22     assert_eq!(
23         rec2.prev_record_hash,
24         Some(rec1.record_hash.clone()),
25         "prev_record_hash rec2 harus = record_hash rec1"
26     );
27     assert_eq!(
28         rec3.prev_record_hash,
29         Some(rec2.record_hash.clone()),
30         "prev_record_hash rec3 harus = record_hash rec2"
31     );

```

Kode 22. Pengujian P-INT-01 (bagian 1): pembangkitan *event* EV4 dan verifikasi hubungan rantai

```

1      // Langkah 4a: Rantai valid sebelum modifikasi
2      let chain = vec![rec1.clone(), rec2.clone(), rec3.clone()];
3      assert!(
4          Utils::verify_chain(&chain).is_ok(),
5          "Rantai tiga record harus valid sebelum modifikasi"
6      );
7
8      // Langkah 4b: Rusak record ke-2, periksa efek domino
9      let mut tampered = chain.clone();
10     tampered[1].event.event.actor_id =
11         ↪ "0xpenyerang_palsu".to_string();
12
13     let result = Utils::verify_chain(&tampered);
14     assert!(result.is_err(),
15         "Perusakan pada record ke-2 harus terdeteksi");
16
17     // Langkah 5: Bukti minimal dari record EV4
18     let ev4_record = &chain[1];
19     assert_eq!(
20         ev4_record.event.event.actor_id,
21         "0xabc123_tenaga_medis"
22     );
23     assert!(matches!(
24         ev4_record.event.event.outcome,
25         AuditOutcome::Success
26     ));
27
28     // Langkah 6: Format JSON-Lines round-trip tetap valid
29     let jsonl = chain.iter()
30         .map(|r| serde_json::to_string(r).unwrap())
31         .collect::<Vec<_>>()
32         .join("\n");
33
34     let decoded: Vec<AuditRecord> = jsonl.lines()
35         .map(|l| serde_json::from_str(l).unwrap())
36         .collect();
37
38     assert!(Utils::verify_chain(&decoded).is_ok());
39 }

```

Kode 23. Pengujian P-INT-01 (bagian 2): verifikasi integritas, deteksi perusakan, dan *round-trip* JSON-Lines

```

1  #[test]
2  fn first_record_has_null_prev_hash() {
3      let event = make_ev4_event();
4      let record = create_audit_record(event, None);
5      assert_eq!(record.prev_record_hash, None,
6                  "Record pertama dalam rantai harus punya prev_hash null");
7  }
8
9  #[test]
10 fn record_hash_is_64_char_hex() {
11     let event = make_ev4_event();
12     let record = create_audit_record(event, None);
13
14     assert_eq!(
15         record.record_hash.len(),
16         64,
17         "SHA-256 hex harus tepat 64 karakter"
18     );
19     assert!(
20         record.record_hash.chars().all(|c| c.is_ascii_hexdigit()),
21         "record_hash harus berupa karakter heksadesimal saja"
22     );
23 }

```

Kode 24. Pengujian *sanity check*: format `prev_record_hash` dan `record_hash`